



School of Technology and Maritime Industries

BEng (Hons) Electronic Engineering

Academic Year: 2025-2026

Student: Vivienne Louise Clark

Student ID: Q16203216

Title: A Satellite Communications Software-Defined Radio Transmitter Implemented on a Field Programmable Gate Array

Supervisor: Dr Nils Bausch

Date of Presentation: April 2026

ACKNOWLEDGMENTS

First and foremost, thank you to my project supervisor, Dr Nils Bausch, for your encouragement and guidance throughout this project.

My colleague at Airbus Defence and Space Ltd, Dr Mark Bowyer, was a constant source of encouragement. His support was invaluable. I would also like to thank Richard Tooth, also of Airbus Defence and Space Ltd, for his assistance in the final few weeks of the project.

Finally, thank you to my parents for your support and patience throughout my apprenticeship.

AI USE DECLARATION

AI tools were used in the following capacity during this project:

- ChatGPT was used during the literature review to identify suitable resources in Section 2.
- ChatGPT was used to check the syntax of VHDL files when Vivado was unavailable (i.e when a different computer was being used).
- ChatGPT was used to generate some of the boilerplate code structures used in the testbenches of individual IP cores. These testbenches were subsequently reviewed and adapted by the author. Information about the use of AI is documented in the header of the VHDL file if AI was involved in the development of that file.

ABSTRACT

Within the field of secure communication technologies, there has been a movement away from equipment built from many discrete components. Instead, highly integrated FPGA and SoC devices are preferred. These SoC devices, such as the AMD Zynq-7000 family, combine programmable logic with processing cores. This reduces power consumption, size and weight. Such platforms allow for rapid software-based reconfiguration of waveforms, without complex and costly hardware modification.

This report details the design, development and evaluation of a compact and affordable software-defined radio (SDR) transmitter system. This system is implemented on the *Digilent ZedBoard Zynq-7000 ARM/FPGA SoC Development Board*, and the *Analog Devices AD9361 RF Agile Transceiver*. The transmitter supports four distinct satellite communications waveforms, two of which are compatible with CCSDS standards, and another which has frequency hopping functionality. All the waveforms have distinct modulation, channel coding and framing characteristics.

Experimental validation confirmed successful waveform generation and configurability. Overall, the system capable of maintaining a stable RF transmission. This highlights the suitability of such FPGA-based SDR architectures in modern satellite communications systems.

CONTENTS

ACKNOWLEDGMENTS	ii
AI USE DECLARATION	ii
ABSTRACT	iii
FIGURES	vii
TABLES	ix
NOTATION	x
1 INTRODUCTION	1
1.1 Background	1
1.2 Project Aims and Objectives	1
2 LITERATURE REVIEW AND MARKET RESEARCH	3
2.1 Introduction	3
2.2 Software Defined Radios Within Satellite Communication Systems	3
2.3 Existing Software-Defined Radio Platforms	4
2.4 Space Communications Standards and Regulatory Frameworks	5
2.4.1 CCSDS Standards	6
2.4.2 Additional Space Communications Standards	6
2.5 Digital Modulation and Filtering	8
2.5.1 Modulation	8
2.5.2 Channel Coding	8
2.5.3 Filtering and Pulse Shaping	9
2.6 Spread Spectrum Techniques	9
2.7 Embedded SDR Platforms and Architectures	10
2.7.1 Digilent ZedBoard FPGA	10
2.7.2 AD9361 FMCOMMS3 RF Transceiver	10
2.8 Performance Metrics in Digital Communications	11
2.9 Research Gap	12
3 PROJECT METHODOLOGY, MONITORING AND CONTROL	13
3.1 Project Management and Control	13
3.1.1 Risk Management	13
3.1.2 Resource Management Considerations	15
3.1.3 Phase 1 and Phase 2 Time Management	15
3.1.4 Phase 3 Time Management	16
3.2 Project Methodology Overview	18
4 PRODUCT DESIGN SPECIFICATIONS	20
5 PRODUCT DESIGN DEVELOPMENT	23

5.1 Initial Proposal	23
5.2 Initial System Design Refinement.....	23
5.2.1 Options Analysis Overview	23
5.2.2 Detailed Options Analysis	25
5.2.3 Phase 2 Chosen Solution.....	28
5.3 Phase 3 Design Amendments	29
5.3.1 Summary of Additional Design Changes.....	29
5.3.2 Additional Options Analysis for Telemetry (Category A) Channel Coding	30
5.3.3 Phase 3 Final Chosen Waveform Configurations	31
6 IMPLEMENTATION AND TESTING	32
6.1 Custom Main Transmit Chain Design Considerations	32
6.1.1 Pseudorandom Binary Sequence Data Source.....	32
6.1.2 Modulation Mappers.....	33
6.1.3 Framers.....	34
6.1.4 Encoders	39
6.1.5 Mute Multiplexer	41
6.1.6 SRRC Filter	42
6.2 Custom IP Generation and Implementation	44
6.2.1 Summary of IP Cores Design and Implementation	44
6.2.2 Choice of AXI Protocol	45
6.2.3 Overall Implementation of AXI4-Lite Protocol.....	46
6.2.4 AXI4 Registers Function	47
6.2.5 Integration of Custom Transmit Chain into AD9361 Reference Design	47
6.3 Frequency Hopping Implementation.....	50
6.4 Software-Based User Control and Configurability	51
6.5 Testing Overview.....	52
6.5.1 Planned Testing Phases.....	52
6.5.2 Full System Evaluation Setup.....	53
7 ISSUES ARISING FROM IMPLEMENTATION AND TESTING	54
7.1 FPGA Bitstream Generation and Deployment Issues	54
7.2 Inconsistencies in AXI Block Instantiation.....	55
7.3 Timing Closure Failure During SRRC Filter Integration.....	56
7.4 AD9361 Sample Rate Mismatch.....	58
7.5 Appearance of Spurious Narrowband Emissions	60
7.6 GNU Radio and COTS Modem Symbol Rate Limitations.....	61
8 RESULTS.....	63

8.1 RF System Evaluation	63
8.1.1 Filtered and Unfiltered Spectrum Plots	63
8.1.2 Constellation and Eye Diagram Waveform Comparisons	64
8.1.3 Beacon Waveform COTS Modem Testing	67
8.2 Software Validation and Testing	69
8.2.1 Power Configurability	69
8.2.2 User-Controlled Muting Capability	71
8.2.3 Internal Muting During Waveform Switching	71
8.2.4 Waveform Switching and Identification	72
8.3 System Stability	73
8.4 Implementation Results in Vivado	74
8.4.1 Timing Results	74
8.4.2 Resource Utilisation	74
9 IMPLEMENTATION AND TESTING EVALUATION	75
9.1 Evaluation of Results Against PDS	75
9.2 Evaluation of Results Against Project Aims and Objectives	77
10 CONCLUSIONS	79
11 RECOMMENDATIONS AND FUTURE WORK	80
11.1 Data and Symbol Rate Configurability	80
11.2 Graphical User Interface	80
11.3 Improved Frequency Hopping Capability	80
REFERENCES	81
APPENDICES	87
Appendix 1: Competitive Market Analysis	87
Appendix 2: Detailed Space Communications Standards	91
CCSDS Physical Layer	91
CCSDS Data Link Layer	91
ECSS Standards	91
ITU-R Standards	92
DVB-S2 Standard	92
Appendix 3: OSI Model Reference	93
Appendix 4: Resource Requirements	94
Appendix 5: Custom IP Cores Summary	97
Appendix 6: AXI Address Map	99

FIGURES

Figure 1: CCSDS Mission Categories (CCSDS 2021).....	6
Figure 2: Risk Matrix Before Mitigation Actions.....	13
Figure 3: Risk Matrix After Mitigation Actions.....	13
Figure 4: Gantt Chart (22/09/2025 - 10/11/2025).....	15
Figure 5: Gantt Chart (17/11/2025 - 05/01/2026).....	16
Figure 6: Gantt Chart Derived at End of Phase 2 (19/01/2026 - 23/03/2026).....	16
Figure 7: Gantt Chart Derived at End of Phase 2 (30/03/2026 - 20/06/2026).....	16
Figure 8: Gantt Chart Followed During Phase 3 (29/12/2025 - 09/03/2026).....	17
Figure 9: Gantt Chart Followed During Phase 3 (16/03/2026 - 16/05/2026).....	18
Figure 10: Full Project Methodology Diagram	19
Figure 11: Initial System Design Proposal	23
Figure 12: Custom Transmit Chain Block Diagram	32
Figure 13: Galois and Fibonacci Architecture Comparison (Moria 2022).....	32
Figure 14: BPSK Expected Constellation (Roelandts n.d)	33
Figure 15: QPSK [left] and OQPSK [right] Expected Constellations (Electroagenda 2024) ..	34
Figure 16: TC Transfer Frame Structure (CCSDS 2021b)	34
Figure 17: TC Transfer Frame Header Format (CCSDS 2021b).....	35
Figure 18: Telecommand Framer FSM.....	36
Figure 19: TM Transfer Frame Structure (CCSDS 2021c).....	36
Figure 20: TM Transfer Frame Header Format (CCSDS 2021c)	37
Figure 21: TM Transfer Frame Data Field Status (CCSDS 2021c).....	37
Figure 22: Telemetry Framer FSM	39
Figure 23: BCH Codeword Structure (CCSDS 2021a).....	39
Figure 24: BCH Encoder Structure.....	40
Figure 25: BCH Encoder FSM	40
Figure 26: Convolutional Encoder Block Diagram (CCSDS 2023).....	41
Figure 27: Convolutional Encoder FSM.....	41
Figure 28: SRRC Filter Impulse Response (MathWorks 2026).....	43
Figure 29: SRRC Filter Frequency Response (MathWorks 2026)	44
Figure 30: Custom IP Blocks Within AD9361 Reference Design (AMD 2022).....	45
Figure 31: AXI Interconnect Connections (AMD 2022).....	46
Figure 32: AXI Interconnect Architecture (Esergün 2024).....	47
Figure 33: AD9361 Reference Diagram Transmit Chain	48
Figure 34: Disconnected Unpacker (AMD 2022)	48
Figure 35: Rewired DAC FIFO (AMD 2022)	49
Figure 36: DAC FIFO and axi_ad9361 Connections (AMD 2022)	50
Figure 37: Spectrum Map (Ofcom 2026).....	51
Figure 38: Full System Evaluation Setup	53
Figure 39: Failure to Connect to Board in Hardware Manager (AMD 2022).....	54
Figure 40: Use of bootgen in command line	55
Figure 41: Defective Symbol Timer Elaborated Design Schematic (AMD 2022).....	55
Figure 42: AXI Slave File Hierarchy (AMD 2022).....	56
Figure 43: Corrected Symbol Timer Elaborated Design Schematic (AMD 2022).....	56
Figure 44: Initial Vivado Timing Report (AMD 2022)	57
Figure 45: Balanced Adder Tree Structure Example (Memon et al 2013).....	58
Figure 46: Spectral Plot Sample Rate Mismatch	59

Figure 47: DAC FIFO Valid Signals Corrected Wiring (AMD 2022)	59
Figure 48: Unfiltered Waveform Spurs.....	60
Figure 49 : 3D Spectrogram	61
Figure 50: GNU Radio Symbol Rate Limitation.....	62
Figure 51: Filtered SDR Transmitter Output	63
Figure 52: Unfiltered BPSK Spectrum	64
Figure 53: Beacon Waveform Constellation and Eye Diagram.....	64
Figure 54: Telemetry (Category A) Waveform Constellation and Eye Diagram.....	65
Figure 55: Telecommand Waveform Constellation and Eye Diagram.....	65
Figure 56: FHSS Waveform Constellation and Eye Diagram in OQPSK Mode.....	66
Figure 57: FHSS Waveform Constellation and Eye Diagram in QPSK Mode.....	67
Figure 58: Beacon Eb/N0 and BER COTS Modem Measurements.....	67
Figure 59: BER vs Eb/N0 Graph for Uncoded BPSK (Splash 2007)	68
Figure 60: Simultaneous GNU Radio and COTS Modem Operation (GNU Radio Foundation 2023)	68
Figure 61: High Power Level.....	69
Figure 62: Medium Power Level	70
Figure 63: Low Power Level	70
Figure 64: User-Controlled Muting on ILA (AMD 2022)	71
Figure 65: Waveform Switch Muting on ILA (AMD 2022)	71
Figure 66: Waveform Switching and Waveform Status.....	72
Figure 67: FHSS Waveform Start and Restart	73
Figure 68: FHSS Waveform End	73
Figure 69: Beacon Spectrum Plot at 23:47	73
Figure 70: Beacon Spectrum Plot at 00:51	74
Figure 71: Vivado Timing Summary Report (AMD 2022)	74
Figure 72: Post-Implementation Resource Utilisation (AMD 2022).....	74
Figure 73: OSI Model (Mitchell 2019)	93
Figure 74: FMCOMMS2/3/4 Block Diagram (Analog Devices 2026a).....	101
Figure 75: GNU Radio Receiver (GNU Radio Foundation 2023).....	102

TABLES

Table 1: Abbreviations.....	x
Table 2: Units	xii
Table 3: Equation Notation.....	xii
Table 4: Project Aims	1
Table 5: Project Objectives	2
Table 6: SDR Comparison Summary	5
Table 7: Additional Space Communications Standards	7
Table 8: AD9361 FMCOMMS3 Key Specifications (Analog Devices 2024)	11
Table 9: Key Performance Metrics	11
Table 10: User and Market Needs	12
Table 11: Identification of Project Risks and Mitigation Actions	14
Table 12: Project Specifications (Functional Requirements).....	20
Table 13: Project Specifications (Non-Functional Requirements).....	21
Table 14: CCSDS-Compliant Waveforms Standards Summary	24
Table 15: Beacon and FHSS Waveforms Parameters Summary	24
Table 16: FHSS Implementation Methods Summary	25
Table 17: Embedded System Architectures Summary	25
Table 18: Telemetry (Category A) MCDA Matrix	26
Table 19:Telemetry (Category B) MCDA Matrix	26
Table 20: Telecommand MCDA Matrix	27
Table 21: Frequency Hopping Techniques MCDA Matrix	27
Table 22: Embedded Software Architecture MCDA Matrix	28
Table 23: Phase 2 Waveform Configuration	28
Table 24: Additional Telemetry (Category A) Channel Coding MCDA Matrix	30
Table 25: Phase 3 Waveform Configuration	31
Table 26: TC Transfer Frame Implemented Header Values	35
Table 27: TC Transfer Frame Implemented Header Values	37
Table 28: Filter Design Choices and Justification.....	42
Table 29: FHSS Hopping Table.....	50
Table 30: User Commands	51
Table 31: Test Strategy Phases.....	52
Table 32: PDS Evaluation (Functional Requirements)	75
Table 33: PDS Evaluation (Non-Functional Requirements).....	76
Table 34: Aims and Objectives Evaluation.....	77
Table 35: Existing SDR Platforms Comparison (Architecture and Use Case).....	87
Table 36: Existing SDR Platforms Comparison (Bandwidth and Cost).....	88
Table 37: Existing SDR Platform Characteristics Comparison (Timing and Power)	89
Table 38: Existing SDR Platforms Comparison (Openness and Configurability).....	90
Table 39: Required Resources (Hardware)	94
Table 40: Required Resources (Software).....	96
Table 41: Custom IP Blocks Function	97
Table 42: AXI Address Map	99

NOTATION

Table 1: Abbreviations

Abbreviation	Full Form
8PSK	8-Phase Shift Keying
ADC	Analogue-to-Digital Converter
AMD	Advanced Micro Devices
API	Application Programming Interface
ARM	Advanced Reduced Instruction Set Computer Machine
ASM	Attached Sync Marker
AXI	Advanced eXtensible Interface
BCH	Bose–Chaudhuri–Hocquenghem
BER	Bit Error Rate
BRAM	Block Random Access Memory
BPSK	Binary Phase Shift Keying
BUTG	Global Clock Buffer
CCSDS	Consultative Committee for Space Data Systems
CLI	Command Line Interface
COTS	Commercial Off-The-Shelf
CPLD	Complex Programmable Logic Device
CRC	Cyclic Redundancy Check
DAC	Digital-to-Analogue Converter
DDS	Direct Digital Synthesis
DSP	Digital Signal Processing
DSSS	Direct-Sequence Spread Spectrum
ECAD	Electronic Computer-Aided Design
ECSS	European Cooperation for Space Standardization
Eb/N0	Energy per Bit to Noise Power Spectral Density Ratio
FAT	Frequency Allocation Table
FF	Flip-Flop
FIFO	First-In, First-Out
FPGA	Field Programmable Gate Array
FHSS	Frequency Hopping Spread Spectrum
FSM	Finite State Machine
GMSK	Gaussian Minimum Shift Keying
GUI	Graphical User Interface
HDL	Hardware Design Language
I	In-Phase
IDE	Integrated Design Environment
I/O	Input/Output
IIO	Industrial Input/Output
ILA	Integrated Logic Analyser
IP	Intellectual Property
ITU-R	International Telecommunication Union Radiocommunication Sector
I/Q	In-Phase and Quadrature
LDPC	Low-Density Parity-Check

LFSR	Linear Feedback Shift Register
LO	Local Oscillator
LUT	Look-Up Table
LUTRAM	Look-Up Table Random Access Memory
MCDA	Multi-Criteria Design Analysis
MMCM	Mixed-Mode Clock Manager
NCO	Numerically Controlled Oscillator
NFR	Non-Functional Requirement
Ofcom	Office of Communications
OQPSK	Offset Quadrature Phase Shift Keying
OS	Operating System
PDS	Product Design Specification
PL	Programmable Logic
PLL	Phase-Locked Loop
PRBS	Pseudo-Random Binary Sequence
PS	Processing System
Q	Quadrature
QPSK	Quadrature Phase Shift Keying
RF	Radio Frequency
RFIC	Radio Frequency Integrated Circuit
RRC	Raised-Root Cosine
RS	Reed-Solomon
RTL	Register Transfer Level
SDR	Software-Defined Radio
SNR	Signal-to-Noise Ratio
SoC	System on Chip
SPI	Serial Peripheral Interface
SRRC	Squared-Root Raised Cosine
SSH	Secure Shell
TC	Telecommand
TM	Telemetry
UART	Universal Asynchronous Receiver-Transmitter
UKFAT	United Kingdom Frequency Allocation Table
USD	United States Dollar
USRP	Universal Software Radio Peripheral
VHDL	Very High-Speed Integrated Circuit Hardware Description Language

Table 2: Units

Unit	Full Form
μs	Microseconds
dB	Decibels
dBm	Decibel-milliwatt
GHz	Gigahertz
Hz	Hertz
kHz	Kilohertz
Mbit/s	Megabits per second
MHz	Megahertz
MSPS	Megasamples per second
Msps	Megasymbols per second
Msym/s	Megasymbols per second
%	Percent

Table 3: Equation Notation

Notation	Meaning
α	Roll-Off Factor
B	Bandwidth
G(x)	Telecommand Framer CRC Polynomial Function
g(x)	BCH Polynomial Function
N	Number of Taps
R_s	Symbol Rate
sps	Samples per Symbol

1 INTRODUCTION

1.1 Background

The advent of increasingly compact and integrated SoC architectures within the development of secure communication systems has resulted in reduced power consumption, size, cost and weight (AMD 2018, Scholl 2016, Tian *et al* 2019). These platforms now allow for software-based, rapid configuration of waveforms, without the need for complex and costly hardware modifications.

Long project lifecycles, complicated certification processes and hardware-dependent designs, often limit the adoption of the latest devices. Hence, many commercial SDR systems are tied to older programmable logic devices and are not optimised in terms of cost, efficiency and size.

Comprehensive market research identified a significant gap between the expensive mission-grade SDR systems and low-cost academic platforms. This project aims to bridge this gap by providing a user-configurable SDR transmitter with deterministic timing. This system seeks to offer a balance between configurability, performance and cost.

1.2 Project Aims and Objectives

The following tables define the aims and objectives for this project and therefore outline the overall project scope.

Table 4: Project Aims

Aim Reference	Aim Description
Aim 1	The primary aim of this project is to design, build and test a flexible FPGA-based SDR transmitter capable of supporting several distinct waveform configurations. These profiles will represent a different combination of filtering approaches, channel coding methods and types of modulation. This will allow the user, using an interface, to configure the system and emulate a range of practical space and ground segment communication scenarios.
Aim 2	The second aim is to create a transmitter able to interface with a COTS satellite modem. The modem will provide realistic link-level testing by acting as a receiver. Additionally, a laptop running the GNU Radio software will act as a configurable software-based receiver and will provide a visual interface to support the system verification.
Aim 3	Finally, the intention is to obtain a deeper understanding of both the <i>Digilent ZedBoard Zynq-7000</i> and <i>Analog Devices AD9361</i> reference designs. Additionally, the project will help to build practical experience in the use of the Vivado Integrated Development Environment, and the VHDL programming language.

Table 5: Project Objectives

Objective Reference	Objective
O1	To research existing FPGA-based SDR architectures and relevant satellite communication standards to determine the design requirements and operational constraints.
O2	To define four feasible waveform configurations suitable for implementation on the ZedBoard and AD9361 combined platform.
O3	To determine a suitable hardware-software partition between the Zynq-7000 processing system and the programmable logic.
O4	To design and simulate the full FPGA transmit chain, including channel coding, modulation, filtering and frequency-hopping control.
O5	To develop a user-friendly interface to allow intuitive selection of the waveform profiles.
O6	To validate the full system performance using both a COTS satellite modem and a GNU Radio receiver.

2 LITERATURE REVIEW AND MARKET RESEARCH

2.1 Introduction

This chapter begins with an introduction to the use of SDRs within satellite communications. This is then followed by a comprehensive review of existing SDR platforms, identifying potential market opportunities for a compact and configurable SDR system.

Relevant international standards will be reviewed. These standards will inform the selection of the framing, channel coding, modulation and filtering techniques chosen within the overall system design. The *Digilent ZedBoard* and *Analog Devices AD9361* are evaluated, with their suitability assessed and justified. Technologies underpinning configurable FPGA systems will then be explored, including the Zynq-7000 technology and spread-spectrum techniques.

Finally, methods of evaluating SDR system performance will be reviewed. The most applicable metrics are selected and used to inform the testing strategy.

2.2 Software Defined Radios Within Satellite Communication Systems

In this context, software-defined radio refers to technology where at least some of the physical-layer functions (for instance, framing, modulation or analogue-to-digital conversion) are software-defined (Viavi Solutions n.d). This allows radio-signal processing to be modified through software and reconfigurable logic, thus reducing reliance on traditionally bespoke hardware solutions.

The advent of software-defined satellites and digital infrastructure represent one of the most significant advancements within the satellite communications industry (Behlke 2025). The flexible nature of these software-defined payloads has created a need for ground stations that mirror their flexibility. This contrasts with the static, mission-specific architectures seen previously. Combining FPGA-based digital back-ends with high-performance radio front ends, allows significant adaptations to waveform configurations and communications protocols to be made with minimal hardware modifications (Malatest 2023).

Globally, the software-defined radio market is projected to have grown significantly by 2030 (Grand View Research 2026). This growth is partially driven by the multi-band and multi-protocol capability of SDR systems. This reduces associated hardware costs. The aerospace, defence and telecommunications industries remain the most significant end-use markets, reflecting the flexible SDR-based communications in both civil and military applications.

The broader shift from hardware-dependent systems towards flexible software-defined systems has presented opportunities within the satellite ground segment specifically. For designers, this evolution presents an opportunity to engage with reusable IP cores, utilise open or popular commercial development tools and interoperable communications standards. As widespread industry adoption accelerates, FPGA-based SDR transmitters can be utilised within experimental research platforms, but

also within scalable software-defined satellite networks designed to meet the demands of next-generation communication systems (Satsearch 2025, AWS n.d),

To ensure the precise and predictable timing control required in space applications, the deterministic execution of digital signal processing is critical. Here, determinism refers to a system where the later states of the system can be determined by earlier states (Darling n.d). In baseband processing applications, this means that the timing must be fixed and predictable, with repeatable latency across the entire RF chain (Malatest 2022).

In a satellite communication system, precise timing is crucial to ensure that time-critical data streams arrive at their intended destination within a predictable timeframe (Maral *et al* 2020). This ensures correct frame synchronisation, symbol timing and link reliability, thus providing a robust and reliable communication link. There are several potential sources of non-determinism within an SDR system. Software-centric SDRs are, by design, subject to processes such as OS scheduling, interface latency and USB/Ethernet buffering (Stewart *et al* 2015). Such processes can result in unpredictable timing, and non-deterministic execution.

FPGA-based baseband processing allows for signal processing to be performed synchronously with a fixed hardware clock. Flexible, synchronous input/output structures also ensure that input samples can be captured at exact clock edges and output samples provided to the communications channel with fixed latency (AMD 2017). This provides very predictable and fixed timing. Latency is still inevitable within an FPGA-based system, but is accepted as being lower, and more deterministic, than systems incorporating CPUs and GPUs (Curtiss-Wright 2020).

Consequently, software-based SDR platforms are shown to offer fast deployment and flexibility, whereas FPGA-based architectures provide the predictable latency and deterministic execution required for real-time satellite communications systems. Therefore, an FPGA-based SDR transmitter architecture will be implemented during this project.

2.3 Existing Software-Defined Radio Platforms

The defence, commercial and research sectors there is a plentiful array of existing SDR platforms. They all offer varying degrees of performance and flexibility, with a wide variation in cost. Enterprise-level solutions, such as those provided by Kratos and L3Harris, offer high-capacity and heavily virtualised ground segment infrastructures, but are largely proprietary and require heavy initial investment (Kratos Space n.d, L3Harris Technologies 2024). Conversely, cheaper SDRs (such as PlutoSDR and LimeSDR) are largely orientated towards education and research. However, they lack the bandwidth, power and scalability required to be integrated into a satellite uplink application. A detailed comparison of existing SDR platforms is provided in *Appendix 1*.

Table 6: SDR Comparison Summary

Platform	Target Market	Bandwidth	Cost Tier
L3Harris C/TT-600 SDR	Defence SATCOM ground systems	~ 50 MHz	£££££
NI Ettus USRP-295x series	Industrial/advanced SDR research	80-160 MHz	£££
NI Ettus B200/B210 series	Academic SDR research	~ 56 MHz	££
BladeRF 2.0 Micro	Academic SDR research	~ 56 MHz	££
LimeSDR	SDR prototyping	30-60 MHz	£-££
PlutoSDR	SDR education	~ 20 MHz	£
HackRF One	Hobbyist and entry-level SDR experimentation	~ 20 MHz	£

* Cost tier key: £ (< £500), ££ (£1000 to £2000), £££ (£3000 to £15000), ££££ (£15,000 to £50,000), £££££ (> £50,000)

This market research identified that there was a significant gap between the expensive mission-grade SDR systems, which offer exceptional performance but lack configurability, and low-cost academic platforms. This project could bridge this gap by providing a user-configurable SDR transmitter with deterministic timing at a much lower cost.

2.4 Space Communications Standards and Regulatory Frameworks

To ensure global interoperability, manage the shared radio frequency spectrum and guarantee reliability across a range of service providers and borders, satellite communications systems must operate within a framework of internationally recognised standards. These standards may define or recommend technical requirements for the system. They also provide regulatory guidance to ensure that such a system complies with the law of the jurisdiction it operates in, as well as international law. Standards organisations and regulatory bodies are therefore essential in shaping the architecture of satellite communications systems.

There are a variety of organisations that contribute to this every-growing and evolving regulatory framework. The most prominent are the Consultative Committee for Space Data Systems (CCSDS), the European Cooperation for Space Standardisation (ECSS) and the International Telecommunication Union Radiocommunication Sector (ITU-R). Additional commercial satellite communications standards (like DVB-S2) can provide recommendations for specific applications. The Institute for Electrical and Electronics Engineers (IEEE) are responsible for the creation and maintenance of various wireless communications standards which may also be applicable to the development an SDR system.

2.4.1 CCSDS Standards

The CCSDS is a global forum involved in the development of standards for space communications (CCSDS 2026). This suite of standards is extensive and constantly evolving. Founded in 1982, the CCSDS collaborates with major space agencies to develop common technical standards so that spacecraft and ground systems can communicate effectively (CCSDS 2026).

The CCSDS provides its technical recommendations in the form of documents known as “Blue Books”. These “Blue Books” are recommended standards that define specifications for a wide variety of system parameters. This includes data formats, modulation methods and operational procedures for space communications systems (CCSDS 2026a). These standards broadly correspond to the first five layers of the OSI model [see *Appendix 3: OSI Model Reference*]. Only the physical layer and data link layer were to be included within the scope of the project. This is because the transmitter does not form part of a larger network, and so higher levels are therefore outside of the scope of the project.

CCSDS standards also explicitly differentiate between two categories of spacecraft link. Telecommand (TC) links are involved in the control, configuration and operation of remote systems (such as a spacecraft). Such instructions are transmitted from a ground system to a spacecraft. In contrast, telemetry (TM) links are employed to transmit data from spacecraft to ground stations. This may include sensor measurements, mission data and spacecraft status information (CCSDS 2026a).

The standards also outline two types of systems: Category A and Category B. Category A systems are typically used for missions less than 2,000,000 km above the Earth, whereas Category B links are required for missions more than 2,000,000 km above the Earth (CCSDS 2021). This is defined as the boundary between deep space and near-Earth missions.

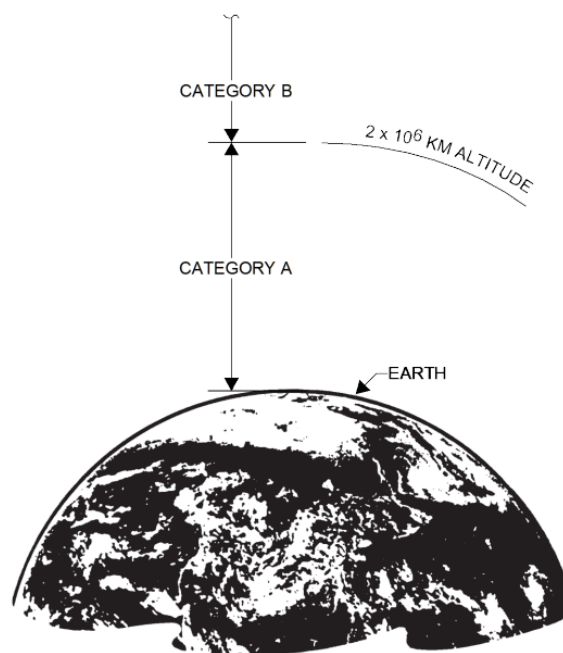


Figure 1: CCSDS Mission Categories (CCSDS 2021)

2.4.2 Additional Space Communications Standards

The following table summarises all the space communications standards that were reviewed. It details their scope and applicability to the project. A more detailed analysis can be found in *Appendix 2*. Ultimately, this comparison of standards determined that the CCSDS framework was the most appropriate because it was the most applicable to the space communications purpose of this transmitter (Aim 1, O1 and O2). The ITU Radio Regulations will be utilised to ensure conformance with frequency allocation regulations. Other frameworks were considered less suitable because of the increased implementation complexity that would be required, their lack of applicability to space communications or, in the case of DVB-S2, their use in one specific application.

Table 7: Additional Space Communications Standards

Standards Body	Relevant Standards	Scope	Applicability
ECSS	<i>ECSS-E-ST-10C</i> <i>ECSS-E-ST-50C</i>	Provides an overall systems engineering framework with detailed verification processes (ECSS 2017, ECSS 2024).	<i>Indirect relevance.</i> It supports system engineering processes on large projects but still largely references the CCSDS standards.
ITU-R	<i>ITU Radio Regulations</i> <i>Ofcom UKFAT</i>	Defines the global frequency allocation and usage rules to prevent unnecessary interference (Ofcom 2026).	<i>Required.</i> Needed to ensure regulatory compliance regarding frequency band allocation.
DVB-S2	<i>ETSI EN 302 307</i>	A specific space communications standard designed for broadcasting links. Cover the physical and data link layers (ETSI 2024).	<i>Limited applicability.</i> Optimised mainly for commercial broadcasting. May be more complex to implement than the CCSDS standards.
IEEE	<i>IEEE 802.11</i> <i>IEEE 802.16</i>	Defines terrestrial wireless communications systems including wireless local area networks and broadband access technologies (IEEE 2017, IEEE 2020).	<i>Contextual relevance.</i> Useful for understanding wireless systems design but not related to space communications.

2.5 Digital Modulation and Filtering

The CCSDS standards define acceptable waveform characteristics for use in space communications systems. Since telecommand and telemetry waveforms have different functions, different design considerations must be made.

2.5.1 Modulation

Modulation permits digital information to be mapped onto a carrier signal for transmission to be facilitated. The choice of modulation scheme is often a trade-off between power efficiency, spectral efficiency, robustness to channel conditions, and implementation complexity.

2.5.1.1 Telecommand Recommendations

The *CCSDS 401.0-B-32: Radio Frequency and Modulation Systems – Part 1* only strongly recommends one modulation scheme: Binary Phase Shift Keying (CCSDS 2021). Telecommand links must prioritise robustness and reliability over raw speed, and BPSK is generally considered to provide this over other phase shift keying methods (Link-PP 2025). Uncoded BPSK is also permitted but not recommended (CCSDS 2021).

2.5.1.2 Telemetry Recommendations

In contrast, telemetry links prioritise speed and spectral efficiency. The following schemes are therefore recommended by *CCSDS 401.0-B-32* (CCSDS 2021).

For Category A and Category B links:

- Binary Phase Shift Keying (BPSK)
- Quadrature Phase Shift Keying (QPSK)
- Offset Quadrature Shift Keying (OQPSK)
- Gaussian Minimum Shift Keying (GMSK)

For Category B links only:

- 8-Phase Shift Keying (8PSK)

2.5.2 Channel Coding

Channel coding schemes introduce controlled redundancy into the transmitted data. This improves reliability by allowing the receiver to detect and control errors. These errors may have been caused noise, interference or channel impairments.

2.5.2.1 Telecommand Recommendations

CCSDS 231.0-B-4: TC Synchronization and Channel Coding dictates that the following channel coding schemes are appropriate for telecommand links (CCSDS 2021a).

- Modified Bose-Chaudhuri-Hocquenghem (BCH)
- Low-Density Parity Check (LDPC)

2.5.2.2 Telemetry Recommendations

Coding gain and efficiency are important within telemetry links. Therefore, *CCSDS 131.0-B-5: TM Synchronization and Channel Coding* provide a wider list of acceptable

channel coding schemes (CCSDS 2023). Such schemes provide varying coding gain values, with varying bandwidth expansions.

- Convolutional
- Reed-Solomon (RS)
- Turbo Coding
- Low-Density Parity Check (LDPC)

A concatenated coding option is also available where a convolutional code is used as the inner code, and a Reed-Solomon code is used as the outer code. This generally results in improved performance (CCSDS 2023).

2.5.3 Filtering and Pulse Shaping

Filtering and pulse shaping are required to reduce inter-symbol interference and to control the bandwidth of the output signal.

The CCSDS standards make recommendations regarding acceptable spectral efficiency, advising the use of Square-Root Raised Cosine Filters and Root Raised Cosine Filters (CCSDS 2021).

2.6 Spread Spectrum Techniques

Spread spectrum techniques can be used to enhance security, reduce interference, and improve operability in contested environments. They are therefore paramount in defence applications because they are employed to reduce the likelihood of signal detection.

The two primary spread spectrum techniques are Direct Sequence Spread Spectrum (DSSS) and Frequency Hopping Spread Spectrum (FHSS) (Burrell 2024). DSSS operates by making the transmitted signal much wider in bandwidth than the original information signal (Ciciora *et al* 2004). This reduces the transmitter signal's vulnerability to narrowband interference. This is performed by multiplying the data with a pseudo-random spreading sequence. FHSS operates utilises a predefined hopping sequence to rapidly switch the signal frequency (RFWirelessWorld n.d).

Implementing DSSS can operate a very low signal-to-noise ratios but require precise synchronisation between the transmitter and receiver to function (Waite 2024). DSSS also intrinsically require continuous spreading and despreading operations, which will likely require significant amounts of FPGA resources to be implemented at high data-rates. In contrast, FHSS can be implemented alongside relatively simple modulation schemes such as BPSK or QPSK. Therefore, a frequency hopping waveform could be designed in accordance with the CCSDS guidelines for telecommand and telemetry links (CCSDS 2021). This would likely be more suitable than attempting to implement a DSSS waveform.

Review of current literature suggests that there may be several ways of implementing FHSS within as SDR system. Each approach presents different trade-offs in terms of precision, complexity, testability and spectral performance. The first approach is to use a numerically controlled oscillator (NCO) and DDS within the FPGA. Frequency hopping is performed by altering the phase increment at each hop (AMD 2024). This

allows for rapid and very precise hopping but can result in spurious emissions. The second approach is to retune the local oscillator (LO) of the RF transceiver, configuring the hardware to modify the carrier frequency (Analog Devices 2021). Latency is introduced due to the tuning settling times, but the signal benefits from high spectral purity. An alternative is to store multiple pre-generated signals at pre-defined frequencies within the FPGA BRAM and select between them during operation (Scholl 2016). However, such a method will be constrained by memory requirements. It also reduces flexibility and user configurability, as the waveforms are fixed within the BRAM. Finally, frequency shifted I/Q samples can be streamed through software to the transmitter (Mitola 2000). This perhaps offers the greatest flexibility but is not fully deterministic as a result (Simon *et al* 1994).

2.7 Embedded SDR Platforms and Architectures

This section seeks to evaluate the suitability of the *Digilent ZedBoard Zynq-7000 ARM/FPGA SoC Development Board* and the *Analog Devices AD9361 RF Agile Transceiver*. These boards were specifically bought by Airbus Defence and Space Ltd for use in this project. Therefore, any solution would have to be suitable for implementation on these two boards.

2.7.1 Digilent ZedBoard FPGA

The *Digilent ZedBoard* is based on the AMD (Xilinx) Zynq-7000 SoC (AMD 2018). A dual-core ARM Cortex-A9 processing system is integrated into programmable logic, forming a single device. The main advantage of this is that the device permits hardware-accelerated and deterministic signal processing, whilst enabling software-based control and monitoring. This supports a modular system design and allows for portioning of system functionality. This board can be purchased from Digilent for 589 USD, thus making it reasonably affordable for this project (Digilent 2026).

2.7.2 AD9361 FMCOMMS3 RF Transceiver

The Analog Devices AD9361 offers a much wider frequency range than earlier narrowband or band-segmented RFICs (Analog Devices 2024). The transceiver integrates much of the RF signal chain including PLLs, dual transmit and receive channels, mixers, local oscillators and high-speed 12-bit DACs/ADCs (Analog Devices 2024a). The integrated fractional-N synthesizer and Fastlock mechanism enables LO retuning within tens of microseconds, hence making frequency hopping feasible. This contrasts with previous discrete PLL/VCO devices which were much slower and therefore could not support agile FHSS (Analog Devices 2024a).

Table 8: AD9361 FMCOMMS3 Key Specifications (Analog Devices 2024)

Parameter	Value	Applicability
Frequency Range	46.875 MHz to 6 GHz	Supports multi-band operation and flexibility
Maximum Sample Rate	61.44 MSPS	Supports high data rates, oversampling and pulse-shaping
LO Retune Time	20 μ s (Fastlock) 250 μ s (Standard)	Permits the implementation of a frequency hopping capability.
Tuneable Channel Bandwidth	<200 kHz to 56 MHz	Allows wideband modulation and high data rates.
Cost	1107.45 USD (Analog Devices 2026)	Relatively affordable for research and educational users.

2.8 Performance Metrics in Digital Communications

These metrics can be considered collectively but are inherently interdependent. The use of E_b/N_0 enables a normalised comparison between different modulation and coding schemes, whilst BER provides a normalised measure of system reliability given varying noise conditions (Proakis and Salehi 2008). There is a trade-off between reliability and spectral efficiency. Often, improvements in data throughput typically requires a greater E_b/N_0 to maintain an acceptable error performance.

Accurate evaluation E_b/N_0 and BER in a practical system will, however, require knowledge of the transmitted bitstream and the creation of a fully synchronised receiver. If the system includes framing, encoding and configuration of different waveforms, this may be difficult to perform. Consequently, the system performance may need to be verified using theoretical expectations, spectral analysis and observed internal signal characteristics, with simpler metrics (such as SNR) prioritised.

Table 9: Key Performance Metrics

Metric	Definition	Applicability
Signal-to-Noise Ratio (SNR)	Ratio of signal power to noise power within a given bandwidth.	Gives a general indication of channel quality.
Energy per Bit to Noise Spectral Density Ratio (E_b/N_0)	Ratio of energy per transmitted bit to noise spectral density.	Normalises performance with respect to bit energy.
Bit Error Rate (BER)	Probability of bit being received incorrectly.	Defines system reliability under varying E_b/N_0 conditions.
Spectral Efficiency	Number of bits transmitted per second per Hz of bandwidth.	Defines the trade-off between data rate and robustness. Higher efficiency generally requires a higher E_b/N_0 .
Bandwidth	Frequency range occupied by the transmitted signal.	Determines spectrum utilisation. Can be used to determine whether an output spectrum is as expected.

2.9 Research Gap

The literature review identified the following market and user needs. They will be used to influence the direction of any additional research, and the final system implementation.

Table 10: User and Market Needs

Market/User Need	Description and Justification
MN-1: Space-Relevant Waveforms	Provides a range of waveforms compliant with acknowledged space standards.
MN-2: Deterministic Baseband Processing	Provides hardware-scheduled, deterministic processing that permits sufficiently accurate synchronisation, encoding and frequency hop timing.
MN-3: Supports Full Access to the Underlying HDL	This allows students and researchers to configure and evaluate waveforms, and therefore the full transmit chain.
MN-4: Financially Accessible	Remains financially accessible to small research groups and universities.
MN-5: Integrated with COTS RF Transceivers	Use of the AD9361 promises verified RF performance and is commercially available if others wish to replicate the system.
MN-6: Use in Collaboration with Open-Source Tools	Can be used in conjunction with GNU Radio for testing, and an open-source operating system (if a bare metal architecture is not chosen).

3 PROJECT METHODOLOGY, MONITORING AND CONTROL

3.1 Project Management and Control

3.1.1 Risk Management

In accordance with ISO 31000, the primary project risks were identified and mitigation measures proposed (ISO 2018). Initially, there were several risks with a high likelihood-severity rating. After the implementation of the mitigation strategies, the remaining risks were reduced to medium or low levels.

Severity Likelihood	Negligible (1)	Minor (2)	Moderate (3)	Severe (4)	Very Severe (5)
Highly Likely (5)					
Likely (4)			R5, R6	R2, R4, R9	
Possible (3)			R3, R7	R8	
Unlikely (2)					R1
Rare (1)					

Figure 2: Risk Matrix Before Mitigation Actions

Severity Likelihood	Negligible (1)	Minor (2)	Moderate (3)	Severe (4)	Very Severe (5)
Highly Likely (5)					
Likely (4)		R5, R6, R8			
Possible (3)		R2, R3, R7	R4, R9		
Unlikely (2)					
Rare (1)			R1		

Figure 3: Risk Matrix After Mitigation Actions

Table 11: Identification of Project Risks and Mitigation Actions

Risk Reference Number	Risk Description	Mitigation Actions
R1	Development computer failure	• Use OneDrive and GitHub to save any work regularly. • Make a computer available at work for use if primary laptop fails.
R2	Loss of project files/data	• Use Git to ensure version control. Utilise frequent commits whenever work is completed and have backup files for important documents (such as deliverables).
R3	Software failure	• Use stable tool versions that are compatible with the utilised operating system. • Back up files in OneDrive and use frequent commits in Git for effective change control.
R4	Project tasks taking longer than planned (late submission of deliverables will result in a grade cap of 40%)	• Regularly review progress with project supervisor and seek advice if required. • Consistently monitor progress against Gantt chart. • Prioritise critical tasks and adjust the scope of the project (by removing a data input method or waveform configuration) if necessary.
R5	FPGA timing or implementation failures	• Incremental design and testing. • Review resources provided from previous FPGA courses.
R6	AD9361 operating in an incorrect configuration	• Change parameters gradually. • Only configure settings that are understood. • Consult AD9361 datasheet before configuring settings.
R7	Waveform switching causes glitching	• Testing as extensive as possible before full deployment. • Halt transmission during switching.
R8	Hardware overheats	• Use the Vivado power analysis tool before implementing the design on the board physically.
R9	User interface does not function as intended and therefore waveform switching cannot occur	• Testing before full deployment. • Keep the GUI as simple as possible, whilst still being user-friendly. • Develop a CLI first, then add the GUI itself later if there is time available.

3.1.2 Resource Management Considerations

During Phase 2, only the AD9361, ZedBoard, Vivado and the development laptop were used consistently. These resources were essential for becoming familiar with the boards, their reference designs and the Vivado IDE, as well as research, simulation, and early experimentation. Non-availability of these resources would have caused delays to these activities, as well as the progression towards full physical implementation.

Phase 3 introduced the implementation, integration and testing parts of the project. This introduces new resource requirements namely for electrical test equipment (such as a spectrum analyser) and potentially for RF cables, RF splitters and a COTS modem. Non-availability of the test-related hardware and software (such as GNU Radio), and the resources required for the user interface (such minicom) would delay the development of the waveforms, system integration and testing activities.

Later key design decisions altered the resource requirements. Kuiper Linux was adopted [see Section 5.2.2.5] so that eliminated the need for the AMD Vitis software. The omission of a GUI [see Section 5.3] eliminated the need for dependencies such as PyQt and Python3.

The full resource requirements can be found in *Appendix 4*. Ultimately, proactive management of resources meant that there were few delays associated with resource non-availability.

3.1.3 Phase 1 and Phase 2 Time Management

Project scheduling and progress monitoring was primarily conducted using a Gantt chart. Progress during Phase 2 largely followed the expected schedule, with some minor delays associated when researching how to design and implement custom AXI IP blocks in Vivado. This was mainly due to the steep learning curve encountered when learning the process in Vivado, but also difficulties associated with inserting this custom IP into the AD9361 reference design. However, these issues did not significantly affect overall progress.

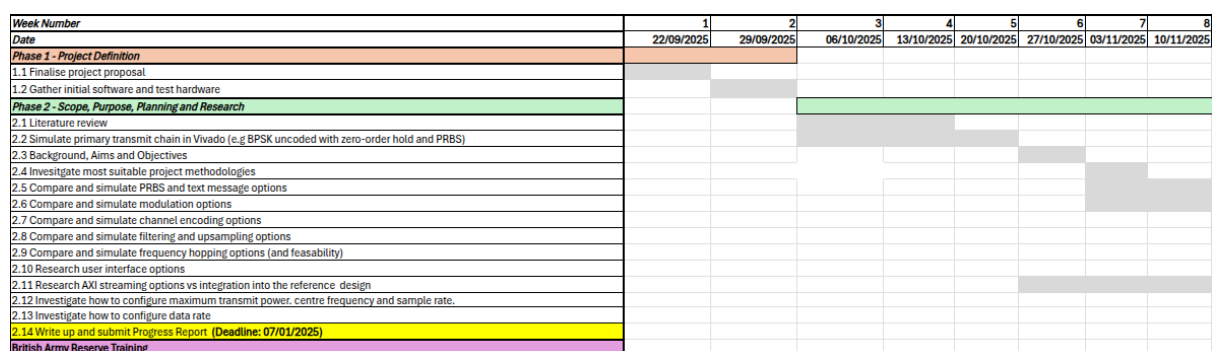


Figure 4: Gantt Chart (22/09/2025 - 10/11/2025)

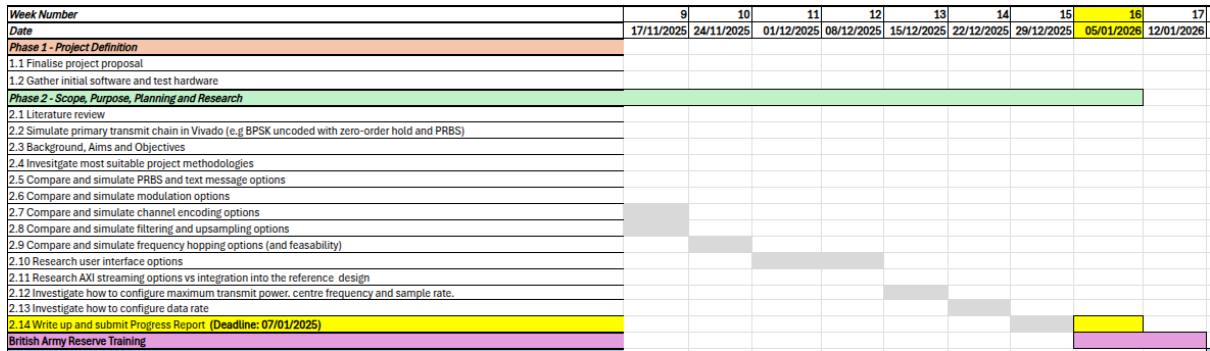


Figure 5: Gantt Chart (17/11/2025 - 05/01/2026)

3.1.4 Phase 3 Time Management

3.1.4.1 Initial Project Scheduling

Research conducted during Phase 2 influenced a significant change in the project methodology. This new methodology prioritised compliance with CCSDS standards over flexible RF transmit chain customisation to ensure realistic space communications waveform development [see Section 5.1].

Work package prioritisation was directly influenced by the project risk analysis. During Phase 3, higher risk and dependency-heavy waveforms were scheduled earlier to reduce schedule and integration risk, simpler or optional features were generally scheduled later, with the intention of omitting them if this was required. This helped to ensure that the core system functionality could be delivered under the time constraints. Therefore, the achievement of the project aims and objectives could be safeguarded.

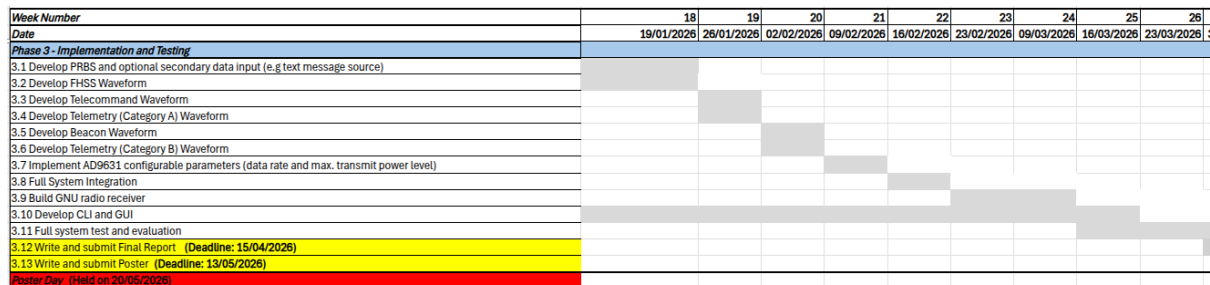


Figure 6: Gantt Chart Derived at End of Phase 2 (19/01/2026 - 23/03/2026)

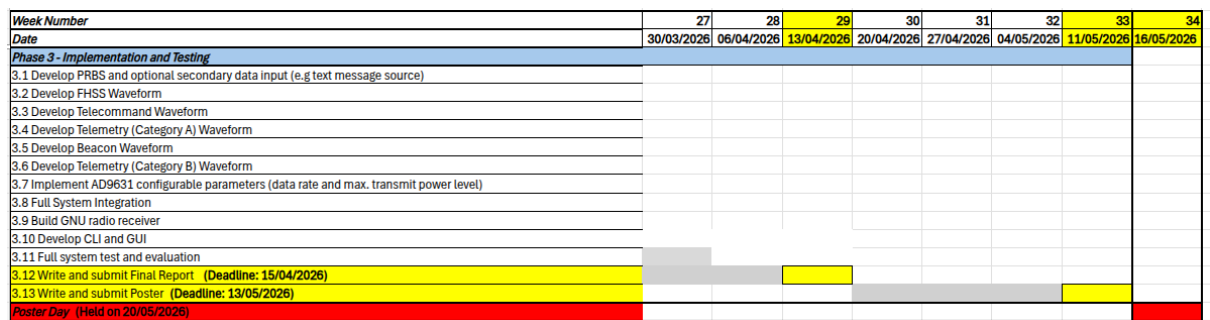


Figure 7: Gantt Chart Derived at End of Phase 2 (30/03/2026 - 20/06/2026)

3.1.4.2 Deviations from Initial Plan

Several deviations from the Phase 3 original project schedule were encountered, affecting the scope and duration of various work packages.

Initially, two weeks were scheduled into the project plan to account for British Army Reserves training. The assumption was that no project work would be carried out during this period. However, the training ended a week earlier than expected, so an additional week could be devoted to project work. This improved scheduling flexibility, and the additional time was devoted to developing the PRBS and modulation mappers.

Secondly, it took more time to develop the PRBS RTL and testbench than anticipated, so the optional secondary text message data input was ultimately omitted. Work on the first waveform (FHSS) was started but again development was taking longer than anticipated. An additional change in methodology was decided towards the end of January because of this. Instead of developing each waveform individually, it was decided that the modulation RTL and testbenches would be developed, then the framing RTL and testbenches, followed by the encoding aspects and then any filtering components. The frequency hopping was scheduled later in case other key aspects of the transmit chain took longer to be completed than expected.

This shift from developing each waveform individually to prioritised shared waveform components (such as the PRBS and modulation mappers) was a deliberate mitigation in response to these deviations in the original plan. The change in methodology allowed for a primary transmit chain to be developed early, permitting early validation. This reduced the risk of failing to deliver any verifiable implementation and ensured that some core system functionality required to meet the project aims and objectives could be successfully demonstrated within the project timeframe (Aim 1, O4 and O6). It was important to effectively apply project control strategies to overcome deviations from the schedule whilst retaining alignment with the project aims and objectives. Consideration of these deviations and mitigation strategies directly informed the design amendment detailed in *Section 5.3*.

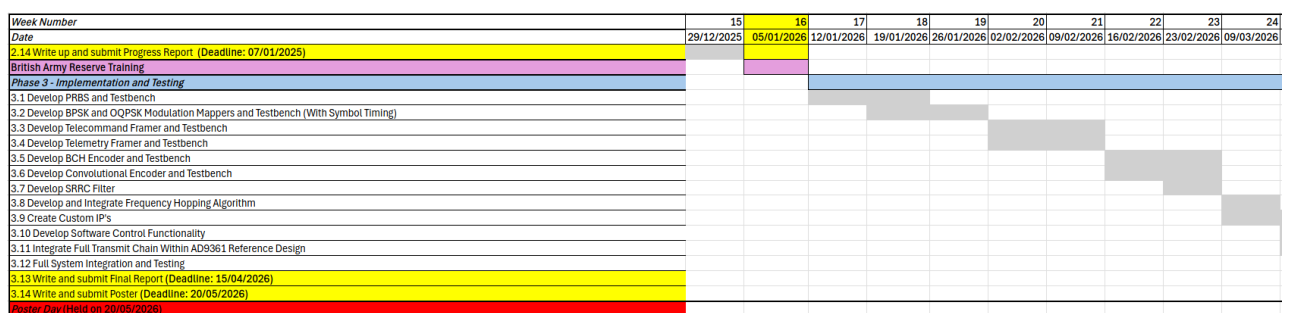


Figure 8: Gantt Chart Followed During Phase 3 (29/12/2025 - 09/03/2026)

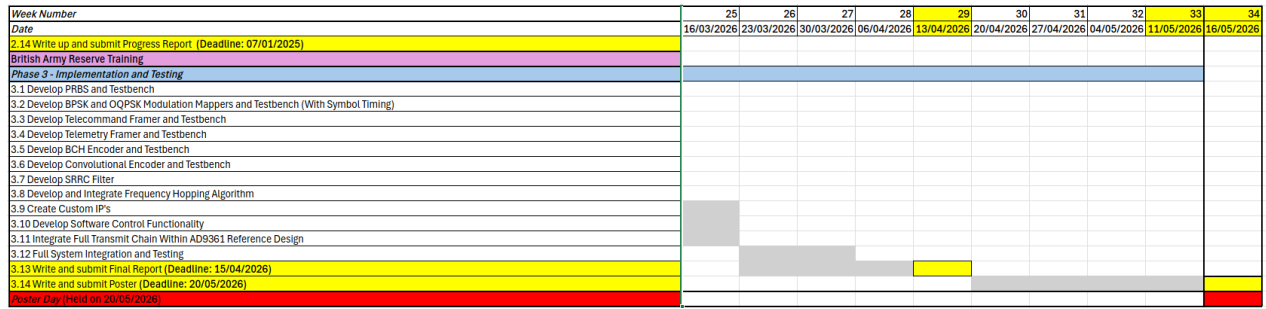


Figure 9: Gantt Chart Followed During Phase 3 (16/03/2026 - 16/05/2026)

3.2 Project Methodology Overview

The overall project methodology was broken down into roughly three phases.

Phase 1 (22/09/2025 to 05/10/2025): Phase 1 focused on assessing project feasibility and the likely system constraints. Essential hardware and software were obtained which permitted early experimentation, informing the project specification and providing possible performance metrics.

Phase 2 (05/10/2025 to 11/01/2026): Phase 2 focused on research into space communications standards. Evaluation of the *AD9361* and *ZedBoard* datasheets further enabled the development of suitable waveform designs (Digilent 2024, Analog Devices 2021). Market analysis, embedded software architecture options, and GUI development workflows further refined the project specification. This phase culminated in the submission of the Progress Report due on 07/01/2026.

Phase 3 (12/01/2026 to 20/05/2026): Phase 3 focused on implementation, integration and testing. The transmit chain was developed incrementally with individual HDL IP blocks (such as the data source, framing and modulation) designed and verified before wider system integration. The AXI4-Lite interface was subsequently incorporated to permit software-based configuration and control. The deliverables for this phase were the Full Report (15/04/2026) and the Poster (20/05/2026).

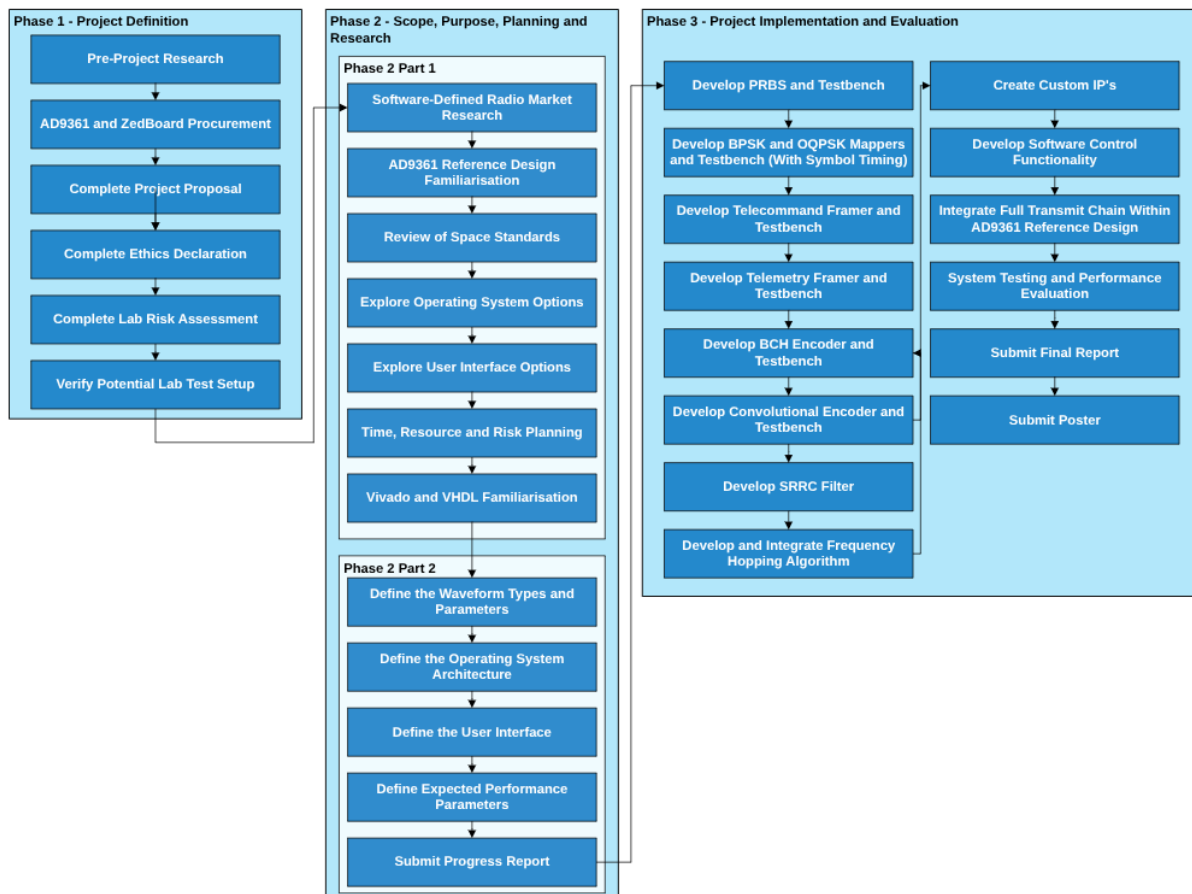


Figure 10: Full Project Methodology Diagram

4 PRODUCT DESIGN SPECIFICATIONS

Functional requirements dictate the required behaviour of the SDR transmitter, whereas non-functional requirements specify the system constraints and performance criteria (Beaton 2024).

Table 12: Project Specifications (Functional Requirements)

Requirement Reference	Requirement Description	Objectives and Market Needs Supported
Functional Requirements		
FR-1	The system must generate at least four predefined, selectable waveforms (telemetry, telecommand, beacon and FHSS).	O1, O2, O4 MN-1
FR-2	The user must be able to change between the waveforms during the runtime utilising a configurable user interface.	O2, O3, O5 MN-3, MN-6
FR-3	The system must provide both a PRBS and, optionally, a text message data input, which the user can switch between at runtime. During the transitions, the interface should provide an I/Q output that contains either valid samples of the waveform or a defined muted state ($I = Q = 0$) with no undefined value presented to the AD9361.	O3, O4, O5, O6 MN-2, MN-3
FR-4	The waveform configurations must all include predefined parameters: • Channel coding • Modulation • Filtering • Frequency range	O1, O2, O4 MN-1, MN-3
FR-5	The system must generate a complex I/Q baseband output which is compatible with the AD9361 transmit interface using AXI.	O1, O3, O4, O6 MN-5
FR-6	The user must be able to configure the transmit power level (within the limits of the AD9361) at runtime.	O3, O5 MN-3, MN-5
FR-7	The system must allow the user to configure the data rate at runtime with the system automatically calculating the appropriate symbol and sample rates for the modulation and coding schemes.	O3, O4, O5 MN-2, MN-3
FR-8	The system must support a FHSS waveform that periodically changes carrier frequency.	O1, O2, O4 MN-1, MN-2
FR-9	The system must allow real-time monitoring of the following key transmitter parameters: • Selected waveform name • Output frequency • Coding scheme • Modulation type • Filtering type • Symbol rate	O3, O5, O6 MN-3, MN-6

	• Sample rate • Data rate	
FR-10	The system must provide a mechanism to stop, start and restart the transmitter without requiring FPGA programming or software reload.	O3, O5 MN-2, MN-3
FR-11	The telemetry and telecommand waveforms are to be CCSDS-compliant. They are to follow CCSDS recommendations for coding, modulation and frame structure.	O1, O2, O4, O6 MN-1
FR-12	The transmitter must permit verification and testing using a software-defined radio receiver built in GNU Radio using a loopback connection.	O4, O6 MN-4, MN-6
FR-13	The transmitter must permit verification and testing using a COTS modem which serves as a receiver.	O6 MN-5
FR-14	The transmitter must permit both the GNU Radio receiver and the COTS modem receiver to operate simultaneously.	O3, O4, O5, O6 MN-6
FR-15	The system must validate any user-configured parameters to ensure that they do not exceed the limits of the AD9361. If an invalid configuration occurs, a warning is to be displayed to the user, and the transmitter must mute its output ($I = Q = 0$).	O3, O4, O5, O6 MN-2, MN-5
FR-16	The system must be able to function fully within a standard laboratory environment.	O5, O6 MN-4

Table 13: Project Specifications (Non-Functional Requirements)

Requirement Reference	Requirement Description	Objectives and Market Needs Supported
Non-Functional Requirements		
NFR-01	The transmitter must output I/Q samples without interruptions to the data pipeline and in real-time.	O3, O4, O6 MN-2
NFR-02	The symbol and sample rates must remain within 1% of their configured values to allow for successful demodulation.	O4, O6 MN-2
NFR-03	The transmitted carrier frequency must remain within the AD9361 PLL stability tolerance so that frequency lock is maintained.	O4, O6 MN-5
NFR-04	The system should be able to operate for at least 1 hour without requiring a reset (manual, hardware or software).	O4 MN-2, MN-4
NFR-05	Any telemetry waveforms output by the transmitter should achieve a bit error rate performance consistent with the theoretical BER vs E_b/N_0 curve of the selected modulation and coding methods.	O1, O4, O6 MN-1, MN-6

NFR-06	Any telemetry waveform outputs must maintain a stable SNR with variations of less than +/- 5 dB over a 10-second observation window.	O4, O6 MN-1
NFR-07	Any telemetry waveform outputs must support an accurate estimation of Eb/No at the receiver, and the measured EbNo must remain within +/- 0.5 dB of the value predicted given the parameters of the transmitter.	O1, O6 MN-1, MN-6
NFR-08	Out-of-band emissions should be attenuated by at least 40 dB relative to the main lobe to ensure a clean spectrum.	O1, O4, O6 MN-1, MN-5
NFR-09	Whilst utilising the FHSS waveform, frequency hops must not introduce excessive distortion (especially to phase or amplitude). Any temporary effects should not interrupt continuous data transmitter.	O2, O4, O6 MN-2
NFR-10	The complete transmitter must fit within the logic, BRAM, I/O pins, DSP, routing and power constraints of the Zynq-700 device with at least 10% headroom.	O1, O3, O4 MN-3, MN-4
NFR-11	The FPGA design must meet timing closure with no setup or hold violations.	O1, O3, O4 MN-2, MN-3

5 PRODUCT DESIGN DEVELOPMENT

5.1 Initial Proposal

During August 2025, the initial high-level system architecture for the SDR transmitter was proposed. This planned to allow the user to configure a modular FPGA-based signal processing chain using either a command line or a GUI. It would also permit real-time configuration of parameters such as data rate, centre frequency and transmit power level. Optional frequency hopping was also intended to be implemented in hardware. This system design was flexible and mostly deterministic. Allowing for the gradual incorporation of secondary (optional) features alongside the primary (mandatory) features meant that the scope of the project could be flexible. Therefore, risks associated with time management could be mitigated. However, it did not allow the user to switch through distinct waveforms and was not compliant with globally recognised space communications standards.

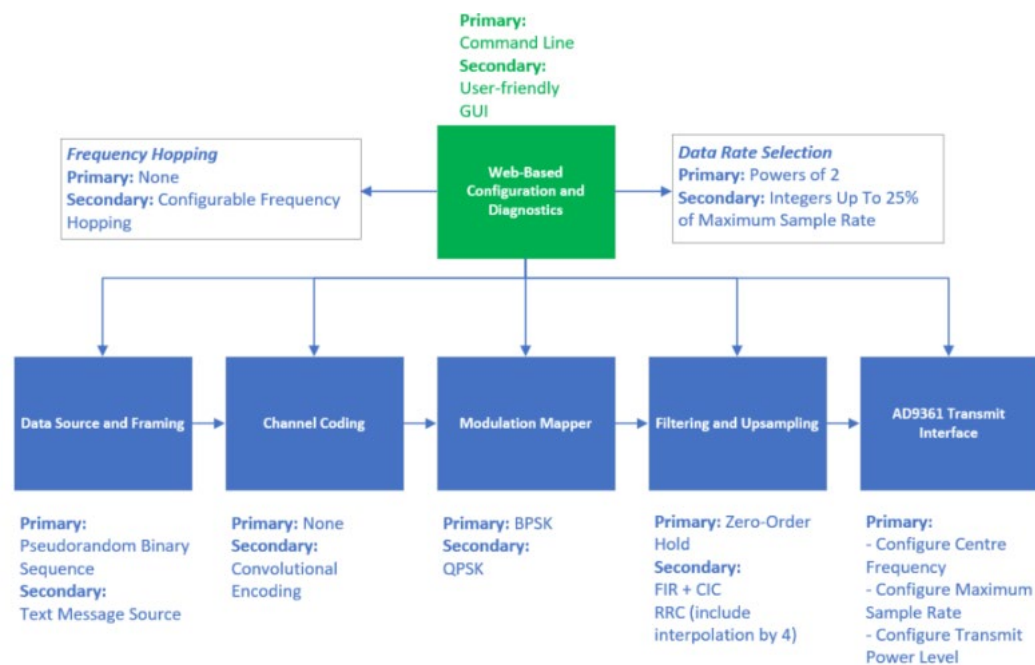


Figure 11: Initial System Design Proposal

5.2 Initial System Design Refinement

Following the clear identification of the user and market needs [see Section 2.8: Research Gap], it was decided that the SDR transmitter should incorporate five waveforms. These waveforms would be: Telecommand, Telemetry (Category A), Telemetry (Category B), Beacon and FHSS. The Beacon waveform was intended to be a simple, uncoded waveform which could permit a simple evaluation of the whole system.

5.2.1 Options Analysis Overview

For the Telecommand and Telemetry waveforms, the applicable CCSDS standards define a fixed frame structure, as well as recommending suitable parameters for the SRRC filter design (CCSDS 2021, CCSDS 2021a, CCSDS 2021b, CCSDS 2021c,

CCSDS 2023). Hence the various encoding and modulation options will be further evaluated.

Table 14: CCSDS-Compliant Waveforms Standards Summary

Waveform Name	Framing	Encoding	Modulation	Filtering
Telecommand	CCSDS 232.0-B-4	CCSDS 231.0-B-4	CCSDS 401.0-B-32	CCSDS 401.0-B-32
Telemetry (Category A)	CCSDS 132.0-B-3	CCSDS 131.0-B-5	CCSDS 401.0-B-32	CCSDS 401.0-B-32
Telemetry (Category B)	CCSDS 132.0-B-3	CCSDS 131.0-B-5	CCSDS 401.0-B-32	CCSDS 401.0-B-32

The Beacon and FHSS waveforms are not defined by any CCSDS standard. The below characteristics have been chosen so that the transmitter produces a diverse set of waveforms, without over-complicating the design and testing of the system.

Table 15: Beacon and FHSS Waveforms Parameters Summary

Waveform Name	Framing	Encoding	Modulation	Filtering
Beacon	None	None	BPSK	Simple SRRC (for improved spectral containment)
FHSS	None	None	BPSK	Simple SRRC (for improved spectral containment)

Several different FHSS implementation methods appear to be feasible. This includes a mixed of hardware-based and software-based approaches.

Table 16: FHSS Implementation Methods Summary

FHSS Implementation Technique	Description
FPGA DDS/NCO Hopping (Hardware)	Frequency hopping is implemented with the FPGA's programmable logic (AMD 2024). A numerically controlled oscillator will alter the carrier frequency by updating the phase increment of the DDS at each hop.
AD9361 LO Retune Hopping (Software-Controlled and Hardware-Executed)	Software retunes the AD9361's local oscillator. The carrier frequency is altered because the LO configuration changes at each hop (Analog Devices 2021).
LUT/BRAM Waveform Switching (Hardware)	Multiple pre-generated waveforms are stored within the FPGA BRAM. Each waveform is centred on a different carrier frequency. Hopping occurs when a different waveform is selected at runtime by the FPGA's programmable logic (Scholl 2016).
Software-Generated IQ Hopping (Software)	I/Q samples at the desired carrier frequencies are streamed, in software, to the AD9361 (Mitola 2000).

Three potential embedded software architectures were identified to support system control and configuration.

Table 17: Embedded System Architectures Summary

Embedded Software Architecture	Description
Bare Metal	The software runs directly on the CPU without an operating system. This allows direct access to the hardware with low overheads and deterministic timing. However, all drivers, peripheral control and other system-level functions must now be dealt with manually.
PetaLinux (25.02)	PetaLinux provides a command-line workflow developed by AMD/Xilinx for Zynq devices (AMD 2025). It can support drivers, networking, file systems and further application development. This introduces additional overheads and less determinism than in a bare metal implementation.
Kuiper Linux (11.02)	Based on the Raspberry Pi OS, this is a free open-source embedded Linux operating system developed by Analog Devices (Analog Devies 2024). It provides in-built support for Zynq devices and the AD9361 RF transceiver. It is well-suited to rapid prototyping and hardware bring-up. However, there is less determinism than in a bare metal approach, and greater overheads.

5.2.2 Detailed Options Analysis

There were different criteria for each of the five main design decisions. Therefore, using a weighted scoring Multi-Criteria Design Analysis (MCDA) method allows multiple competing objectives to be evaluated, and each trade-off balanced and compared (Belton and Stewart 2002). Each MCDA matrix provides three or four candidate solutions, each evaluated against five different parameters (with its own weighting). Each parameter is assigned a score (1-5). For the parameters, a score of

5 represents the most favourable performance (for example highest reliability, best conformance with standards and lowest latency).

Given the evaluation of the CCSDS standards in *Section 2* of this report, often there were more than three permitted combinations of modulation and channel coding schemes for a waveform. In such cases, three feasible options were shortlisted with the intention of keeping the waveforms distinct in character. This helped to keep the options analysis process clear and feasible.

5.2.2.1 Telemetry (Category A)

Telemetry (Category A) signals are continuous communications from a spacecraft and are used to monitor the health and operational status of the system (CCSDS 2021). Therefore, FPGA complexity and ease of testing were prioritised to allow verification that the system can run reliably for long durations (NFR-04, NFR-10 and NFR-11). CCSDS-compliance was weighted equivalently to ensure the development of a suitable space communications waveform (FR-11 and NFR-05). Spectral efficiency is less important because stability, reliability and signal quality are to be prioritised over throughput in this type of communications.

Table 18: Telemetry (Category A) MCDA Matrix

Waveform	CCSDS Compliance	Development Complexity	Ease of Testing	Coding Gain	Spectral Efficiency	Total Score
Weighting	0.2	0.3	0.2	0.2	0.1	1
TMA-1: OQPSK + Turbo	5	3	4	4	4	3.90
TMA-2: OQPSK + LDPC (Long Block, Rate ½)	5	1	2	5	4	3.10
TMB3-3: GMSK + Turbo	4	4	2	4	5	3.70

5.2.2.2 Telemetry (Category B)

Telemetry (Category B) communications provide non-continuous messages from the spacecraft such as stored data. Category B telemetry is designed to tolerate degraded links (CCSDS 2021). Therefore, reliability has been most heavily considered (NFR-01, NFR-04). Compliance with the CCSDS standards means that the transmission is as suitable to space communications as possible. However, it must permit verification using GNU Radio and a COTS modem in accordance with FR-12 and FR-14. FPGA complexity and latency is rated as less important because the signals are not time-critical and additional processing is acceptable provided that the timing constraints are ultimately met (NFR-10 and NFR-11).

Table 19: Telemetry (Category B) MCDA Matrix

Waveform	CCSDS Compliance	Development Complexity	Ease of Testing	Reliability	Latency	Total Score
Weighting	0.25	0.3	0.15	0.35	0.10	1

TMB-1: BPSK + RS (255, 223) + Conv(1/2, K=7)	5	4	5	5	3	4.55
TMB-2: BPSK + RS-Only	4	3	4	3	4	3.95
TMB3-3: BPSK + Conv(1/2, K=7) only	3	2	5	2	4	3.20

5.2.2.3 Telecommand

Telecommand signals serve as an uplink between a ground station to the spacecraft. Telecommand signals must be highly reliable because they include commands to control and safeguard the space mission. Therefore, bit errors can have operational consequences (CCSDS 2021). Compliance with CCSDS standards has been weighted most strongly in accordance with FR-11. To allow for verification and testing (FR-13), ease of testing is important, and the waveform design must permit realistic development and testing times within the project schedule (R4). As mentioned, reliability is valued highly. Low latency is important because telecommand must reach the spacecraft within the expected time. However, since telecommand traffic is infrequent and generally at lower data rates, a slightly increased latency is acceptable.

Table 20: Telecommand MCDA Matrix

Waveform	CCSDS Compliance	Development Complexity	Ease of Testing	Reliability	Latency	Total Score
Weighting	0.20	0.25	0.25	0.15	0.15	1
TC-1: BPSK + LDPC Full Block	5	1	2	2	1	2.25
TC-2: BPSK + BCH	4	4	4	4	5	4.10
TC-3: BPSK + LDPC Short Block	5	2	3	5	3	3.45

5.2.2.4 Frequency Hopping Techniques

Determinism and speed are prioritised so that the waveform can carry out rapid and repeatable frequency hops without interrupting the output or violating timing constraints (NFR-01 and NFR-11). Complexity and spectral purity criteria aim to balance practical feasibility, without introducing excessive distortion during hops (NFR-08 and NFR-09). Flexibility is weighted lowest as the ability to configure the nature of the frequency hops is useful but is secondary to achieving clean and verifiable hopping behaviour.

Table 21: Frequency Hopping Techniques MCDA Matrix

FHSS Implementation Method	Determinism	Speed	Complexity	Spectral Purity	Flexibility	Total Score
Weighting	0.3	0.25	0.2	0.15	0.1	1

FH-1: FPGA DDS/NCO Hopping (Hardware)	5	5	3	4	5	4.45
FH-2: AD9361 LO Retune Hopping (Software)	2	2	3	3	3	4.30
FH-3: LUT/BRAM Waveform Switching	4	3	2	5	1	3.20
FH-4: Software-Generated IQ Hopping (Software)	1	1	2	3	2	1.60

5.2.2.5 Embedded Software Architecture

Timing and driver support have been prioritised because the system must maintain an uninterrupted output and reliable operation (NFR-01 and NFR-03), and deterministic behaviour must be prioritised. Rapid development, integration and testing are important to ensure successful project completion, and an easily configurable system (FR-02, FR-09 and FR-10). Customisation flexibility during development is beneficial but not essential.

Table 22: Embedded Software Architecture MCDA Matrix

Architecture Option	Timing	Driver Support	Development Complexity	Monitoring Capabilities	Customisation Flexibility	Total Score
Weighting	0.3	0.25	0.2	0.15	0.1	1
OS-1: PetaLinux	3	4	3	4	4	3.50
OS-2: Kuiper Linux	3	5	4	5	3	4.00
OS-3: Bare-Metal	5	2	3	2	5	3.40

5.2.3 Phase 2 Chosen Solution

The following table provides a summary of the five final waveform configurations. Kuiper Linux was determined to be the most suitable embedded software architecture. The hardware-based FPGA DDS/NCO hopping was considered the most suitable FHSS implementation method. This provided an initial solution at the Phase 3.

Table 23: Phase 2 Waveform Configuration

Waveform Name	Framing	Encoding	Modulation	Filtering
Telecommand	CCSDS 232.0-B-4	BCH	BPSK	CCSDS 401.0-B-32

Telemetry (Category A)	CCSDS 132.0-B-3	Turbo	OQPSK	CCSDS 401.0-B-32
Telemetry (Category B)	CCSDS 132.0-B-3	RS	BPSK	CCSDS 401.0-B-32
Beacon	None	None	BPSK	Simple SRRC (for improved spectral containment)
FHSS	None	None	BPSK	Simple SRRC (for improved spectral containment)
Frequency Hopping Implementation				
FH-1: DD/NCO Hopping				

5.3 Phase 3 Design Amendments

5.3.1 Summary of Additional Design Changes

Several key design changes were made during Phase 3 due to implementation challenges and time constraints. All the following changes were made after the PRBS and modulation mapper RTLs and testbenches had been built. By that point, a more realistic estimation of the time required to write the VHDL code became apparent. These changes were made to ensure that the project remained achievable within the given timeframe and maintained compliance with the project specifications, namely FR-1, FR-11 and NFR-10.

5.3.1.1 Omission of Telemetry (Category B) Waveform

Initially, this additional Telemetry waveform was considered. However further review of the CCSDS standards made it clear that Category B waveforms are primarily designed for deep-space communications (CCSDS 2021). The SDR transmitter was not intended to be suitable for deep-space applications. As a result, the Telemetry (Category A) was deemed to be more representative of the intended operational environment. It became clear a few weeks into Phase 3 that inclusion of five distinct waveforms instead of four would have increased the risk of the project overrunning because of the extra development work required (R4).

5.3.1.2 Modification of Telemetry (Category A) Waveform Specification

The original design of the Telemetry (Category A) waveform included the inclusion of Turbo encoding. A more detailed analysis of the applicable CCSDS standard was conducted to permit the development of the VHDL RTL code (CCSDS 2023). However, it became clear that the development of a Turbo encoder would be considerably more complex and time-consuming than for other encoding methods. Therefore, the channel coding method for this waveform was redefined. The full MCDA matrix is included in the following section. It was also decided that the modulation scheme should be changed from OQPSK to BPSK. This is permissible according to the CCSDS standard, and removes any additional complexity associated with the need for symbol offset in OQPSK modulation (CCSDS 2021).

5.3.1.3 Change of FHSS Modulation Scheme from BPSK to OQPSK

The change of the FHSS waveform modulation from BPSK to OQPSK was mainly influenced by the change in the Telemetry (Category A) modulation scheme. By the time that decision was taken, the BPSK and OQPSK modulation mappers had already been developed. Therefore, changing the modulation scheme for the FHSS waveform allowed a diversity in the waveform configurations, and allowed the OQPSK mapper module to still be utilised.

5.3.1.4 Preference for Only One Filter Module

It was also decided that to save time and FPGA resources that all waveforms would use the same SRRC filter. This filter was to follow the *CCSDS 401.0-B-32: Radio Frequency and Modulation Systems – Part 1* guidelines (CCSDS 2021).

5.3.1.5 Change in FHSS Implementation Method

The implementation of frequency hopping was placed later in Phase 3 to ensure that the other waveforms could be developed in the allocated time frame. As a result, by this stage in the project, there was only a very limited amount of time that could be spent on developing the frequency hopping functionality. Therefore, **FH-2: AD9361 LO Retune Hopping** was chosen over the approach planned originally (**FH-1: FPGA DDS/NCO Hopping**) due to its low implementation complexity and its easy integration into the existing developed software.

5.3.2 Additional Options Analysis for Telemetry (Category A) Channel Coding

Given the concerns about the availability of time later in the project, the complexity involved in developing the Telemetry channel coding module was heavily considered. However, the recommendations of the *CCSDS 131.0-B-5: TM Synchronisation and Channel Coding* standard were also considered in terms of which schemes were generally preferred over others (CCSDS 2023). Ultimately, given these new criteria, Convolutional encoding was considered the most appropriate.

Table 24: Additional Telemetry (Category A) Channel Coding MCDA Matrix

Channel Coding Option	CCSDS Compliance	Development Complexity	Total Score
Weighting	0.3	0.7	1
CC-1: Convolutional Only	4	5	4.7
CC-2: Concatenated (Reed-Solomon + Convolutional)	5	1	2.2
CC-3: Reed-Solomon Only	4	3	3.3
CC-4: Turbo	5	1	2.2
CC-5: Low-Density Parity Check	5	2	2.9

5.3.3 Phase 3 Final Chosen Waveform Configurations

This table presents the final waveform configurations for this project. The cells shaded in grey represent the design parameters that were modified during Phase 3.

Table 25: Phase 3 Waveform Configuration

Waveform Name	Framing	Encoding	Modulation	Filtering
Telecommand	CCSDS 232.0-B-4	BCH	BPSK	CCSDS 401.0-B-32
Telemetry (Category A)	CCSDS 132.0-B-3	Convolutional	BPSK	CCSDS 401.0-B-32
Beacon	None	None	BPSK	CCSDS 401.0-B-32
FHSS	None	None	OQPSK	CCSDS 401.0-B-32
Frequency Hopping Implementation:				
FH-2: AD9361 LO Retune Hopping				

6 IMPLEMENTATION AND TESTING

6.1 Custom Main Transmit Chain Design Considerations

The diagram below is a simplified block diagram of the custom transmit chain, and the flow of data through it dependent on the waveform.

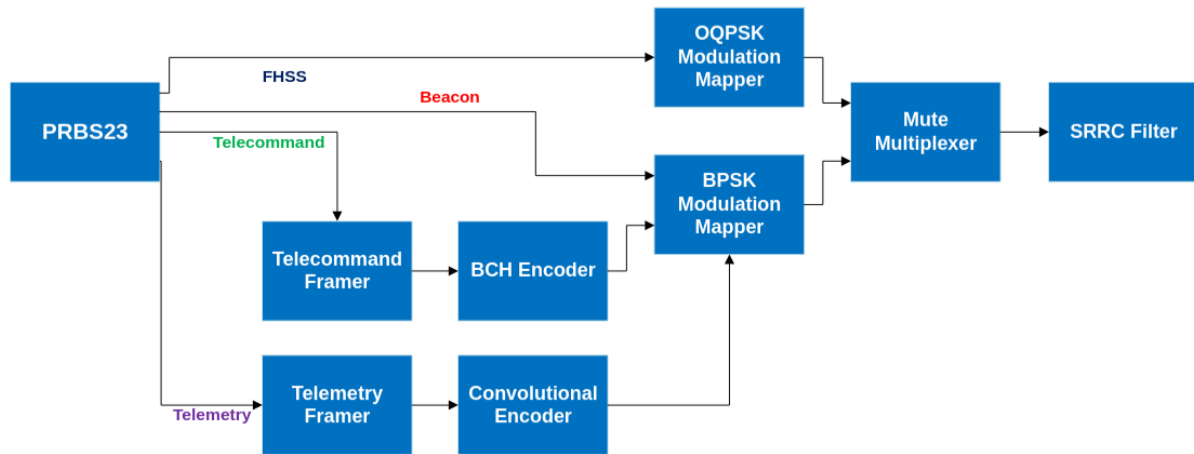


Figure 12: Custom Transmit Chain Block Diagram

6.1.1 Pseudorandom Binary Sequence Data Source

A PRBS was chosen as the data source for the SDR transmitter. It was chosen over other data input sources (such as text or images) because of its deterministic nature, and relative simplicity to implement. The PRBS implemented using a linear-shift feedback register (LFSR), utilising a Galois architecture. An alternative Fibonacci architecture was also considered. Both Galois and Fibonacci structures produce identical binary sequences, but a Galois LFSR will distribute the combinational logic across the registers, as opposed to concentrating it at the input. This means that the LFSR can run faster and is generally considered to be more efficient, if less intuitive (Aduvo Engineering 2024).

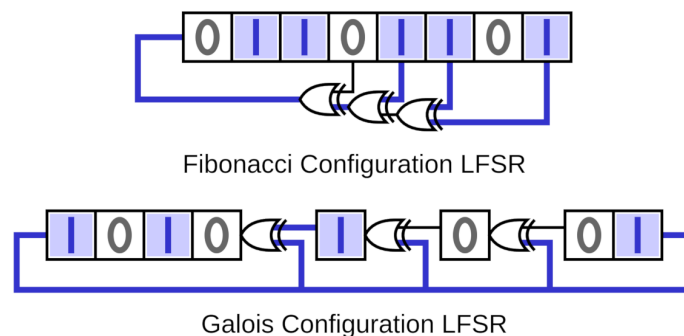


Figure 13: Galois and Fibonacci Architecture Comparison (Moria 2022)

The PRBS23 sequence was chosen due to its long period of $(2^{23} - 1)$ bits. This ensures that the PRBS gives the impression of the data being sufficiently random during practical measurements. PRBS23 is commonly used in high-speed data integrity testing (PRBS23 n.d). The following polynomial is used:

$$x^{23} + x^{18} + 1$$

The design employs a non-zero fixed seed (an all-zero state will result in lock-up), synchronous reset and enable-gating to ensure deterministic and synchronous behaviour.

6.1.2 Modulation Mappers

6.1.2.1 BPSK Mapper

The BPSK mapper takes an input bit and map a '0' to a negative amplitude then '1' to a positive amplitude. This is the standard in digital communications systems (Jerred 2022). This mapping is applied to the in-phase (I) whilst the quadrature (Q) channel is held at zero.

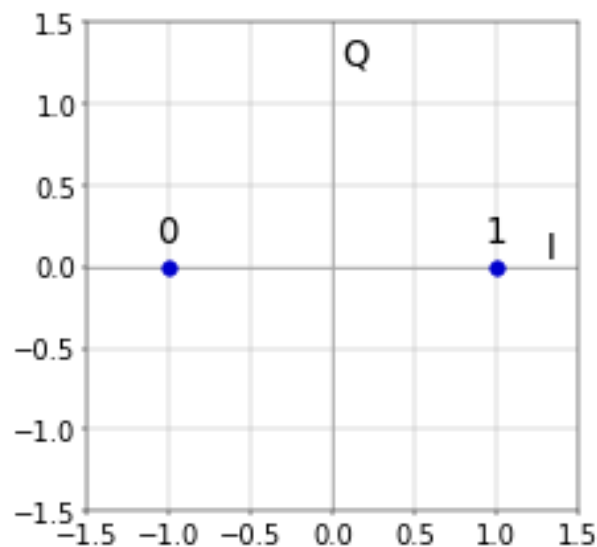


Figure 14: BPSK Expected Constellation (Roelandts n.d)

6.1.2.2 OQPSK Mapper

In QPSK and OQPSK modulation, pairs of bits are mapped onto four constellation points. The same bipolar mapping is applied here, but both I and Q values are mapped. In OQPSK, in contrast to QPSK, the Q channel is delayed by half a symbol period. Therefore, only one channel changes at any given time. In this implementation, a single binary input stream was used.

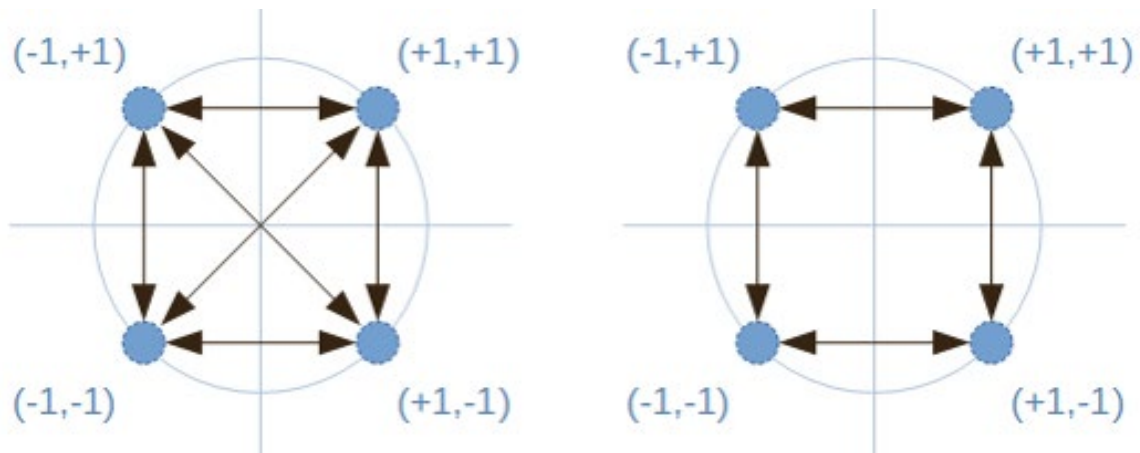


Figure 15: QPSK [left] and OQPSK [right] Expected Constellations (Electroagenda 2024)

6.1.3 Framers

6.1.3.1 Telecommand Framer

The framer for the telecommand waveform is based on the *CCSDS 232.0-B-4: TC Space Data Link Protocol* (CCSDS 2021b). The standard TC Transfer Frame is used as the Protocol Data Unit.

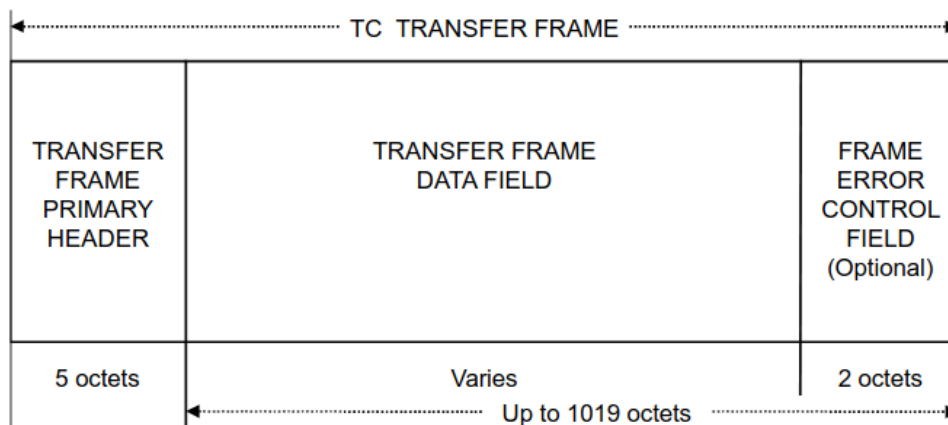


Figure 16: TC Transfer Frame Structure (CCSDS 2021b)

The primary header provides the key information required for frame identification, including spacecraft identification, channel selection, and frame length. The CCSDS standard defines the values that each field must take given a particular configuration. The following table gives the header values chosen during the implementation of the telecommand framer.

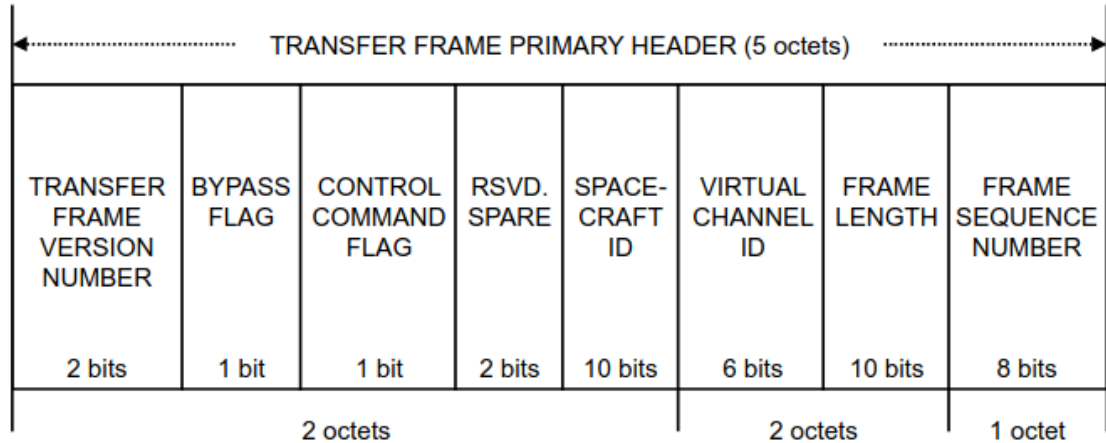


Figure 17: TC Transfer Frame Header Format (CCSDS 2021b)

Table 26: TC Transfer Frame Implemented Header Values

Field Name	Field Value	Justification
Transfer Frame Version Number	00	Specified by <i>CCSDS 232.0-B-4</i> .
Bypass Flag	0	Indicates a Type-A transfer frame where Frame Acceptance and Reporting (FARM) would be carried out at a CCSDS-compliant receiver.
Control Command Flag	0	The frame carries the Frame Data Unit, instead of receiver control information.
Reserved Spare	00	Specified by <i>CCSDS 232.0-B-4</i> .
Spacecraft Identifier	0000000001	Configurable via generic.
Virtual Channel Identifier	00000000	Configurable via generic.
Frame Length	0000010110	Frame Length = Payload bytes + header bytes + CRC bytes - 1.
Frame Sequence Number	8 bits incrementing from 0.	Specified by <i>CCSDS 232.0-B-4</i> . Provides frame ordering and so allows for detection for lost frames at the receiver.

A 16-bit Cyclic Redundancy Check (CRC) was implemented to provide error detection capability. Whilst technically optional, it improves overall reliability, and so was included to create a more authentic waveform. The CRC uses the following polynomial:

$$G(x) = x^{16} + x^{12} + x^5 + 1$$

The CRC is initialised to all ones (0xFFFF), as required by the CCSDS standard, and is then computed across the header and payload fields. Once this process is completed, the final CRC value is stored and then transmitted.

The telecommand frame itself is produced using a synchronous four-state Finite State Machine (FSM). It updates every symbol period. This approach ensures that there are deterministic timing and clear separation of frame components.

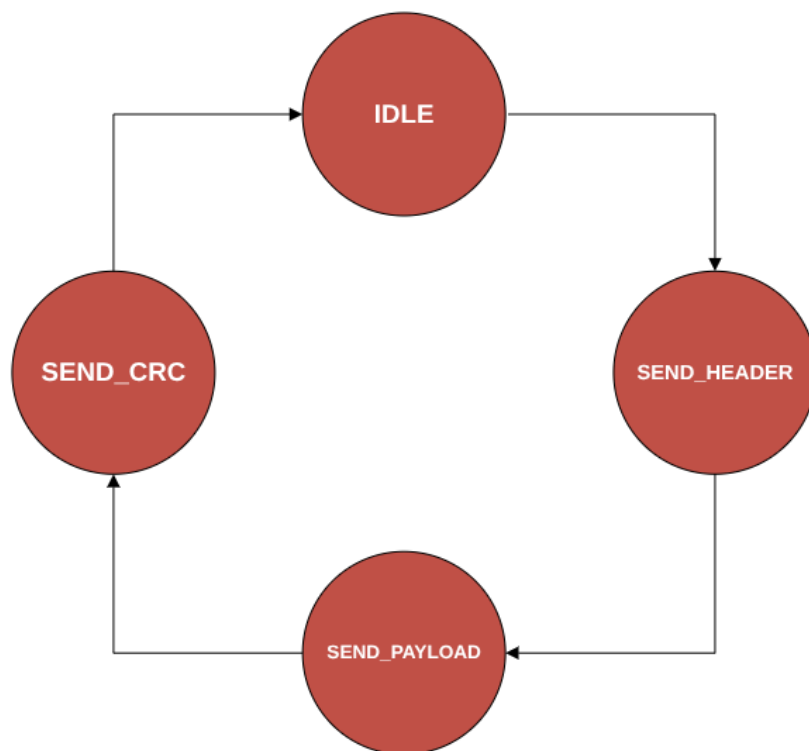


Figure 18: Telecommand Framer FSM

6.1.3.2 Telemetry Framer

The framer for the telemetry waveform is based on the *CCSDS 132.0-B-3: TM Space Data Link Protocol* (CCSDS 2021c). The standard TM Transfer Frame is used.

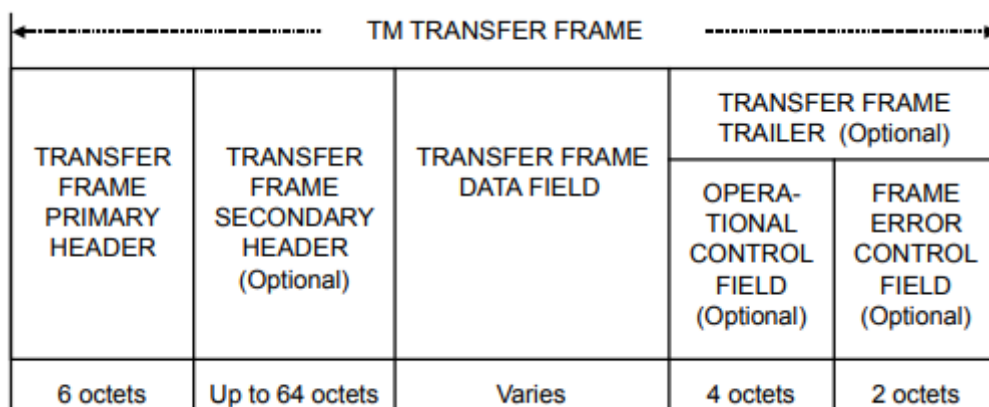


Figure 19: TM Transfer Frame Structure (CCSDS 2021c)

As with the telecommand framing, the header of the telemetry frames contains information essential for spacecraft and channel identification, field status and frame numbering.

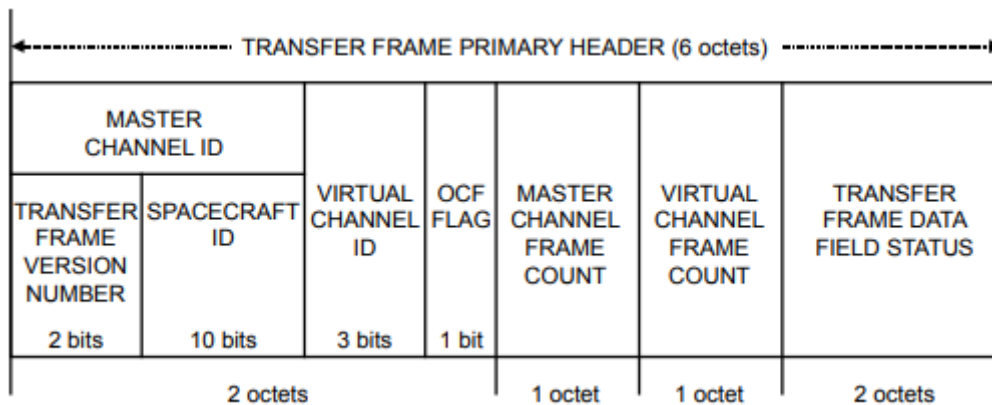


Figure 20: TM Transfer Frame Header Format (CCSDS 2021c)

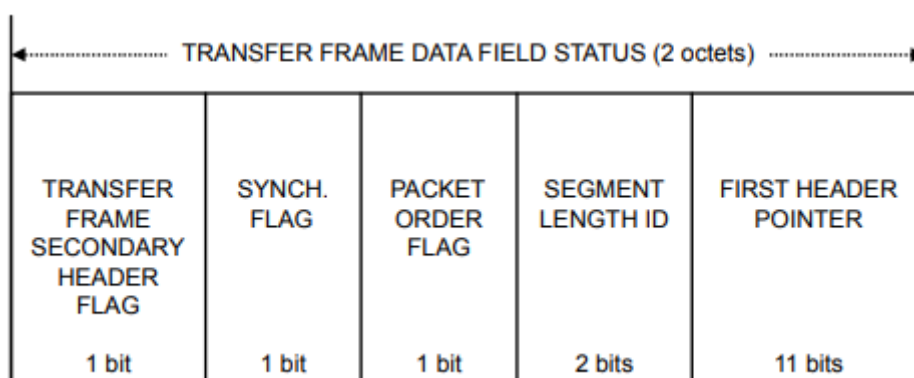


Figure 21: TM Transfer Frame Data Field Status (CCSDS 2021c)

Table 27: TC Transfer Frame Implemented Header Values

Field Umbrella Name	Field Name	Field Value	Justification
Master Channel ID	Transfer Frame Version Number	00	Specified by <i>CCSDS 132.0-B-3: TM Space Data Link Protocol</i>
	Spacecraft ID	0000000001	Configurable via generic.
	Virtual Channel ID	000	Configurable via generic.
	OCF Flag	0	No Operational Control Field Flag
	Master Channel Frame Count	8 bits incrementing from 0	Specified by <i>CCSDS 232.0-B-4</i> . Provides frame ordering and so allows for detection for lost frames at the receiver.
	Virtual Channel Frame Count	8 bits incrementing from 0	Specified by <i>CCSDS 232.0-B-4</i> . Provides frame ordering and so allows for detection for lost frames at the receiver.

Transfer Frame Data Field Status	Transfer Frame Secondary Header Flag	0	Transfer Frame Secondary Header not included.
	Synchronisation Flag	0	Telemetry payload is a continuous PRBS rather than a structured VCA_SDU.
	Packet Order Flag	0	Synchronisation Flag is '0' so Packet Order Flag must be '0'.
	Segment Length ID	11	Synchronisation Flag is set to '0' so Segment Length ID must be '11'.
	First Header Pointer	000000000000	Packets are not being used.

The same CRC scheme is used as both the telecommand and telemetry standards specify the same technique and polynomial (CCSDS 2021b, CCSDS 2021c).

In contrast to the telecommand framing, the telemetry frame must be preceded by an Attached Sync Marker (ASM). The nature of this ASM is specified in *CCSDS 131.0-B-5: TM Synchronization and Channel Coding (CCSDS 2023)*. The ASM allows for proper frame synchronisation at the receiver. This is especially important when using more complex channel coding techniques (for example Reed-Solomon, Turbo and LDPC). The ASM is defined in hexadecimal as 1ACFFC1D when convolutional encoding is utilised.

Again, a synchronous FSM was used to control the sequential construction and then transmission of each frame. To account for the inclusion of an ASM, five states are used.

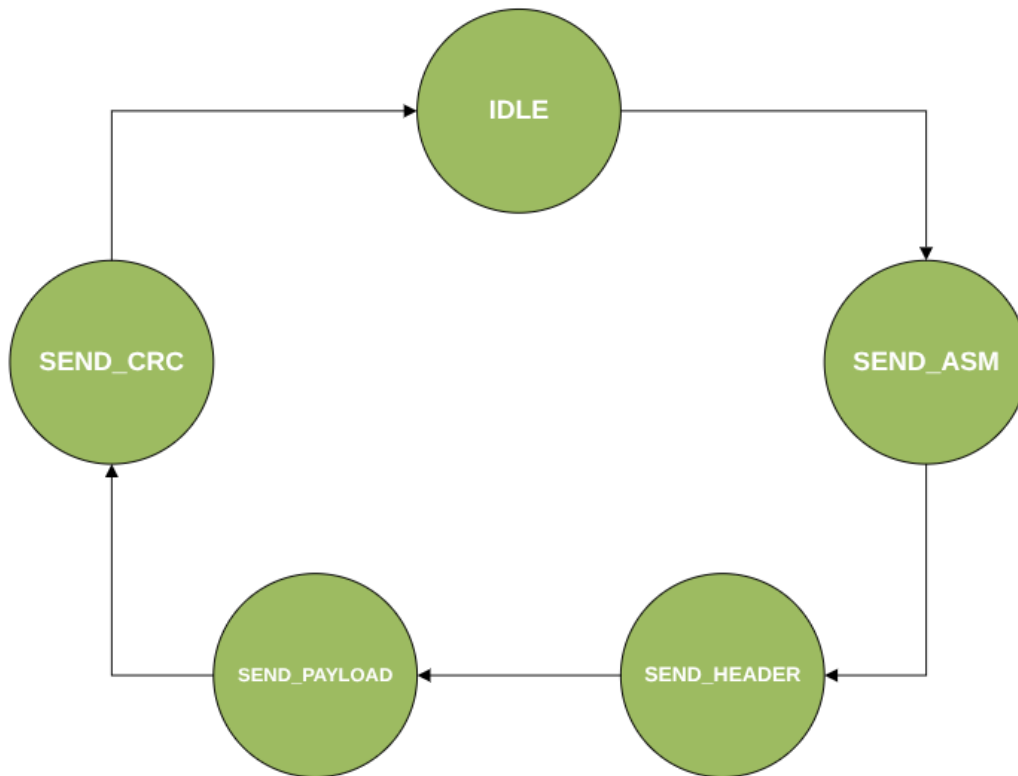


Figure 22: Telemetry Framer FSM

6.1.4 Encoders

6.1.4.1 BCH Encoder

The telecommand encoder is implemented as a BCH encoder compliant with the *CCSDS 231.0-B-4: TC Synchronization and Channel Coding (CCSDS 2021a)*. It operates by creating a codeword with 7 parity bits appended to 56 information bits. An additional filler bit is added to the end to create a 64-bit (8 octets) codeword.

Therefore, to avoid the use of filler information bits which will introduce additional complexity in building the encoder, the telecommand frame size should be a multiple of 7. Hence, a payload size of 14 bytes was chosen.

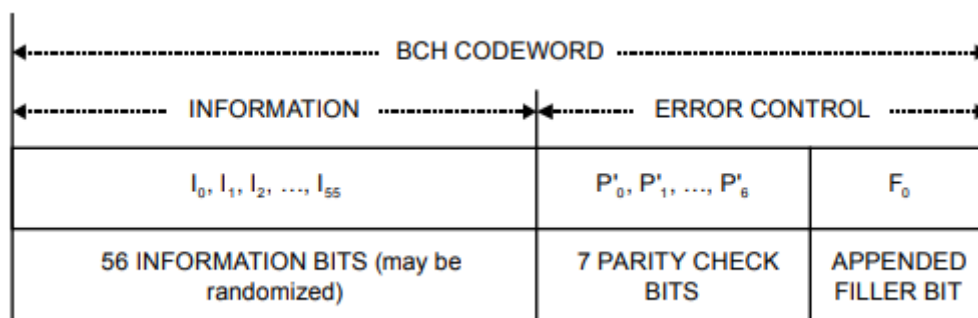


Figure 23: BCH Codeword Structure (CCSDS 2021a)

The encoding itself is performed by a 7-bit parity register using linear feedback logic according to the following polynomial:

$$g(x) = x^7 + x^6 + x^2 + 1$$

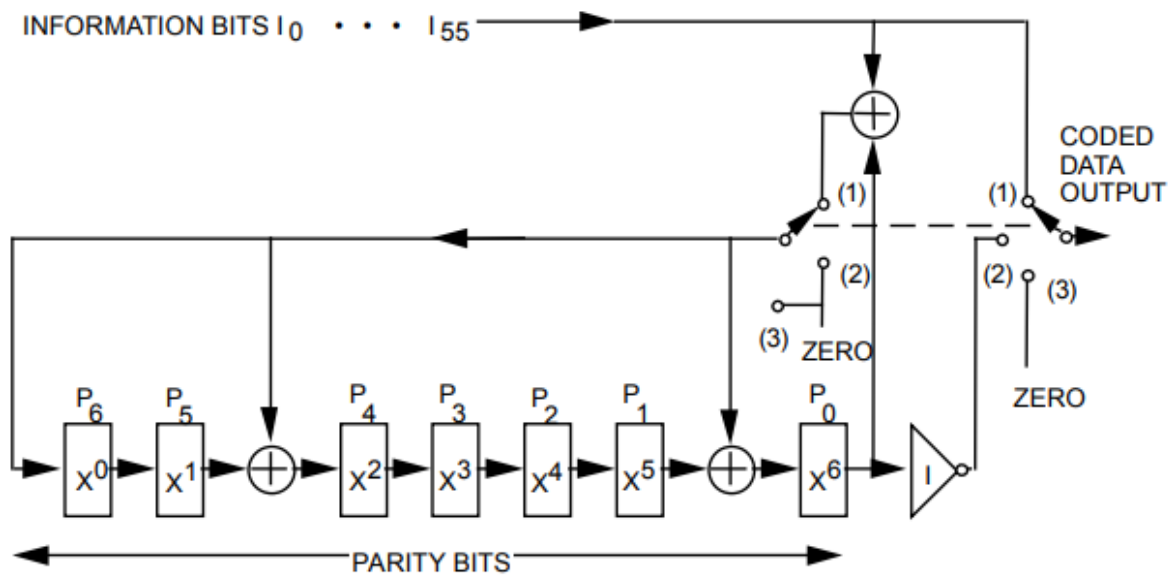


Figure 24: BCH Encoder Structure

To ensure the controlled and correct construction of the BCH codeword, a four-state FSM is utilised.

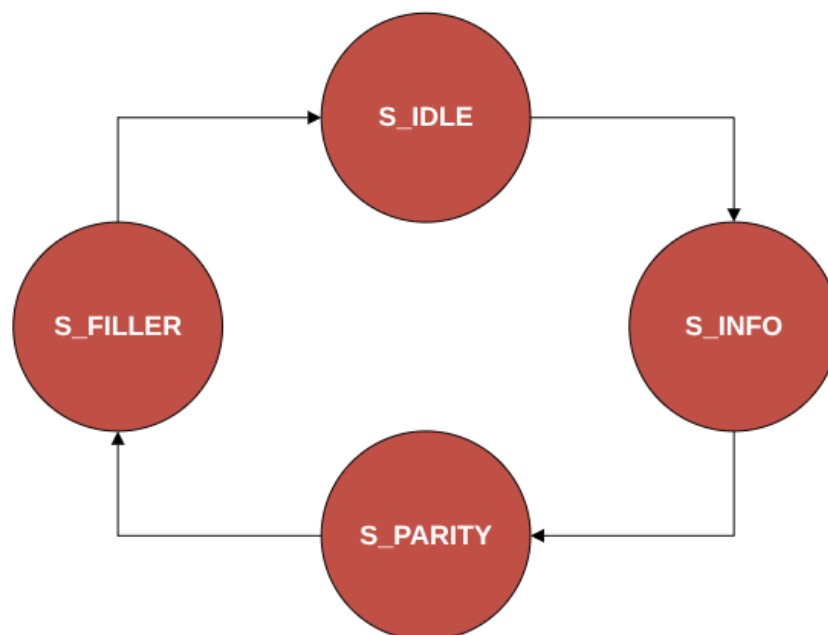


Figure 25: BCH Encoder FSM

6.1.4.2 Convolutional Encoder

The basic convolutional code according to *CCSDS 131.0-B-5: TM Synchronization and Channel Coding* was selected (CCSDS 2023). This has a rate of $\frac{1}{2}$ and constraint length of 7. The standard allows for the code to be punctured so that higher rates can be used. Since the transmitter is only anticipated to operate in a test environment, these punctured convolution encoding schemes were dismissed as unnecessary.

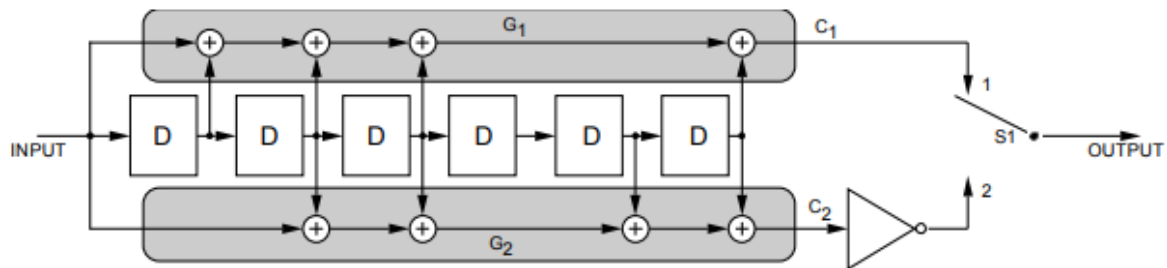


Figure 26: Convolutional Encoder Block Diagram (CCSDS 2023)

The standard also sanctions the use of an additional pseudo-randomiser to ensure sufficient bit transition density, particularly if modulation methods such as QPSK or OQPSK methods are used (CCSDS 2023). However, in this implementation, a pseudo-randomiser was not used because a PRBS is being used as the data input source. This is considered to perform in a pseudorandom manner in the first place, so additional randomisation can therefore be omitted. BPSK is additionally a more robust modulation scheme to low transition density, and this partially influenced the decision for the telemetry waveform to use BPSK instead of OQPSK [see Section 5.3.1].

This encoder employs a two-state FSM. This FSM ensures that for each input bit two output bits are produced. During the S_IDLE state, the encoder waits for a valid input bit then computes the C_1 and C_2 encoded values. The C_1 value is transmitted immediately, and the C_2 value is stored. During the S_OUT_C2 state, the C_2 bit is transmitted.

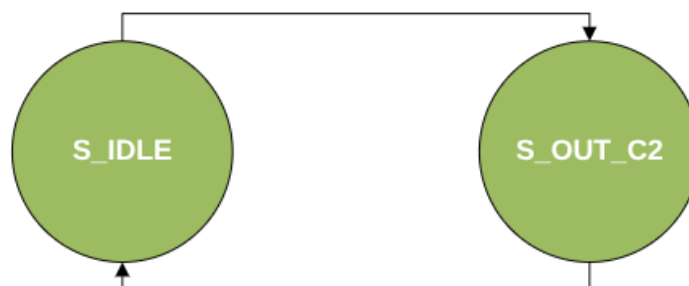


Figure 27: Convolutional Encoder FSM

6.1.5 Mute Multiplexer

Once enabled, this IP core converts both the I and Q input channels into zeroes. It has been positioned after the modulation to comply with FR-3 [see Section 4: Product Design Specifications]. Placing it before the modulation mapper would result in the muted outputs ($I = Q = 0$) being mapped to a negative amplitude.

6.1.6 SRRC Filter

The pulse shaping filter follows the recommendations established in *CCSDS 401.0-B-32* which dictates the use of band-limited modulation techniques (CCSDS 2021). In this case, there are two filtering options: a full raised cosine (RC) filter and a square-root raised cosine (SRRC) filter. RC filters provide zero inter-symbol interference by themselves, but utilising an SRRC filter allows for matched filtering to be employed. This is generally expected within digital communications systems because matched filtering (filtering at both the transmitter and receiver) generally allows for maximal signal-to-noise ratio (SNR) in the presence of additive white Gaussian noise (Spooner 2009).

This SRRC filter was designed and simulated in MATLAB. MATLAB produced the filter taps which could then be directly copied into the VHDL RTL. These taps were all 12 bits wide. Hence, the maximum amplitude of the modulator is $\pm 2^{12}$ instead of $\pm 2^{16}$. This is because the input data and the coefficients will be multiplied together, which could result in clipping if the amplitude of the input data is too great.

Table 28: Filter Design Choices and Justification

Filter Parameter	Value	Justification
Roll-Off Factor (α)	0.35	Balance between spectral efficiency and implementation complexity. Higher values will increase the bandwidth, lower values will increase the filter sharpness, which will require greater FPGA resources. The relationship between bandwidth and roll-off factor is represented by: $B = R_s(1 + \alpha)$
Samples per Symbol (SPS)	4	Matches with the 25 MSPS at 100 MHz clock rate to enable correct interfacing with the AD9361.
Filter Span	10 symbols	Ensures sufficient truncation of impulse response with sensible FPGA resource usage.
Number of Taps	41	Derived from: $N = span \cdot sps + 1$

The impulse response is as expected for an SRRC filter. It shows that the calculated filter taps are symmetrical around a centre tap. This confirms the filter's behaviour as linear-phase finite impulse response (FIR) filter. This means that it is not expected to introduce undue phase distortion.

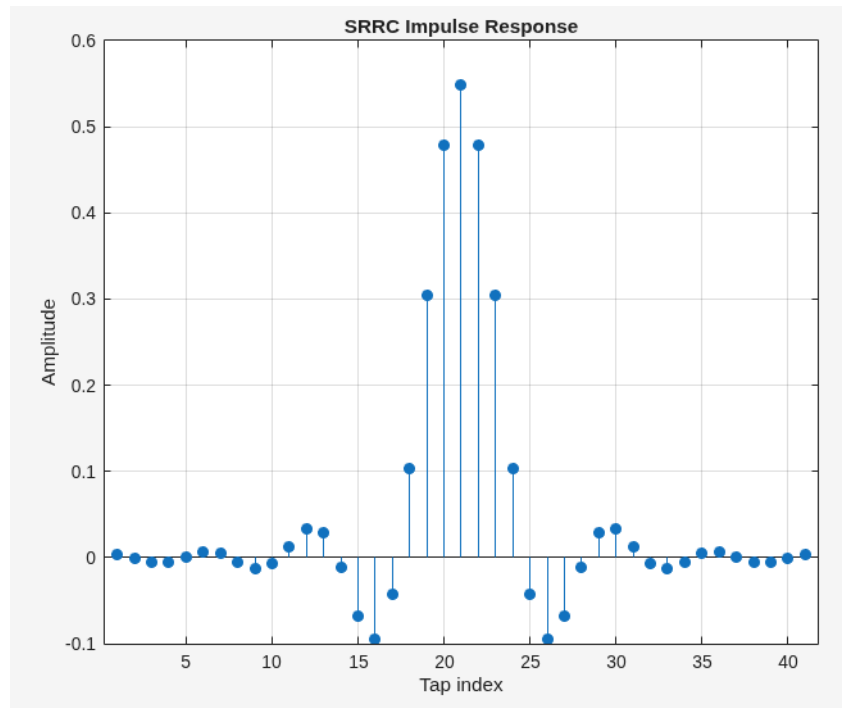


Figure 28: SRRC Filter Impulse Response (MathWorks 2026)

The magnitude response again demonstrates the expected behaviour. It appears to be a low-pass filter with a relatively flat passband. The flatness is desired because it will reduce any amplitude distortion. There is then a smooth transition between the passband and the stopband with no sharp transitions that could again add distortion. Significant attenuation is achieved in the stopband, with sidelobe attenuation often below -40 dB.

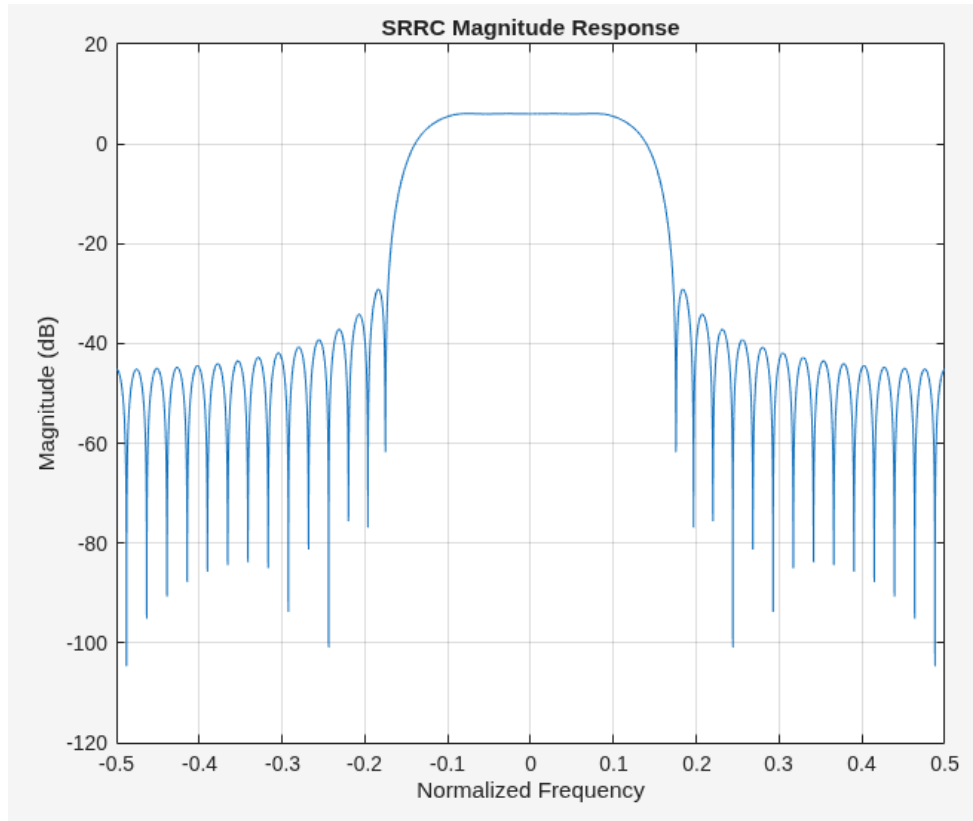


Figure 29: SRRC Filter Frequency Response (MathWorks 2026)

6.2 Custom IP Generation and Implementation

6.2.1 Summary of IP Cores Design and Implementation

The next stage in the implementation is to create the custom IP blocks required to realise the developed transmit chain. It was decided that a modular design approach should be adopted, where each functional aspect of the transmitter had its own IP core. This makes for much easier debugging and verification, because each block can be developed and tested independently. It also provides a clear separation of overall functionality, improving the clarity and structure of the overall system architecture.

The below figure displays the custom IP blocks (in green) within the wider AD9361 reference design. In total, 18 additional IP blocks were added to the reference design. Of these, 15 were created from scratch, and 3 were Constant blocks from the Vivado IP catalogue. *Appendix 5* contains a table that lists the functions of all 18 IP cores.

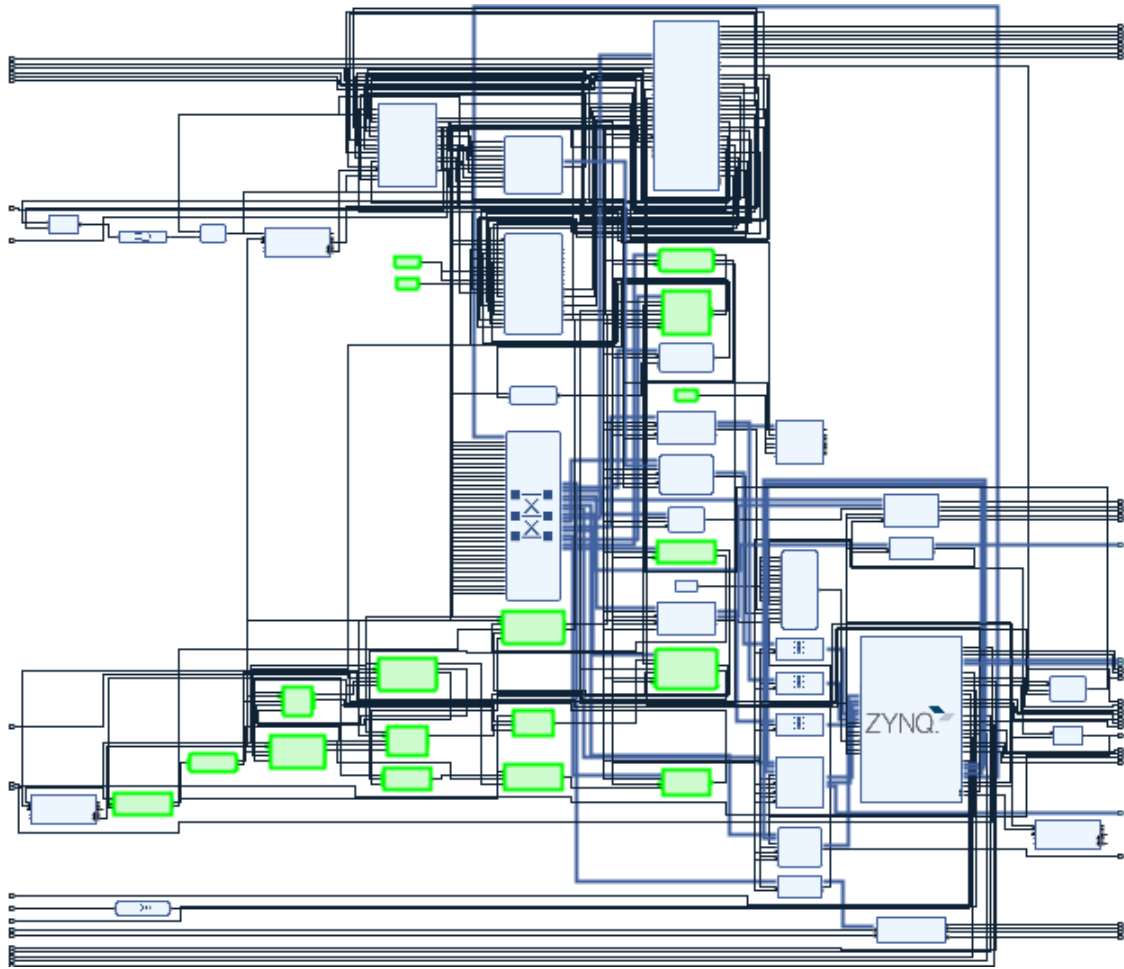


Figure 30: Custom IP Blocks Within AD9361 Reference Design (AMD 2022)

6.2.2 Choice of AXI Protocol

Several AXI interface options were considered to permit integration within this Zynq-based architecture. Ultimately, AXI4-Lite was selected for the user control path because it enables straightforward memory-mapped register access from the processing system (AMD 2023). AXI4-Lite is well-suited to low bandwidth interactions with small registers. This allows for user configuration of key transmitter parameters. Runtime control was achieved using the *devmem* utility, a lightweight user-space tool that allows the user access to the physical address space.

AXI4-Stream was considered for implementation within the data path. AXI-Stream supports high-throughput data streaming between IP blocks (AMD 2023). However, the custom transmit chain uses deterministic sample-based timing, based on valid and enable signals. This is a simpler scheme and therefore was easier to test and implement. In this case, the data rates were not considered to be so high that AXI-4 Stream was required.

6.2.3 Overall Implementation of AXI4-Lite Protocol

The following five custom IP blocks contain the AXI4-Lite interface:

- PRBS23
- Modulation Mapper
- Symbol Timer
- SRRC Filter
- Transmit Controller

As a result, these IP cores are all connected to the AXI Interconnect within the AD9361 design.

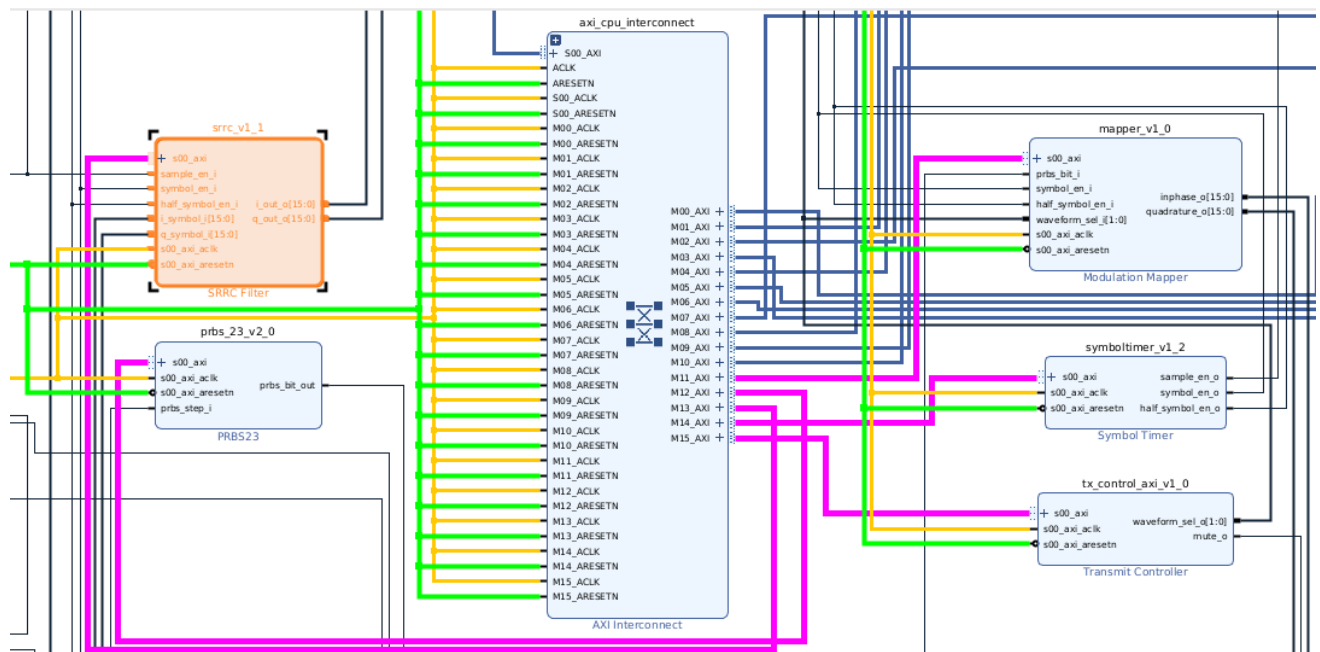


Figure 31: AXI Interconnect Connections (AMD 2022)

In this case, the AXI Interconnect serves as the intermediary between the AXI4-Master (Processing System of the Zynq) and AXI4-Slaves (five custom IP blocks).

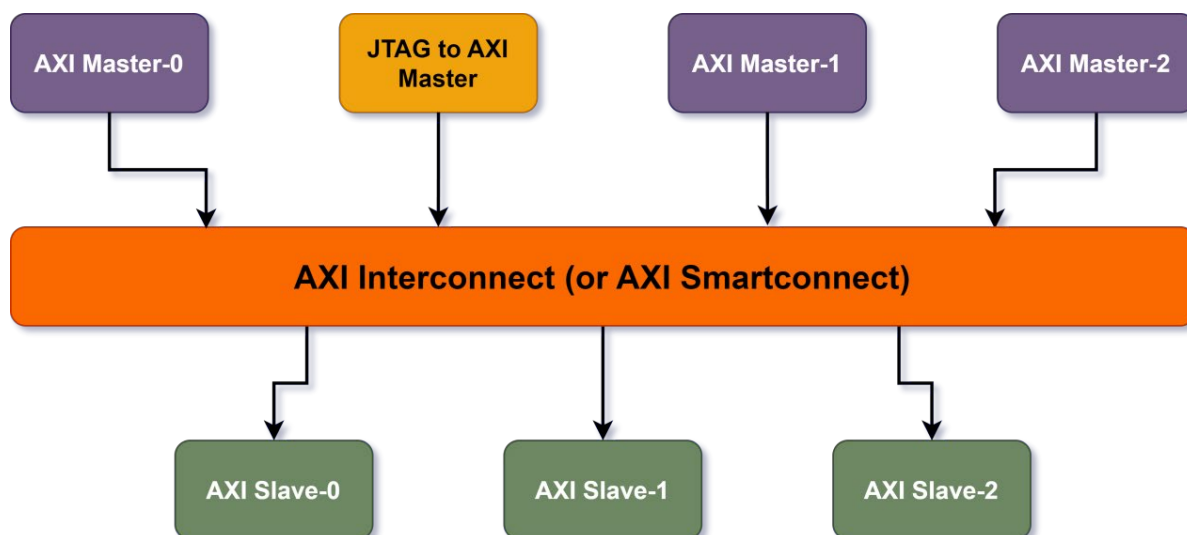


Figure 32: AXI Interconnect Architecture (Esergün 2024)

6.2.4 AXI4 Registers Function

Every AXI4 peripheral has a set of control registers that permit the runtime configuration of the system. They are accessed using memory-mapped addresses from the processing system and directly control the behaviour of the IP cores within the transmitter hardware. The full list of registers along with their respective purposes and addresses are listed in *Appendix 6*.

These registers are broadly responsible for the following functions:

- Enabling the PRBS23, Symbol Timer, Modulation Mapper, SRRC Filter and Transmit Controller IP cores.
- Runtime configuration of waveform selection (beacon, telecommand, telemetry and FHSS)
- User-controlled muting of the transmitter output
- Exposing the internal states and status signals, including the LFSR internal state, to enhance debugging and verification

6.2.5 Integration of Custom Transmit Chain into AD9361 Reference Design

To incorporate the custom transmit chain into the wider AD9361 reference design, modifications had to be made to the original transmit chain. The original transmit chain involves four distinct stages. Those marked in purple run in the 61.44 MHz clock domain and those in orange operate within the 245.76 MHz clock domain. The custom transmit chain is exclusively connected to the AXI slave clock at 100 MHz.

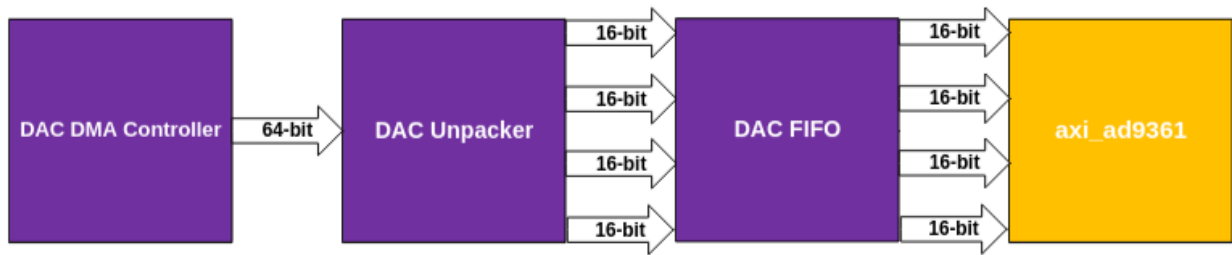


Figure 33: AD9361 Reference Diagram Transmit Chain

The custom transmit chain provides two separate 16-bit outputs (I and Q) and generates its own data. Therefore, the DMA (which streams data from memory) is not required anymore. The unpacker is also not required as the custom transmit chain can output data in a format that is already acceptable to the DAC FIFO. Hence both the DMAC and unpacker were disconnected from the rest of the wider design. The input enable pins on the unpacker were held 0 to prevent errors during the design validation in Vivado.

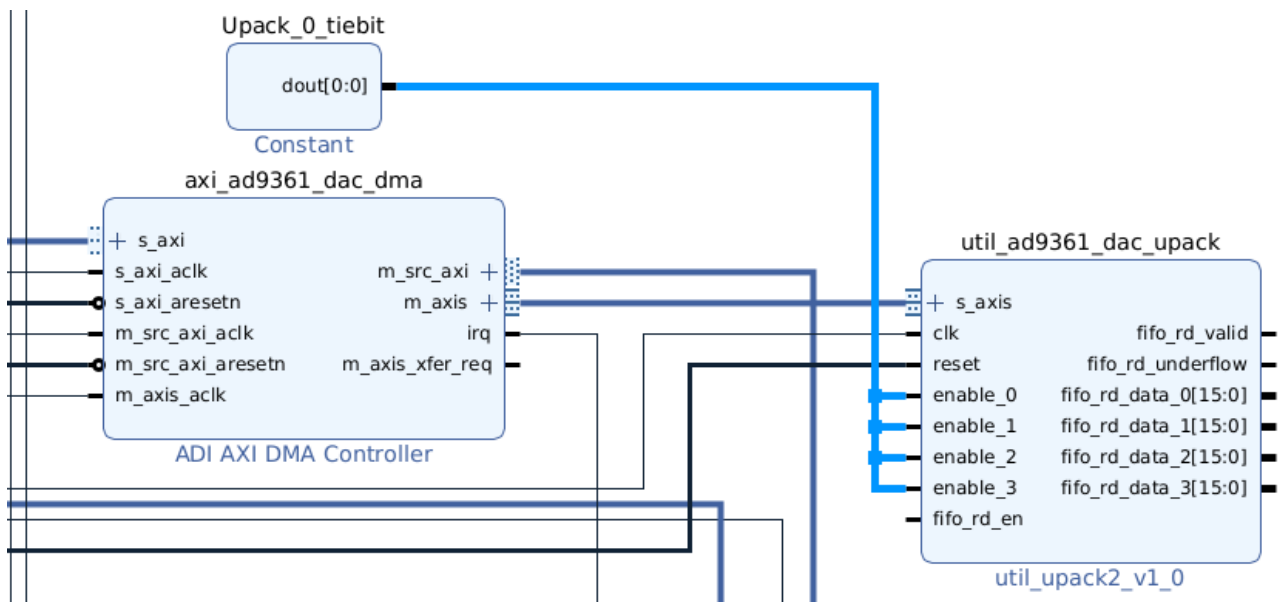


Figure 34: Disconnected Unpacker (AMD 2022)

Since the custom transmit chain operates on a different clock domain to the *axi_ad9361* IP core (which connects the FPGA fabric to the AD9361 RF transceiver), cross-domain clocking is now required. It would have been possible to get around this by using the AXI slave clock for only the user configuration and using the AD9361 clock for the data transmit chain itself. However, the custom IP cores had already been developed with the assumption that they would run on the AXI slave clock, so it would have introduced additional complexity if new clock ports had to be added. In addition,

IP cores that had previously passed timing closure may have been unsuitable because they are now running on a different clock rate.

Therefore, the chosen solution was to perform the cross-domain clocking using an asynchronous FIFO. It would have been possible to develop an asynchronous FIFO IP core from scratch. However, it was possible to rewire the DAC FIFO to accept the AXI slave clock as input and output the data at the AD9361 clock rate.

This unpacker block would have output a *valid* signal into the FIFO. This is replaced by the sample strobe from the Symbol Timer IP block. This strobe sets a sample rate of 25 MSPS. The first two data channels on the FIFO are fed using the outputs of the SRRRC filter. The other two are held zero by a Constant IP. The input clock on the FIFO is fed from the AXI slave clock (as opposed to the original 61.44 MHz clock). The output clock is the *axi_ad9361* clock.

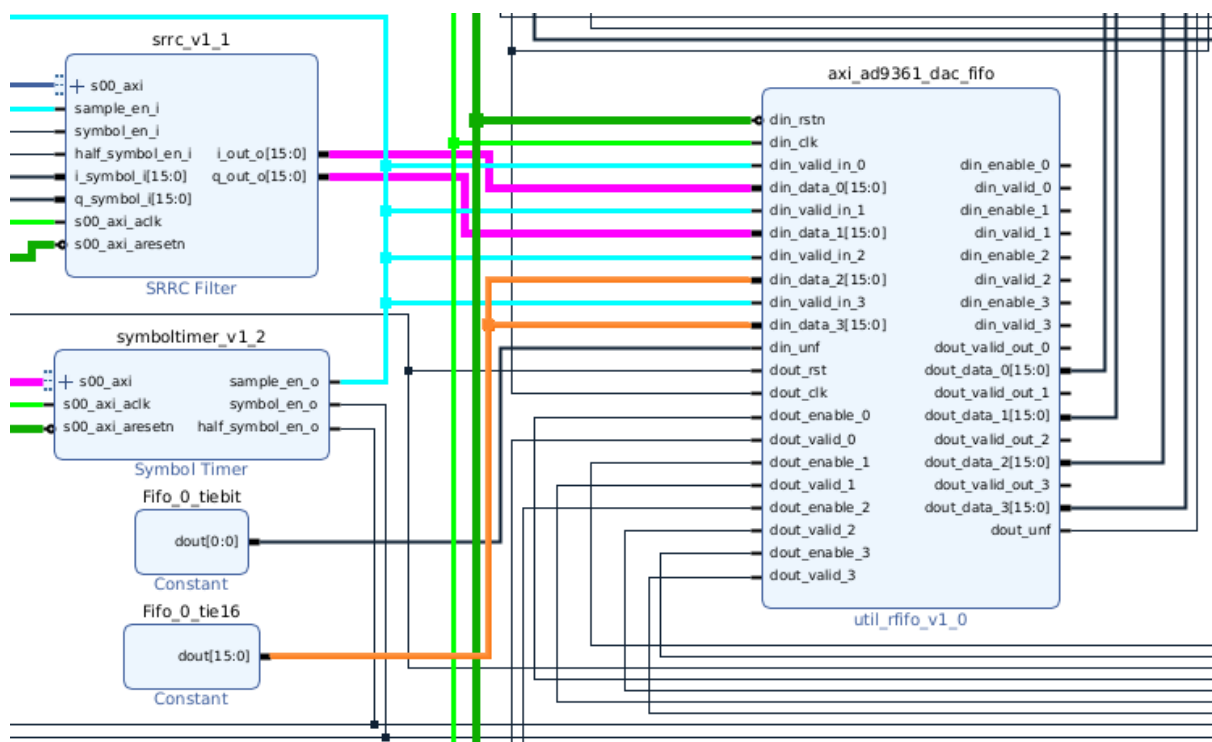


Figure 35: Rewired DAC FIFO (AMD 2022)

The AD9361 has two transmit ports but this SDR transmitter only uses one. Hence, the *axi_ad9361* core has two sets of DAC data input ports, but only those highlighted in purple are used. The other two could be disconnected but have not been to prevent design validation errors. “Enable” and “valid” flow control signals are used between the FIFO and *axi_ad9361*. They are connected in their original configuration as there was no need to amend this part of the reference design.

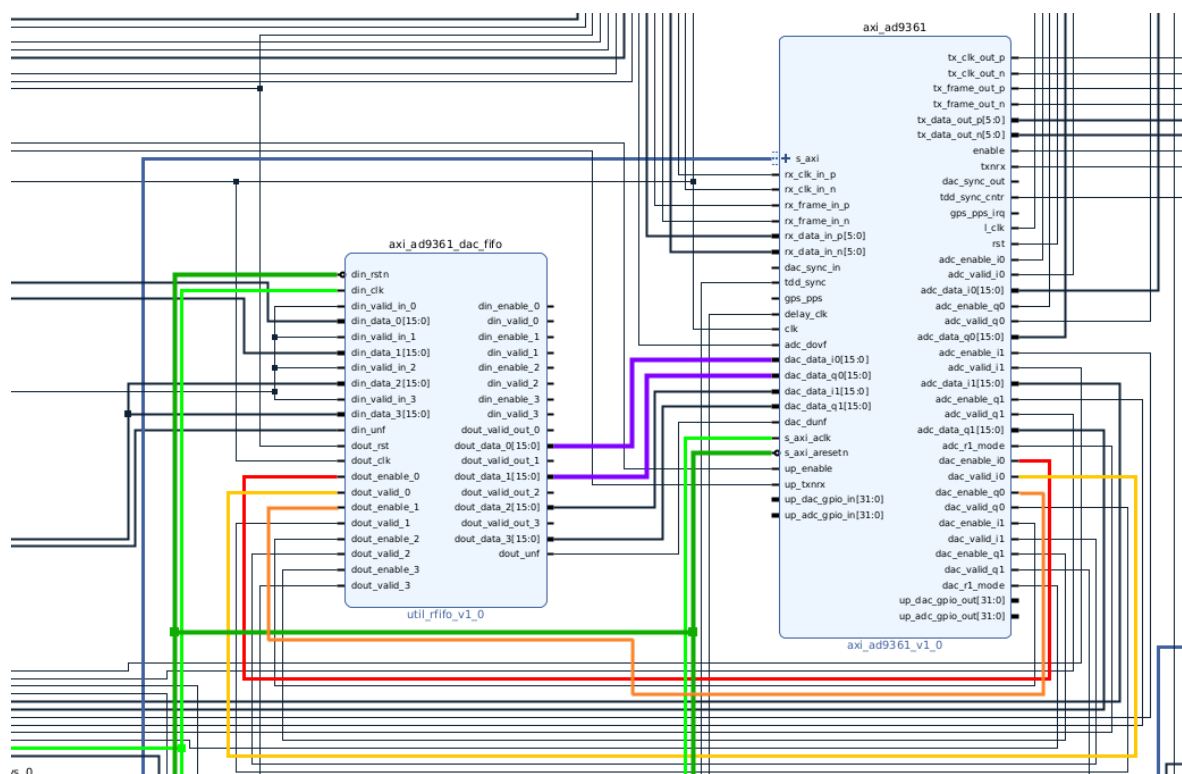


Figure 36: DAC FIFO and axi_ad9361 Connections (AMD 2022)

This integration required a degree of design creativity because there was a distinct lack of documented methods for incorporating custom transmit chains into the wider AD9361 reference design.

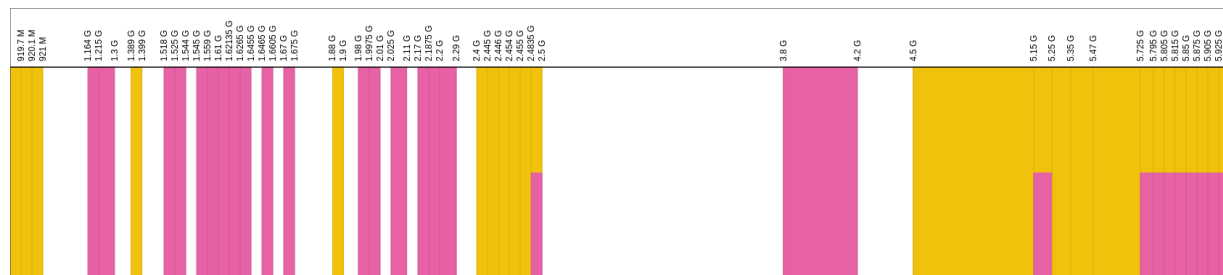
6.3 Frequency Hopping Implementation

FHSS was ultimately implemented by employing the *Industrial I/O (IIO)* subsystem to write new frequency values to the transmit LO control attribute. At runtime a frequency is selected in a pseudorandom manner. A dwell time is set to allow the AD9361 PLL to settle properly. This is all encapsulated within a single Bash script.

Table 29: FHSS Hopping Table

Frequency Number	Frequency (GHz)
1	2.4
2	2.42
3	2.44
4	2.46
5	2.48

The 2.4 to 2.48375 GHz range was chosen to encapsulate the range of frequencies during FHSS operation. This range was selected because of its license-free status. This range is sufficiently wide range for the frequency hopping to be visible on a spectrum analyser.



2.4835 - 2.5 GHz

Land Mobile-Satellite Service Stations (2.4835 - 2.5 GHz)

Show table

Figure 37: Spectrum Map (Ofcom 2026)

6.4 Software-Based User Control and Configurability

Ultimately, a simple command-line interface (CLI) allows the user to control and configure the SDR transmitter at runtime. The commands are issued over a UART interface and uses user-space tools such as *iio_attr*, *iio_info* and *busybox devmem*. Such tools allow direct access to the AD9361 configuration and the memory-mapped AXI4-Lite registers within the FPGA [see Appendix 6].

The *.bashrc* was amended to include the set of user-friendly aliases displayed below.

Table 30: User Commands

User Command Alias	User Command Operation
start	1) Mutes the transmitter. 2) Initialises the following: • PRBS • Modulation Mapper • Symbol Timer • SRRC Filter • Transmit Controller 3) Selects beacon waveform. 4) Unmutes the transmitter.
stop	1) Mutes the transmitter. 2) Disables all AXI-controlled blocks.
restart	1) Runs stop command. 2) Runs start command.
select_waveform	4 options available: • <code>select_waveform telecommand</code> • <code>select_waveform telemetry</code> • <code>select_waveform beacon</code> • <code>select_waveform fhss</code> If <code>select_waveform fhss</code> is used, then the frequency hopping script (<code>fhss.sh</code>) is initiated.
mute_on	Mutes the transmitter
mute_off	Unmutes the transmitter
status	Displays the following information to the user: • Selected waveform name • Output frequency • Coding scheme

	• Modulation type • Filtering type • Symbol rate • Sample rate • Data rate
select_power	The user can select one of the three power levels: low, medium and high.

6.5 Testing Overview

6.5.1 Planned Testing Phases

Testing was an iterative process and happened throughout the development of the transmitter. This allowed the SDR transmitter's functionality to be verified from the individual HDL modules right up to full-system operation. This reduces the integration risk and allows faults to be isolated at the lowest possible level.

Table 31: Test Strategy Phases

Test Phase Number	Test Phase Name	Purpose	Tools/Methods Used
1	Module-Level Testing	Verify functionality of all HDL modules part of transmitter chain (PRBS, Modulation Mapper, Symbol Timer, both framers and both encoders).	Vivado testbenches (developed using ChatGPT 5.2) MATLAB (for filter)
2	Hierarchical Transmitter Testing (Pre-IP Integration)	Validates full transmit chain behaviour within a hierarchical design before the creation of the IP cores and integration of AXI.	Vivado testbenches (developed using ChatGPT 5.2) Vivado Simulation for waveform analysis
3	AXI/IP Integrated Testing	Verifies that the transmitter chain functions correctly after instantiation into independent IP cores and integration of AXI4-Lite interface. Demonstrates that user-space tools (like <i>busybox devmem</i> , <i>iio_attr</i> and <i>iio_info</i>) can be used to alter system parameters. A bitstream must be generated first.	Vivado Integrated Logic Analyser (ILA) KuiperLinux CLI
4	Full AD9361 System Integration	Validates end-to-end operation of the transmit chain once the custom IP cores had been fully integrated into the AD9361 reference design. Demonstrates that DAC FIFO wiring is correct.	Vivado Integrated Logic Analyser (ILA) KuiperLinux CLI

		Demonstrates that user command aliases before as expected.	
5	RF Validation Testing	Assesses the RF behaviour for each waveform.	Spectrum analyser GNU Radio COTS Modem

6.5.2 Full System Evaluation Setup

The setup below was used for the full RF testing of the system. This includes the FPGA and AD9361 the development laptop running KuiperLinux, the test laptop running GNU Radio, the COTS modem (only the top one is used), a spectrum analyser (only the one on the left is used), and an RF splitter to allow simultaneous use of the COTS modem and spectrum analyser.



Figure 38: Full System Evaluation Setup

7 ISSUES ARISING FROM IMPLEMENTATION AND TESTING

This section outlines the most significant challenges encountered during implementation and testing.

7.1 FPGA Bitstream Generation and Deployment Issues

After the first Synthesis, Implementation and Bitstream generation, an issue was encountered when attempting to transfer the new bitstream onto the *ZedBoard*. Often, Vivado can export the bitstream directly to the FPGA through a JTAG port. It does this through its Auto Connect tool within the Hardware Manager.

However, when this was attempted, Vivado could not seem to find a target to connect to, even when the board was connected to the development laptop.



Figure 39: Failure to Connect to Board in Hardware Manager (AMD 2022)

After reviewing the Digilent ZedBoard User Guide, it became clear that the FPGA could boot using several devices including JTAG, SD Card and Quad-SPI (Digilent 2012). By default, it boots using the SD Card. It was decided that instead of reconfiguring the board so that it boots using JTAG, flashing the SD Card with the new bitstream was preferable. This involves using the *bootgen* to create a new BOOT.BIN file. *Bootgen* combined multiple components (First-Stage Boot Loader, FPGA bitstream and U-Boot). The new BOOT.BIN would then be copied onto the SD Card. The SD Card was then reinserted into the FPGA and could be booted to load the new hardware configuration.

```

vc@dev:~/viv_project/hdl/projects/fmcomms2$ cd zed
vc@dev:~/viv_project/hdl/projects/fmcomms2/zed$ ls
BOOT.BIN          fmcomms2_zed.xpr      timing_synth.log
bootgen_sysfiles.tgz fsbl.elf              u-boot_zynq_zed.elf
fmcomms2_zed.cache ip_upgrade.log        vivado_369050.backup.jou
fmcomms2_zed.gen   Makefile              vivado_369050.backup.log
fmcomms2_zed.hw    mem_init_sys.txt      vivado_386489.backup.jou
fmcomms2_zed.ip_user_files system_bd.tcl          vivado_386489.backup.log
fmcomms2_zed.runs  system_constr.xdc     vivado.jou
fmcomms2_zed.sim   system_project.tcl    vivado.log
fmcomms2_zed.srscs system_top.bit         vivado_pid179247.str
fmcomms2_zed.tmp   system_top.bit.bin    vivado_pid386489.str
fmcomms2_zed_vivado.log system_top.v           zynq.bif
vc@dev:~/viv_project/hdl/projects/fmcomms2/zed$ bootgen -arch zynq -image zynq.bif -o i BOOT.BIN -w

***** Xilinx Bootgen v2022.2
**** Build date : Feb 7 2023-10:06:59
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

[WARNING]: Partition fsbl.elf.0 range is overlapped with partition system_top.bit.0 memory range
[INFO] : Bootimage generated successfully
vc@dev:~/viv_project/hdl/projects/fmcomms2/zed$

```

Figure 40: Use of bootgen in command line

This issue was resolved within one day of work, so there were not any significant deviations from the project schedule.

7.2 Inconsistencies in AXI Block Instantiation

This second issue became clear because ILA analysis of the Symbol Timer revealed that the output symbol and half-symbol strobes were always low instead of pulsing on every symbol and half-symbol period respectively.

When editing the IP in Vivado, an elaborated design schematic can be produced. From this schematic, it became clear that the output ports were not fully connected within the packaged IP.

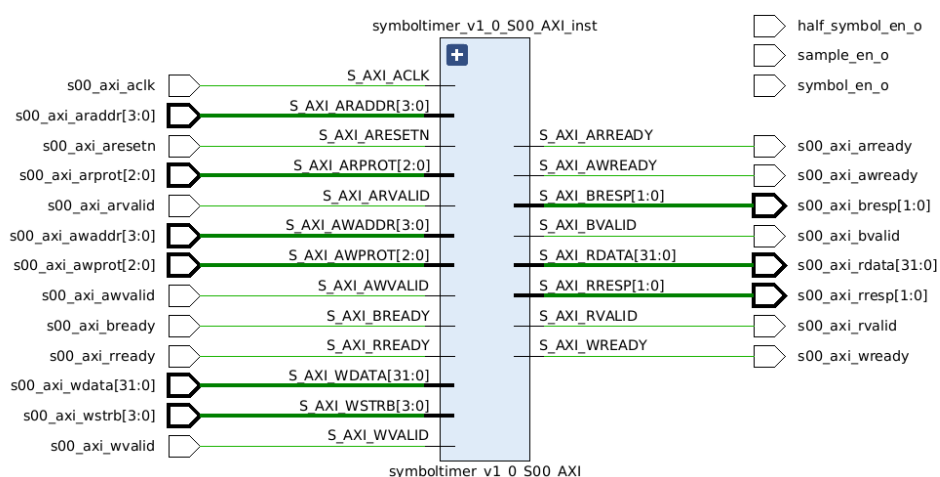


Figure 41: Defective Symbol Timer Elaborated Design Schematic (AMD 2022)

Inspection of the IP hierarchy clarified that the user logic was instantiated in the AXI slave wrapper (`symboltimer_v1_0_S00_AXI.vhd`) but not connected to the architecture of the top-level wrapper (`symboltimer_v1_0.vhd`). As a result, the output

signals existed only as declared ports at the top level and were not driven by any internal logic.

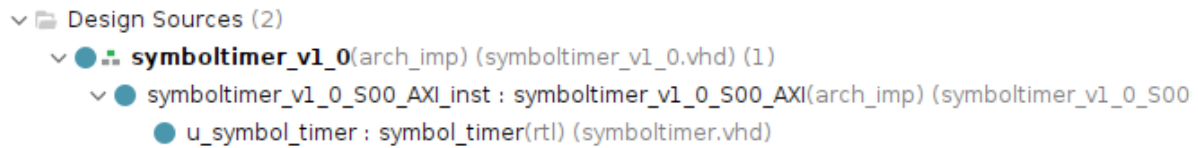


Figure 42: AXI Slave File Hierarchy (AMD 2022)

This was corrected by amending the top-level wrapper to ensure that the connections were made between these outputs and the user logic. Reproducing the elaborated design schematic demonstrated that the outputs were now correctly connected throughout the IP core.

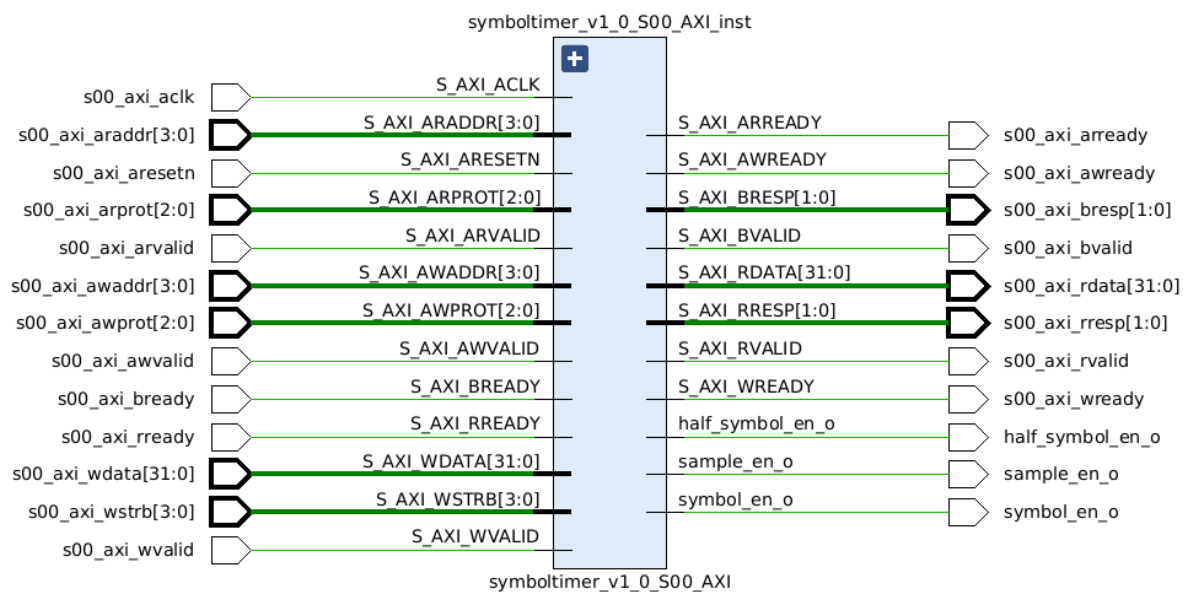


Figure 43: Corrected Symbol Timer Elaborated Design Schematic (AMD 2022)

This fault was detected during the integration of AXI4-Lite into the applicable custom IP cores. As a result, the other AXI-controlled IP cores were checked, and this issue was rectified before additional faults occurred. This helped to contain any further delays to the schedule.

7.3 Timing Closure Failure During SRRC Filter Integration

This issue arose when the SRRC Filter IP was integrated into the wider custom transmit chain. Initially, the Synthesis completed successfully, but the Implementation was completed with very significant timing closure failures.

Timing	Setup Hold Pulse Width
Worst Negative Slack (WNS):	-3.915 ns
Total Negative Slack (TNS):	-238.385 ns
Number of Failing Endpoints:	120
Total Number of Endpoints:	66055
Implemented Timing Report	

Figure 44: Initial Vivado Timing Report (AMD 2022)

Effectively, the worst path took 3.915 ns longer to propagate than permitted. This means that the design, even if a bitstream could be generated, would likely be highly unreliable.

Initially, the SRRC filter was implemented as a large compensational FIR filter consisting of 41 taps. The design was expected to complete the multiplication and then accumulation of all tap products within a single clock cycle. This resulted in long critical paths. As a result, the propagation delay through such paths were larger than the available clock period and therefore multiple setup violations occurred.

Therefore, three major design changes were made to the filter. Firstly, a pipelined architecture was adopted. This involves separating the FIR computation into multiple stages, each separated by registers. This implementation included seven pipeline stages. Introducing registers between each stage helped to reduce the critical path delay, because the design was expected to perform fewer combinational logic operations during a single clock cycle.

Secondly, the fact that the SRRC filter coefficients were symmetrical [see Section 6.1.6] was exploited to reduce the computational complexity. Symmetric pairs were added prior to multiplication, halving the number of required multipliers. This reduces resource utilisation and reduces the routing complexity.

Finally, a balanced adder tree structure was adopted for the accumulation of partial products. In a linear accumulation chain, the delay introduced is proportional to the number taps. In contrast, a balanced adder tree structure performs additions in parallel stages instead of sequentially.

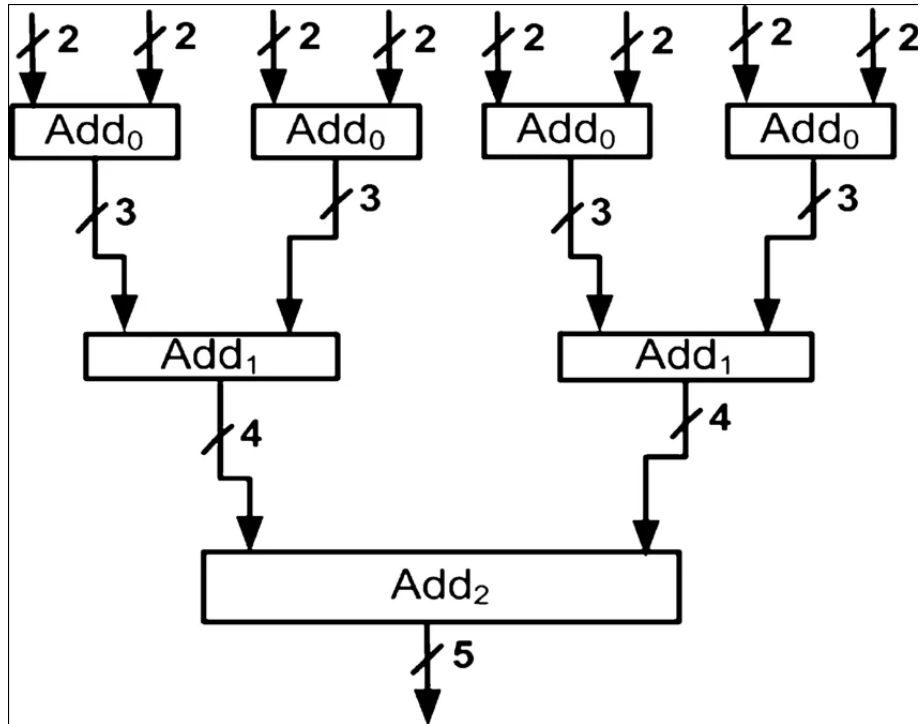


Figure 45: Balanced Adder Tree Structure Example (Memon et al 2013)

As a result of these design changes, the overall design with the SRRC filter integrated passed timing. This issue took several days to resolve but timing and implementation failures were deemed to be a moderate risk to the project (**R5**). Therefore, additional time was factored into the full integration phase to account for this.

7.4 AD9361 Sample Rate Mismatch

To function correctly, the AD9361 must be programmed to operate at a sample rate that is equal to the sample rate of the data entering the DAC FIFO. In software, *iio_attr* was used to set the sample rate equal to the expected value (25 MSPS). However, the output spectrum was not as expected. It was very noisy and disabling the SRRC filter had very little effect on the spectral plot. The bandwidth was also smaller than expected.

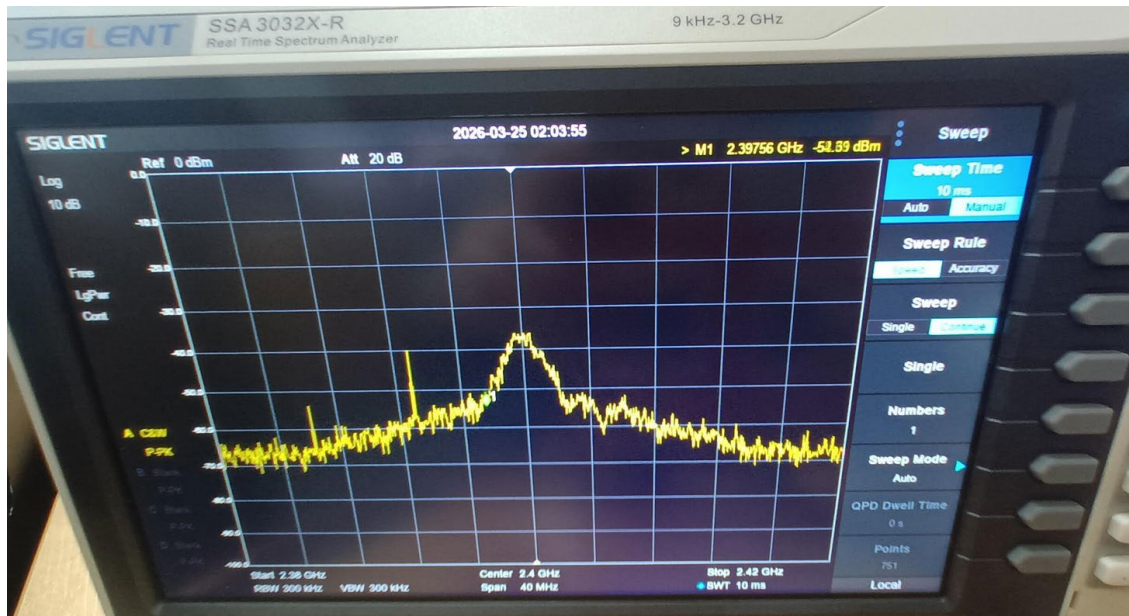


Figure 46: Spectral Plot Sample Rate Mismatch

It was identified that this unexpected spectral plot could be due to a sample rate mismatch. In the DAC FIFO, the *din_valid_in* signals must pulse at the expected sample rate. Originally, these signals were tied high continuously. Since *din_clk* runs at 100 MHz, the sample rates are mismatched (100 MSPS vs 25 MSPS). The Symbol Timer IP core outputs pulses at 25 MSPS, which rectified this mismatch as data was now being fed into the FIFO at 25 MSPS.

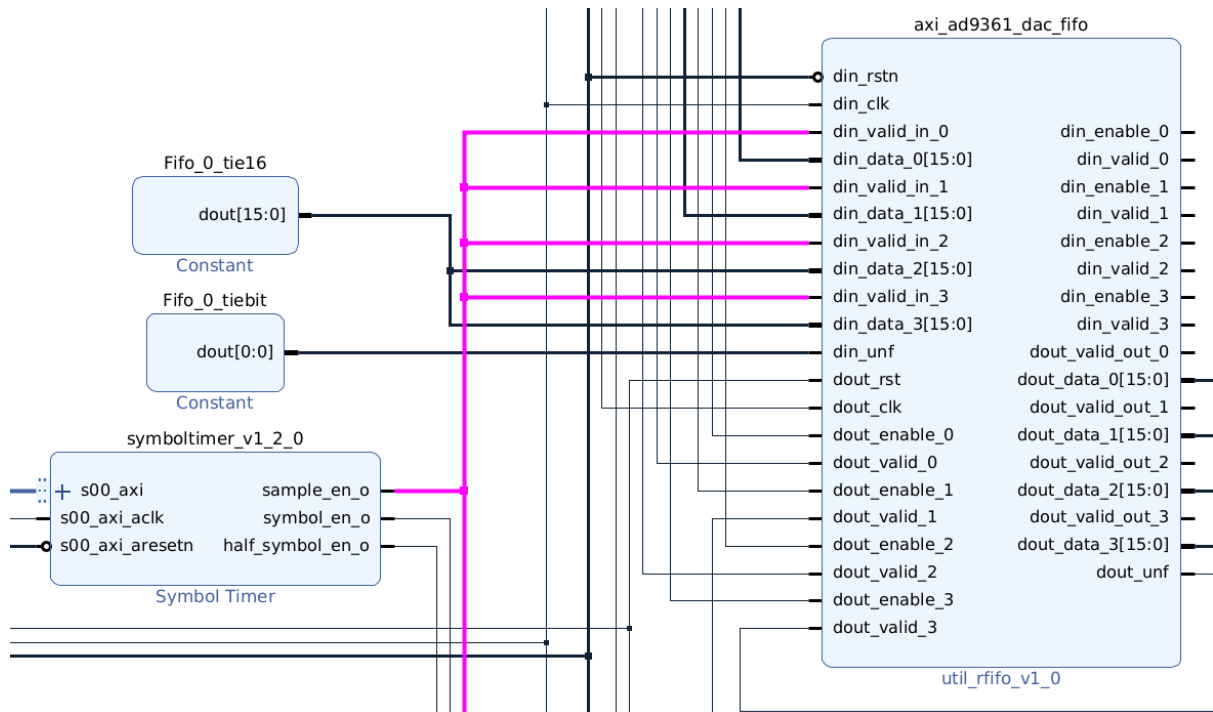


Figure 47: DAC FIFO Valid Signals Corrected Wiring (AMD 2022)

7.5 Appearance of Spurious Narrowband Emissions

The transmitter spectrum exhibits discrete spurious narrowband emissions. When the spectrum analyser sweep time is set to 1 millisecond, these emissions are not visible. However, when the sweep time is increased beyond 1 second, they become increasingly visible. At sweep times of around 1 second, these narrowband spurs increase in frequency, appearing to drift across the spectrum analyser screen. There was little difference in these spurious emissions between the filtered and unfiltered spectrum plots, so this issue is not caused by the SRRC filter. Adjusting the output frequency from 2.4 GHz to 1.2 GHz also made little difference, confirming that these were unlikely to be external emissions.

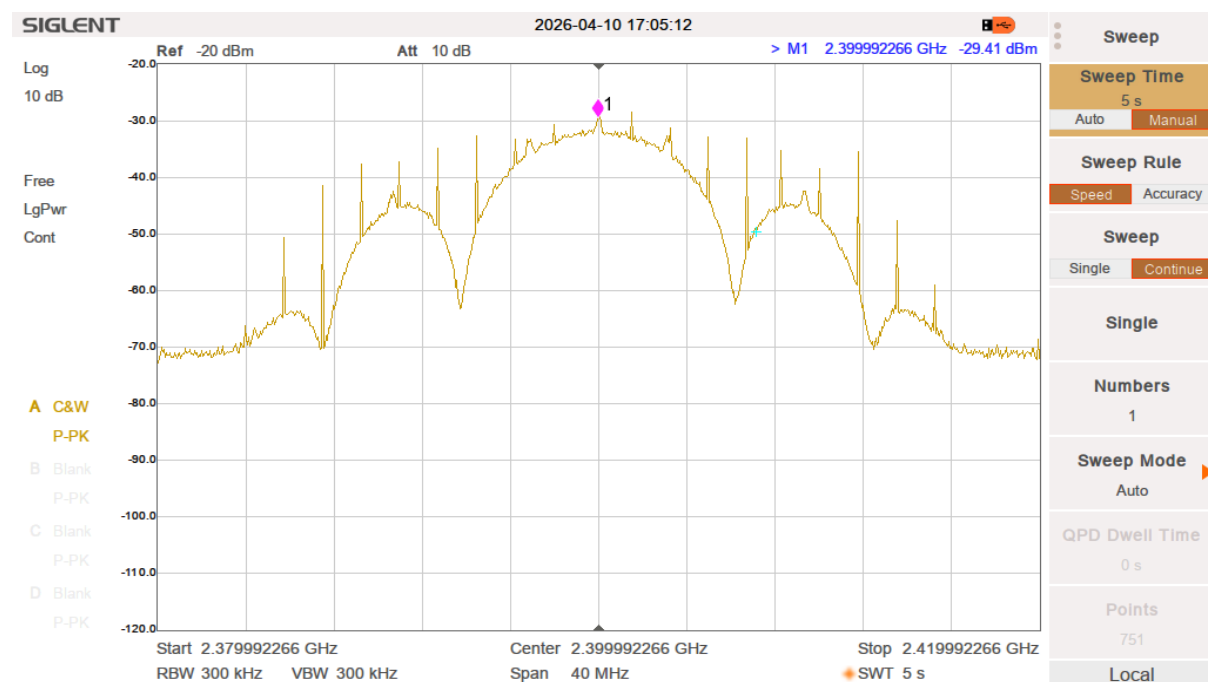


Figure 48: Unfiltered Waveform Spurs

The 3D spectrogram below further illustrates the nature of these emissions. The central region clearly shows the occupied bandwidth of the filtered waveform. This looks as expected. The surrounding regions exhibit a series of evenly shaped lines by frequency. The constant nature of these lines over time strongly suggests that these spurious emissions are deterministic and from the transmitter itself, as opposed to random noise. They may be due to the periodic repetition of the PRBS, or timing issues within the transmit signal chain. However, the stability of these features (and their absence under shorter sweep times) suggests that the transmitter is still capable of reliable operation [see Section 8.1] despite the lack of spectral purity.

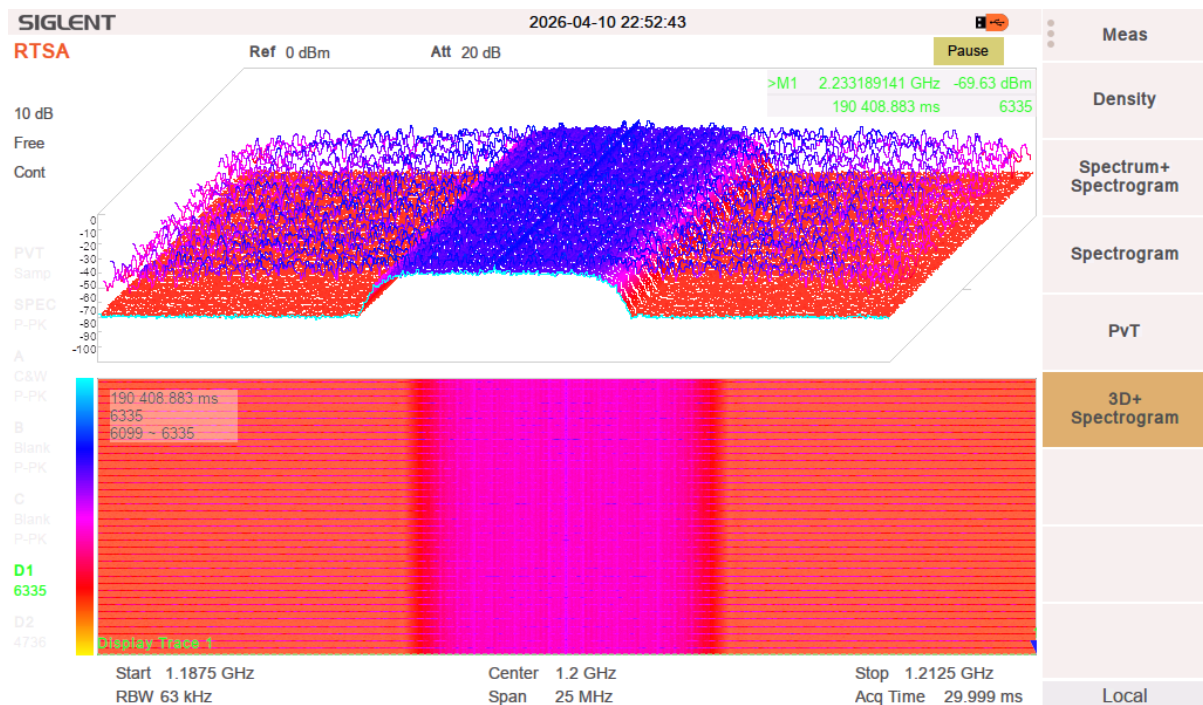


Figure 49 : 3D Spectrogram

7.6 GNU Radio and COTS Modem Symbol Rate Limitations

It was discovered during the full system testing that the symbol rate exceeded that which could be handled by GNU Radio. This was confirmed by using a COTS modem as a BPSK transmitter at a lower sample rate (around 6.25 MSPS) which produced a clean BPSK constellation. The spectrum analyser also produced a clean OQPSK constellation at 6.25 MSPS as GNU Radio was running simultaneously. Therefore, instead of amending the Symbol Timer RTL to provide data at a lower sample rate (4 samples per symbol must be maintained because of the SRRC filter), it was decided that the spectrum analyser should be used to obtain constellation of the output waveforms. This is because the spectrum analyser provided a much greater breadth of RF performance information, as well as eye diagrams and the spectral plot.

A similar issue was encountered with the COTS modem. The performance of the BPSK-modulated waveforms could be validated. However, since OQPSK effectively works at twice the data rate, the Comtech CDM-625 satellite modem would not permit a high enough data rate. This is because the data rate for OQPSK is twice that of BPSK. The modem would allow a data rate of 6.25 Mbits/s to be entered but not 12.5 Mbits/s

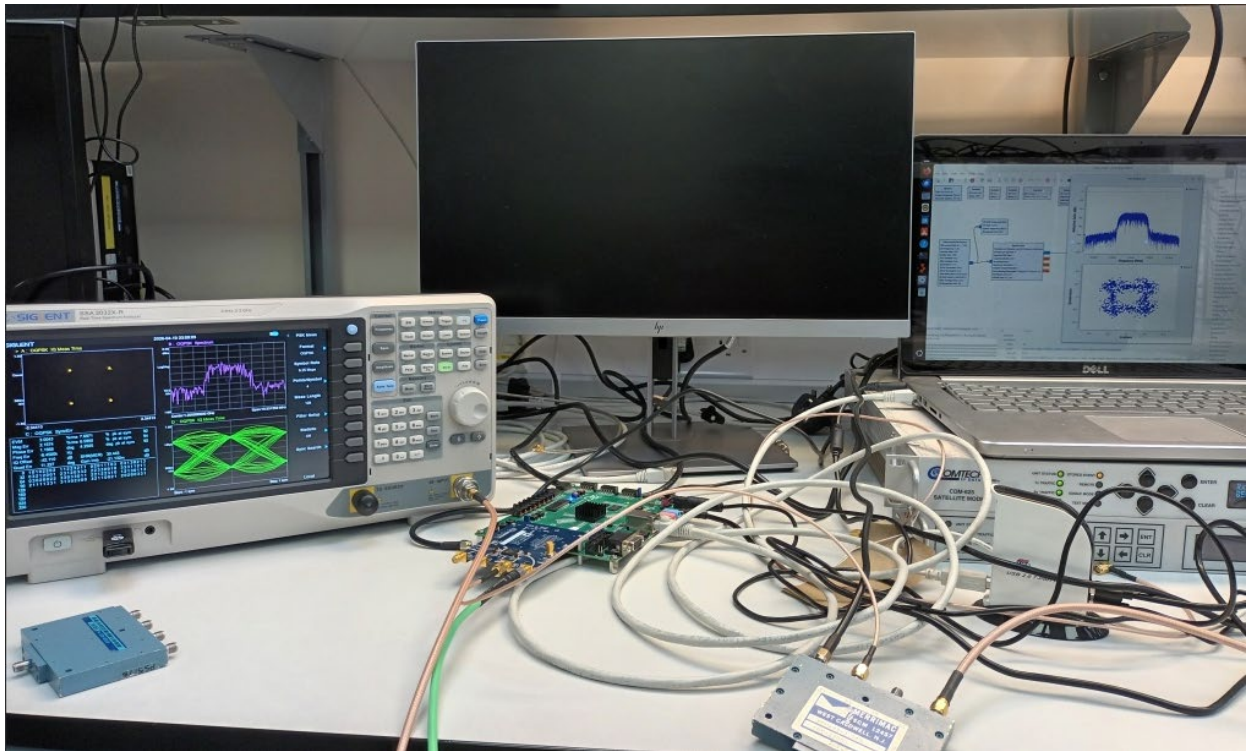


Figure 50: GNU Radio Symbol Rate Limitation

8 RESULTS

This section intends to summarise the results of Test Phases 4 and 5 [see Section 6.5.1]. This includes the RF system evaluation testing, as well as the use of an ILA to test the functionality of the CLI. It will also provide the Vivado-generated results for timing and resource usage.

8.1 RF System Evaluation

8.1.1 Filtered and Unfiltered Spectrum Plots

All four waveforms demonstrated practically identical spectrum plots regardless of whether the SRRC filter was enabled or bypassed. The use of the spectrum analyser also confirmed correct frequency hopping behaviour during the activation of the FHSS waveform.

Given that all waveforms operate at a symbol rate of 6.25 Msym/s (for both BPSK and OQPSK) and the roll-off factor of the SRRC filter is 0.35, an expected bandwidth can be calculated.

$$B = R_s(1 + \alpha)$$

$$B = 6.25(1 + 0.35)$$

$$B = 8.4375 \text{ MHz}$$

Each vertical division on the spectrum analyser represents 4 MHz. The spectrum plot shows that the bandwidth of the waveform is approximately this value.



Figure 51: Filtered SDR Transmitter Output

The below spectrum plot is the expected unfiltered BPSK spectrum plot. The first pair of sidelobes are located approximately 13 dB below the main lobe. The second pair of sidelobes are just about visible above the noise floor.

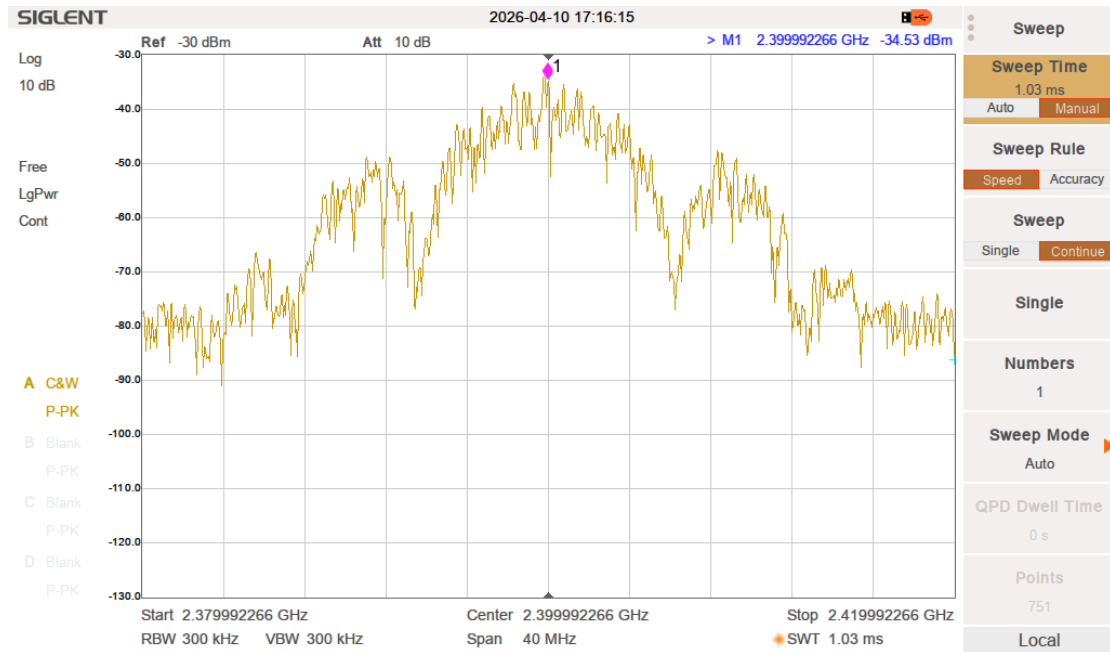


Figure 52: Unfiltered BPSK Spectrum

8.1.2 Constellation and Eye Diagram Waveform Comparisons

8.1.2.1 BPSK-Modulated Waveforms

All three of the BPSK-modulated waveforms display broadly similar behaviour here. All constellation plots display two tightly packed dots as expected. The SNR seems to be around 30 dB for all the waveforms. The eye diagrams all seem to be relatively open, which confirms good timing integrity and minimal inter-symbol interference.

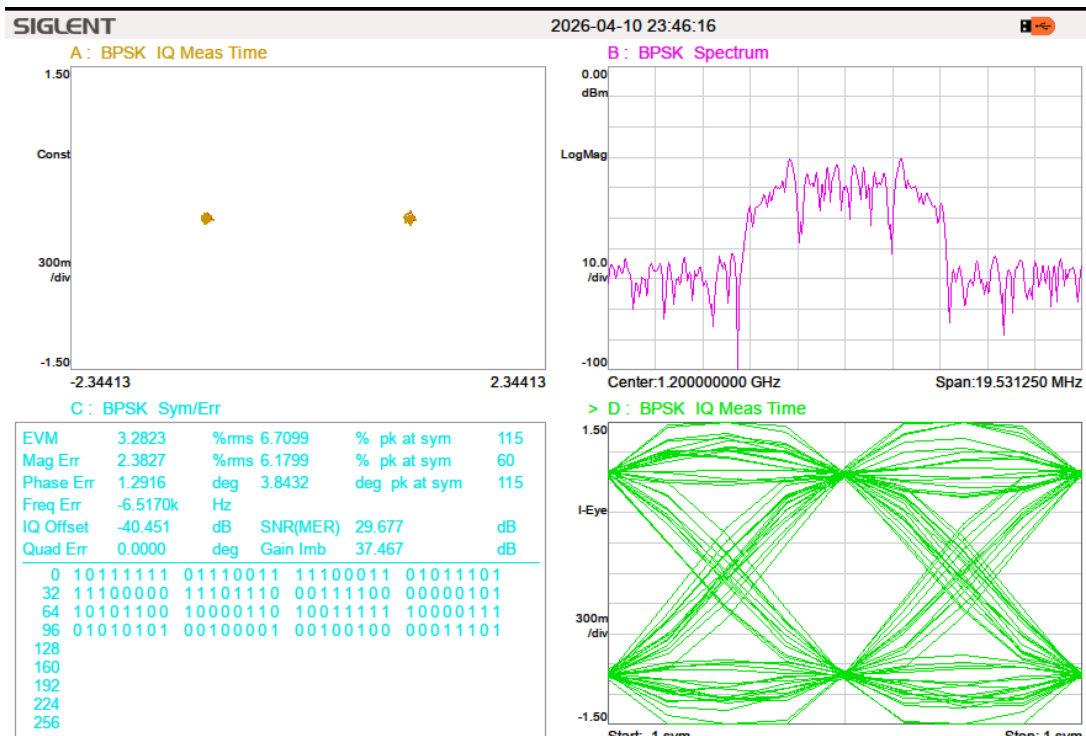


Figure 53: Beacon Waveform Constellation and Eye Diagram

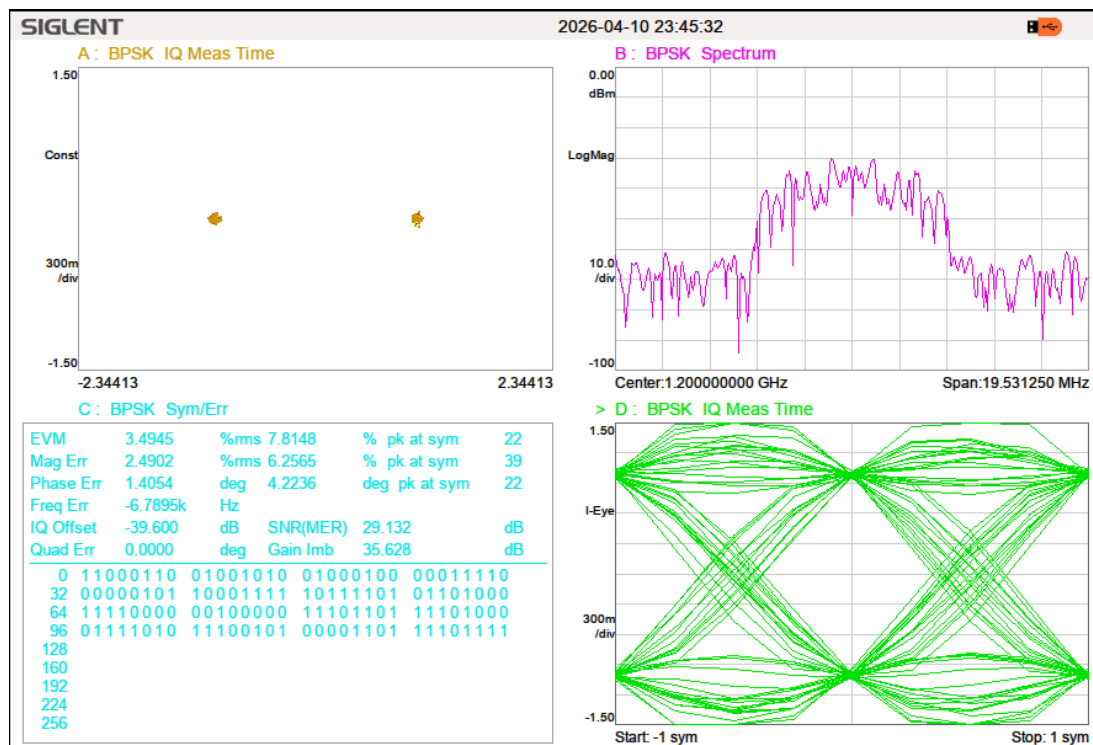


Figure 54: Telemetry (Category A) Waveform Constellation and Eye Diagram

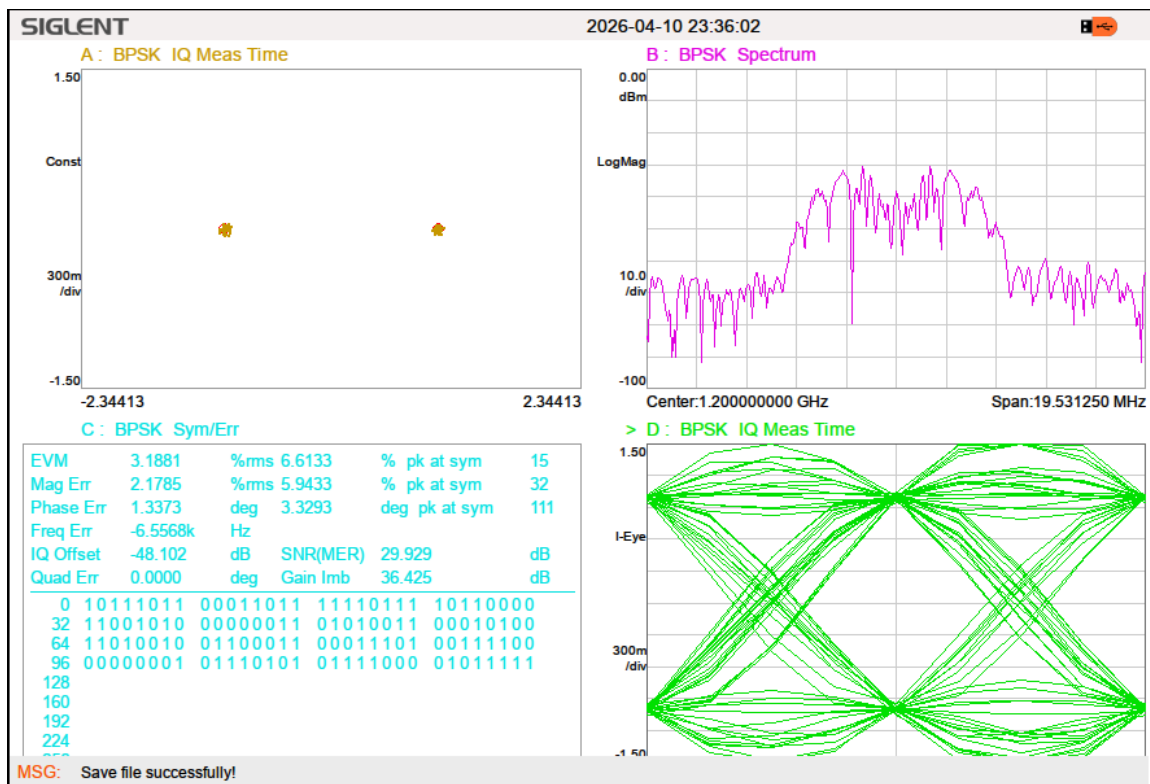


Figure 55: Telecommand Waveform Constellation and Eye Diagram

8.1.2.2 OQPSK-Modulated Waveform

The constellation exhibits, as expected, four tightly packed points. The eye diagram is relatively open, and the SNR seems to hover around 31 dB.

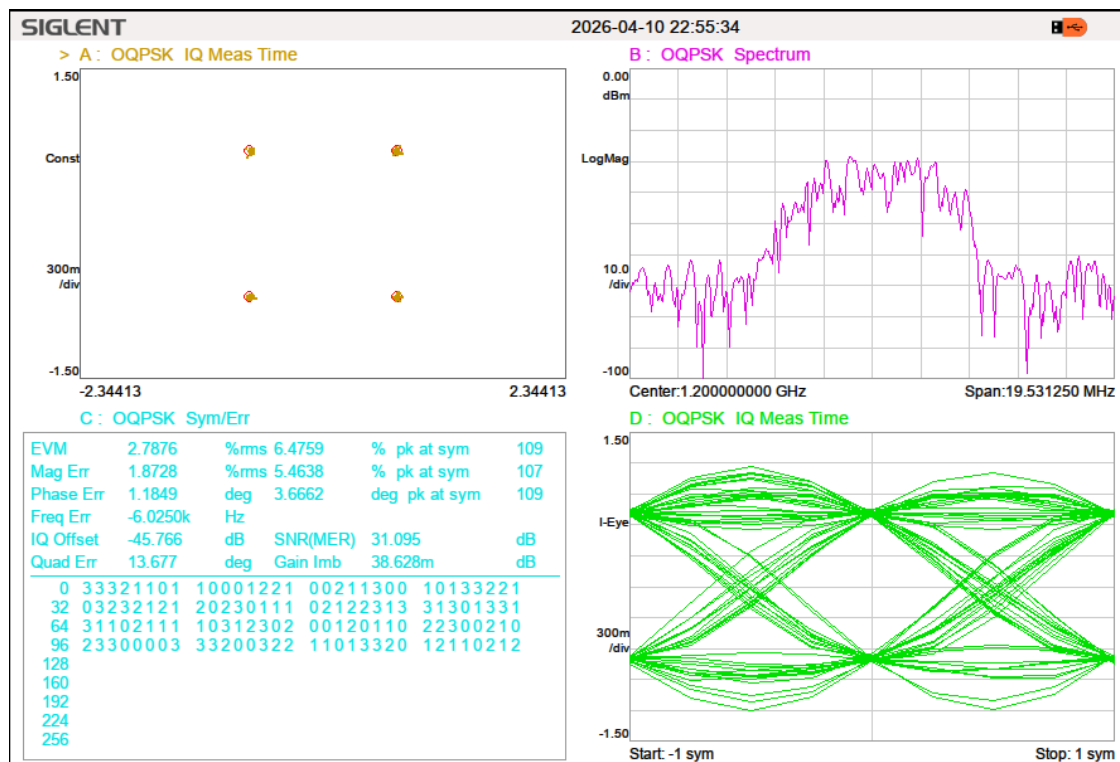


Figure 56: FHSS Waveform Constellation and Eye Diagram in OQPSK Mode

Correct I and Q symbol timing was confirmed by using the QPSK demodulation function on the spectrum analyser instead of the OQPSK demodulation function. The constellation no longer consisted of four tightly packed groups.

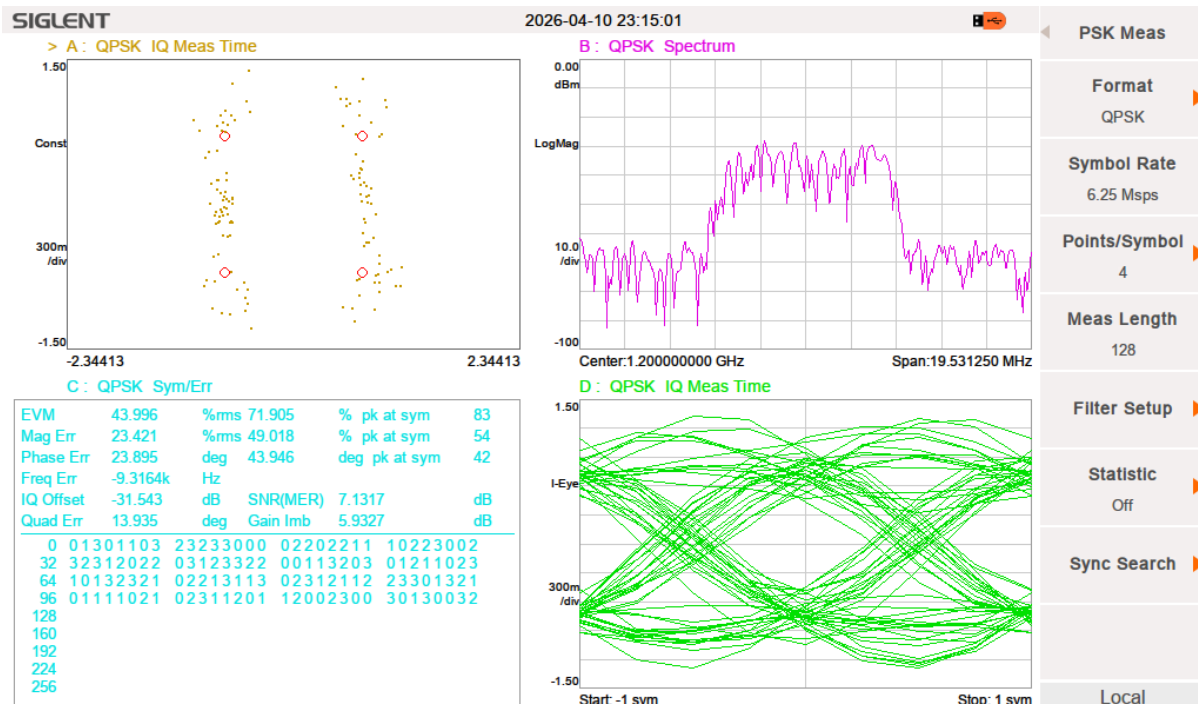


Figure 57: FHSS Waveform Constellation and Eye Diagram in QPSK Mode

8.1.3 Beacon Waveform COTS Modem Testing

Only the beacon waveform was tested using the COTS modem. The E_b/N_0 exceeds 16 dB resulting in a BER of 10^{-9} or lower. This matches the theoretical expectation. It also suggests that the link is of high quality.



Figure 58: Beacon E_b/N_0 and BER COTS Modem Measurements

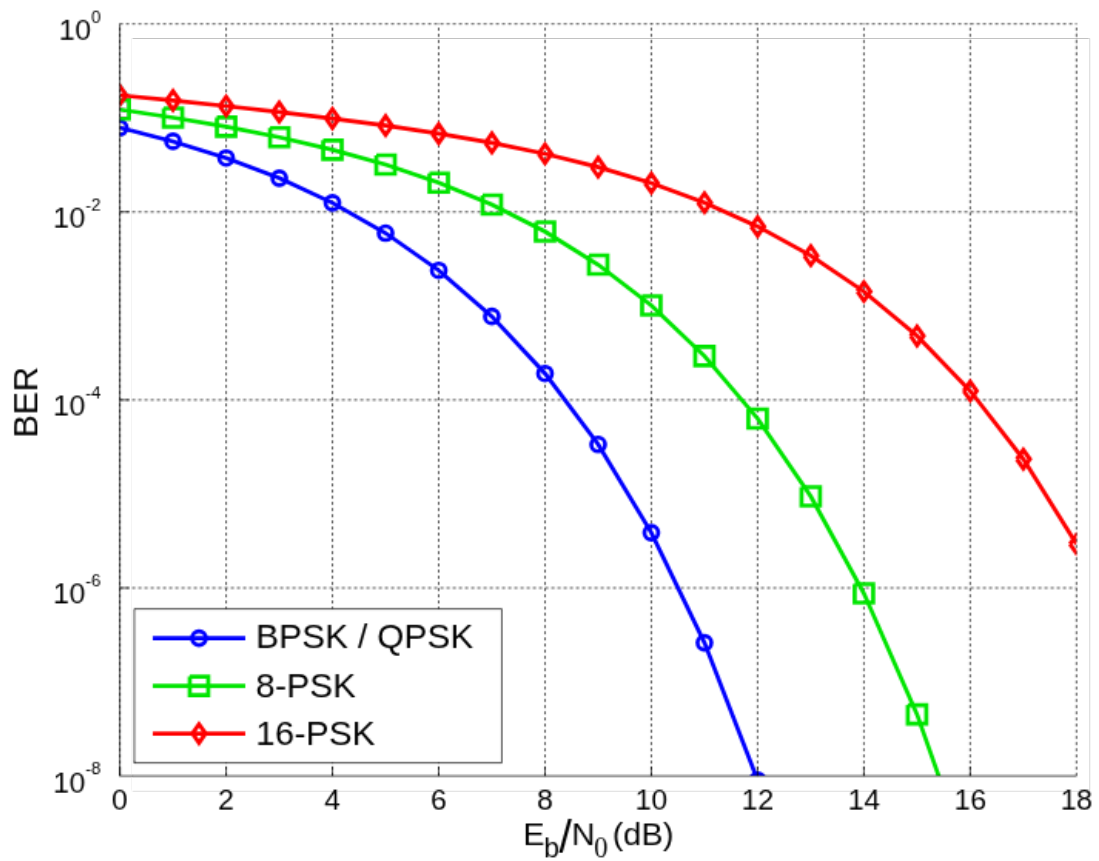


Figure 59: BER vs E_b/N_0 Graph for Uncoded BPSK (Splash 2007)

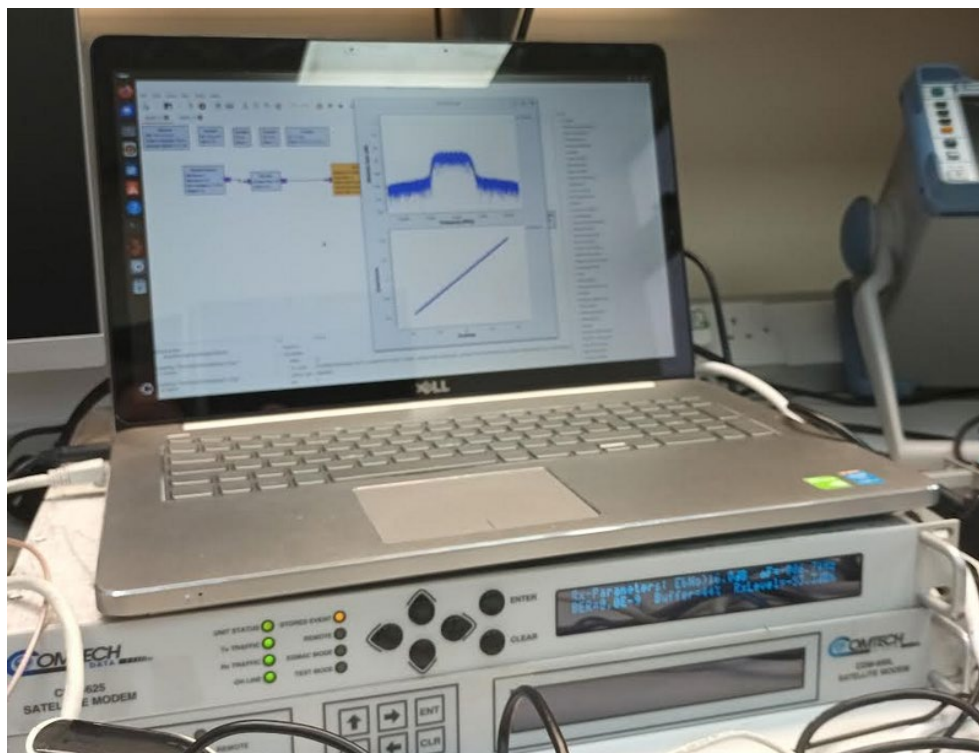


Figure 60: Simultaneous GNU Radio and COTS Modem Operation (GNU Radio Foundation 2023)

8.2 Software Validation and Testing

8.2.1 Power Configurability

The `select_power` command can be used by the user to configure the transmitter power level. There are three distinct power levels available.

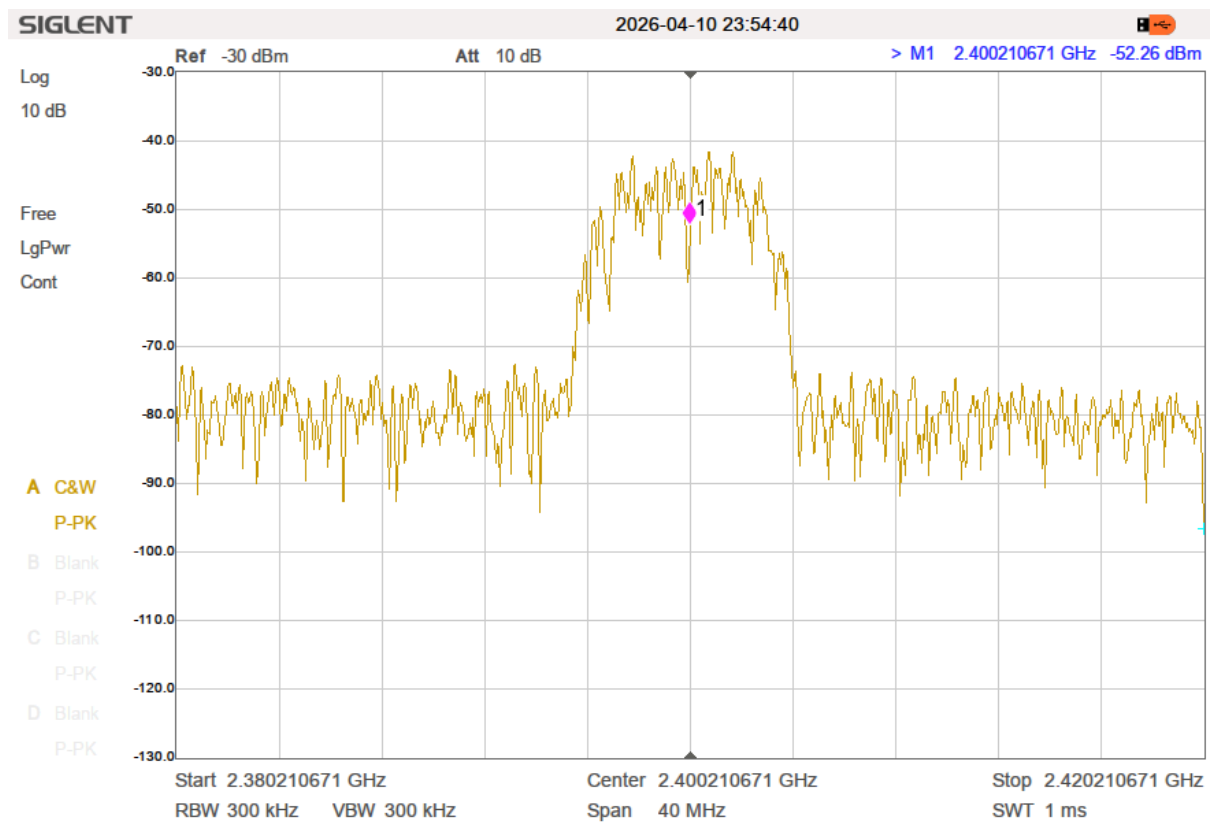


Figure 61: High Power Level

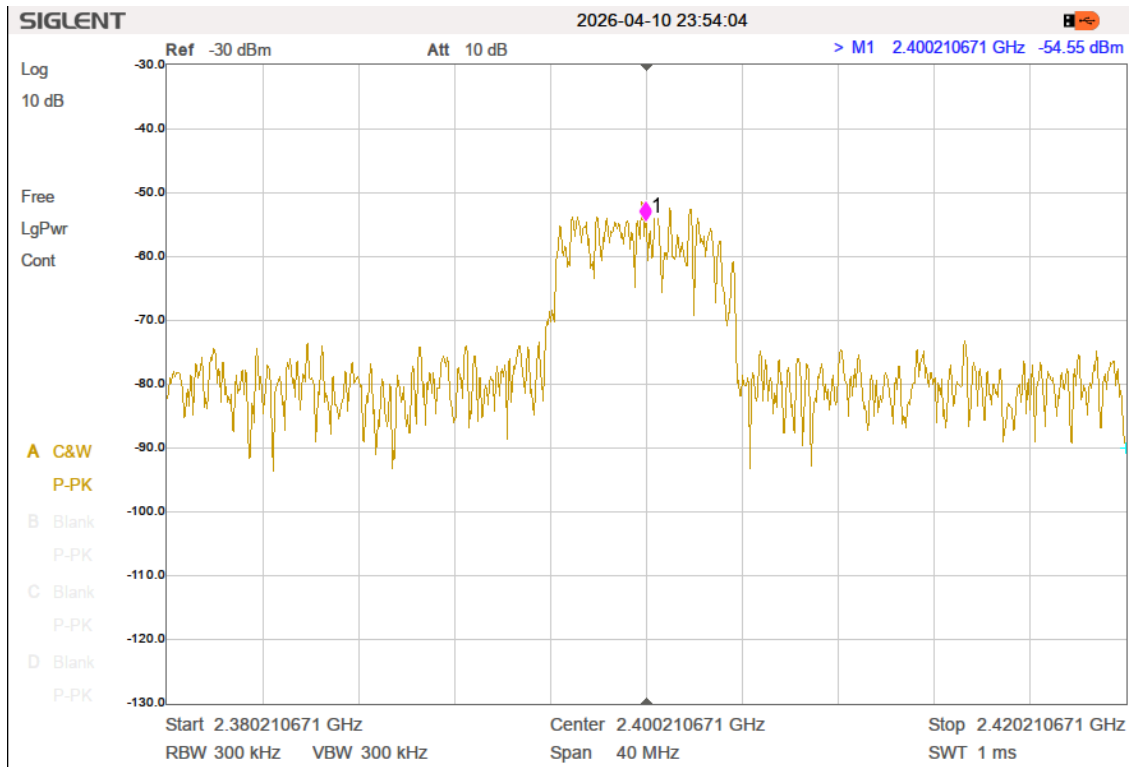


Figure 62: Medium Power Level

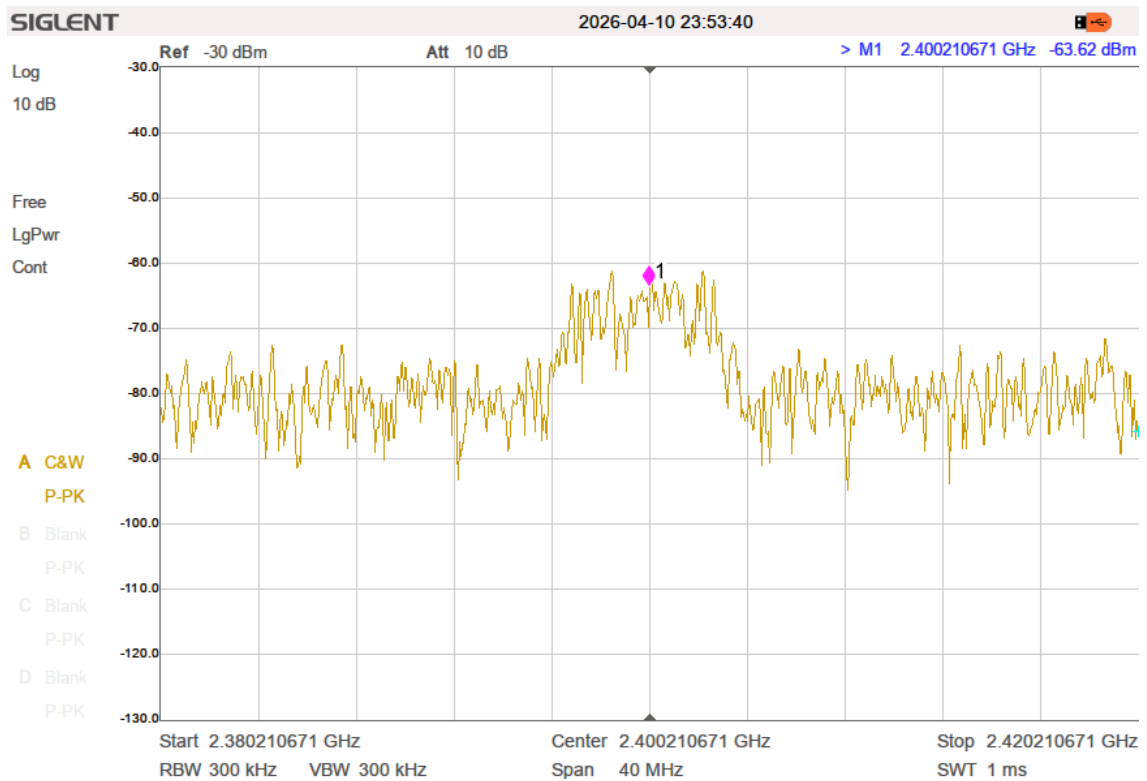


Figure 63: Low Power Level

8.2.2 User-Controlled Muting Capability

In this case the *mute_on* command has been initiated and the *mute_o* signal (the bottom signal of the ILA) has been made high. The I and Q outputs (in blue) are both zero. The *mute_off* command was then used, driving the *mute_o* signal low. The I and Q outputs are now non-zero.

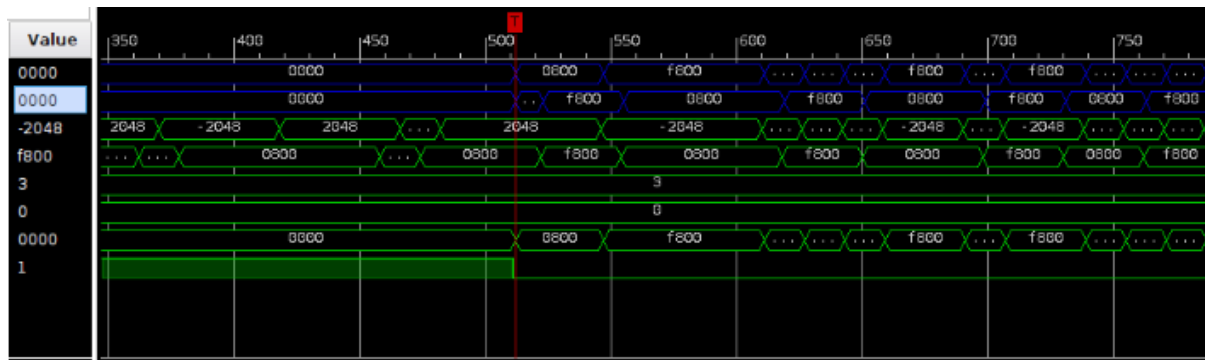


Figure 64: User-Controlled Muting on ILA (AMD 2022)

8.2.3 Internal Muting During Waveform Switching

To avoid glitching during waveform switching, the transmitter output is muted. This is demonstrated on the ILA.

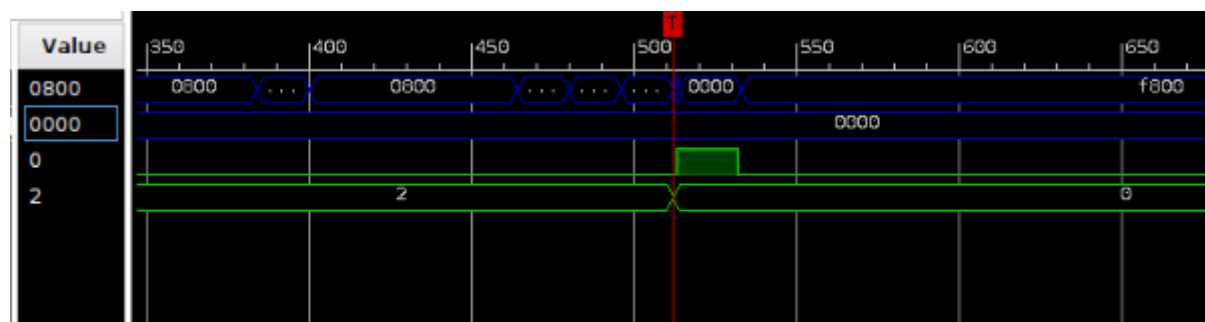


Figure 65: Waveform Switch Muting on ILA (AMD 2022)

8.2.4 Waveform Switching and Identification

The `select_waveform` command can be used to switch between the four available waveforms. The `status` command provides the user with applicable information regarding the current waveform. The data rate unit is mistakenly given as Mbps but has been since been corrected to MSPS in the Bash script.

```
root@analog:~# status
-----
SDR Transmitter Status
-----
Selected waveform name : Telecommand
Output frequency      : 2.4 GHz
Coding scheme         : BCH
Modulation type       : BPSK
Filtering type        : SRRC, alpha = 0.35
Sample rate           : 25 Mbps
Symbol rate           : 6.25 Msps
Data rate             : 6.25 Mbit/s
-----
root@analog:~# select_waveform beacon
2400000000
Waveform changed to beacon
root@analog:~# status
-----
SDR Transmitter Status
-----
Selected waveform name : Beacon
Output frequency      : 2.4 GHz
Coding scheme         : None
Modulation type       : BPSK
Filtering type        : SRRC, alpha = 0.35
Sample rate           : 25 Mbps
Symbol rate           : 6.25 Msps
Data rate             : 6.25 Mbit/s
-----
root@analog:~# select_waveform telemetry
2400000000
Waveform changed to telemetry
root@analog:~# status
-----
SDR Transmitter Status
-----
Selected waveform name : Telemetry
Output frequency      : 2.4 GHz
Coding scheme         : Convolutional
Modulation type       : BPSK
Filtering type        : SRRC, alpha = 0.35
Sample rate           : 25 Mbps
Symbol rate           : 6.25 Msps
Data rate             : 6.25 Mbit/s
-----
```

Figure 66: Waveform Switching and Waveform Status

When `select_waveform fhss` is used, frequency hopping should be initiated. When another waveform is then selected, this frequency hopping should be terminated as shown below.

```

root@analog:~# select_waveform beacon
2400000000
Waveform changed to beacon
root@analog:~# select_waveform fhss
[1] 1138
FHSS started
Waveform changed to fhss
root@analog:~# select_waveform fhss
FHSS already running
Waveform changed to fhss

```

Figure 67: FHSS Waveform Start and Restart

```

root@analog:~# select_waveform telemetry
[1]+ Terminated ./fhss.sh > /tmp/fhss.log 2>&1
FHSS stopped
2400000000
Waveform changed to telemetry
root@analog:~#

```

Figure 68: FHSS Waveform End

8.3 System Stability

The transmitter was run continuously for more than an hour to verify that it could run without user interference.

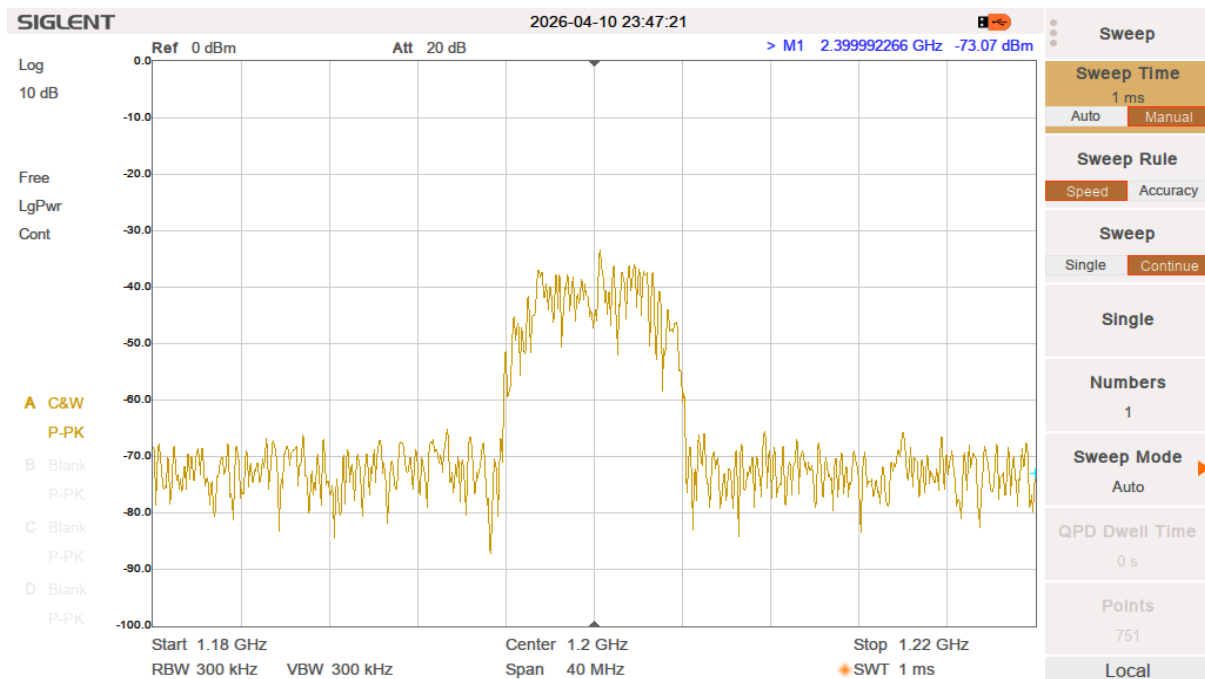


Figure 69: Beacon Spectrum Plot at 23:47

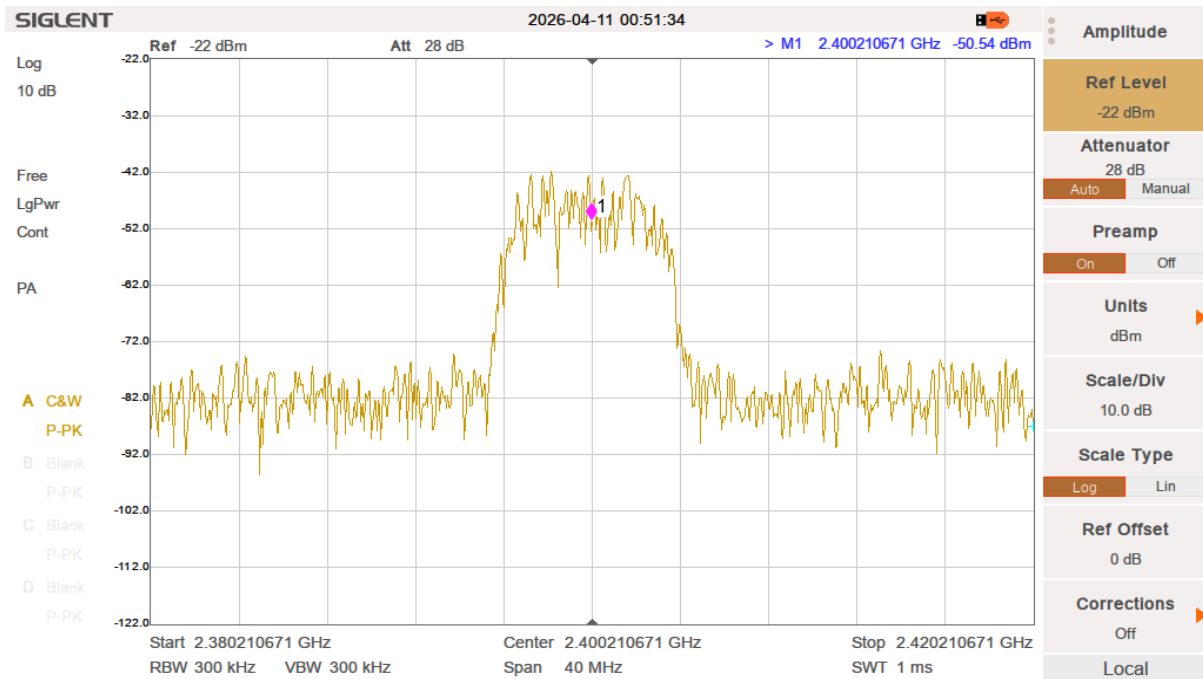


Figure 70: Beacon Spectrum Plot at 00:51

8.4 Implementation Results in Vivado

8.4.1 Timing Results

These results were generated after the implementation of the design in Vivado. They suggest clean timing closure within the setup, hold and pulse width domains.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.133 ns	Worst Hold Slack (WHS): 0.016 ns	Worst Pulse Width Slack (WPWS): 0.264 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 55045	Total Number of Endpoints: 54976	Total Number of Endpoints: 22842

All user specified timing constraints are met.

Figure 71: Vivado Timing Summary Report (AMD 2022)

8.4.2 Resource Utilisation

It is clear from this graph that the *ZedBoard* provides comfortably sufficient resources for this transmitter design.

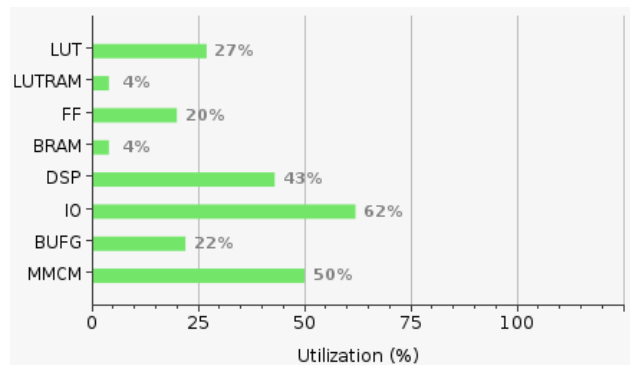


Figure 72: Post-Implementation Resource Utilisation (AMD 2022)

9 IMPLEMENTATION AND TESTING EVALUATION

9.1 Evaluation of Results Against PDS

The following tables evaluate the implemented SDR transmitter against the defined PDS set out in *Section 4* of this report.

Table 32: PDS Evaluation (Functional Requirements)

Requirement Reference	Achievement	Comment	Evidence
Functional Requirements			
FR-1	<i>Achieved</i>	Four distinct waveform configurations implemented and validated.	Section 5.2.3 Section 8.2.4
FR-2	<i>Achieved</i>	Runtime waveform selection is supported via a user interface using a CLI.	Section 6.3 Section 8.2.4
FR-3	<i>Achieved</i>	The design includes both BPSK and OQPSK, as well as a mute state that can be user-initiated or activated using waveform switching.	Section 6.1.5 Section 8.2.2 Section 8.2.3
FR-4	<i>Achieved</i>	Waveform configurations defined using fixed coding, modulation and filtering.	Section 5.2.3 Section 6.1
FR-5	<i>Achieved</i>	Transmitter generates complex I/Q baseband output for all four waveforms.	Section 6.1 Section 6.2
FR-6	<i>Achieved</i>	Power adjustment is possible using a CLI. Only three levels are possible, so the power configuration lacks flexibility.	Section 6.3 Section 8.2.1
FR-7	<i>Not Achieved</i>	Data rate configuration is not currently possible.	N/A
FR-8	<i>Achieved</i>	FHSS implemented through periodic carrier-frequency changes.	Section 5.2.2.4 Section 8.2.4
FR-9	<i>Achieved</i>	CLI ensures real-time monitoring of the properties of each waveform.	Section 6.3 Section 8.2.4
FR-10	<i>Achieved</i>	Stop, start and restart control is provided by a user command for each. Reprogramming the FPGA is not required.	Section 6.3 Section 8.2.4
FR-11	<i>Achieved</i>	Telecommand and telemetry waveforms both conform to the CCSDS standards.	Section 2.4.1 Section 6.1
FR-12	<i>Achieved</i>	GNU Radio receiver built and usable for both the BPSK and OQPSK-modulated waveforms.	Section 8.1
FR-13	<i>Achieved</i>	Verification using GNU Radio and COTS Modem permitted.	Section 8.1
FR-14	<i>Achieved</i>	Simultaneous operation of both the COTS modem and GNU Radio receiver achieved.	Section 8.1
FR-15	<i>Partially Achieved</i>	Kuiper Linux has its own measures to ensure invalid commands and unsuitable parameter values are not entered. Muting not carried out as it is	Section 6.3

		more convenient to keep the transmitter running in its current configuration.	
FR-16	<i>Achieved</i>	All testing and validation performed in an office and laboratory environment.	Section 8.1

Table 33: PDS Evaluation (Non-Functional Requirements)

Requirement Reference	Achievement	Comment	Evidence
Non-Functional Requirements			
NFR-01	<i>Achieved</i>	Continuous real-time I/Q output with no observed interruptions.	Section 6.1 Section 8.4.1
NFR-02	<i>Achieved</i>	Sample and symbol rates had to be matched in hardware and in software to ensure correct operation.	Section 6.1 Section 8.1
NFR-03	<i>Partially Achieved</i>	FHSS implemented using AD9361 LO retuning. Temporary loss of PLL lock occurs during frequency transitions. This requirement is fulfilled during steady state operation, however.	Section 5.2.2.4 Section 8.2.4
NFR-04	<i>Achieved</i>	System demonstrated stable operation for over one hour without any user interference.	Section 8.3
NFR-05	<i>Not Achieved</i>	BER performance not evaluated against Eb/N0 curves. This was due to the absence of a fully compliant receiver. BER and Eb/N0 values obtained for the Beacon waveform only, not telemetry.	N/A
NFR-06	<i>Achieved</i>	Observed SNR remained stable over time and varied by approximately 3.	Section 2.8 Section 8.1
NFR-07	<i>Not Achieved</i>	Accurate Eb/N0 estimation was not implemented	N/A
NFR-08	<i>Partially Achieved</i>	Spectral shaping reduced the out-of-band emissions, but spurious components observed so not fully achieved.	Section 6.1.6 Section 8.1
NFR-09	<i>Achieved</i>	FHSS operation maintained continuous transmission with no significant interruptions or distortion observed.	Section 5.2.2.4 Section 8.2.4
NFR-10	<i>Achieved</i>	Design met FPGA resource and power constraints with sufficient headroom remaining post-implementation.	Section 8.4.2
NFR-11	<i>Achieved</i>	Timing closure was completed without setup or hold violations, albeit with little margin.	Section 8.4.1

9.2 Evaluation of Results Against Project Aims and Objectives

The table below demonstrates that the aims and objectives are achieved. Strengths can be found in the implementation of the FPGA-based transmit chain, and the creativity in implementing this custom chain into the wider AD9361 design. More extensive Eb/N0 and BER determination was not feasible due to time constraints, and an underestimation of the complexity associated with validating complex waveforms with framing and encoding features.

Table 34: Aims and Objectives Evaluation

Aims/Objectives	Evidence	Achievement	Critical Evaluation
Aims			
Aim 1 – Flexible SDR transmitter with configurable waveforms	Four waveform configurations were implemented (TC, TM, Beacon and FHSS).	<i>Achieved</i>	The system demonstrates configurability and flexibility.
Aim 2 – Interface with COTS modem and GNU Radio	RF output validated using GNU Radio and COTS modem.	<i>Achieved</i>	Full GNU Radio and COTS modem setup utilised. Limited BER and Eb/N0 testing reduces the depth of the validation.
Aim 3 – Develop understanding of FPGA and SDR	Successful use of Vivado, as well as full integration of custom IP cores into the AD9361 reference design.	<i>Achieved</i>	A strong practical understanding was obtained and demonstrated through successful debugging and timing closure.
Objectives			
O1 – Research SDR and CCSDS standards	Two out of four of the waveforms are CCSDS-compliant.	<i>Achieved</i>	Standards directly informed implementation. This demonstrates effective and comprehensive application of research.
O2 – Define waveform configurations	Four waveforms implemented.	<i>Achieved</i>	Inclusion of frequency hopping, two modulation schemes and two channel coding schemes increasing system versatility. The four implemented waveforms are distinct.
O3 – Hardware/software partition	The PL handles the whole transmit chain and DSP. The PS handles the frequency hopping and user configuration.	<i>Achieved</i>	All the DSP functions were included within the PL to ensure maximal deterministic performance and timing. It would have been superior deterministically to include the FHSS within the PL as well.

O4 – Full transmit chain design	End-to-end transmit chain implemented and tested.	<i>Achieved</i>	Complete system integration achieved with strong integration across the DSP stages.
O5 – User interface	CLI implemented for control and monitoring.	<i>Achieved</i>	The CLI is functional but lacks the usability of a full GUI.
O6 – System validation	GNU Radio and COTS modem testing used. Spectrum plots obtained.	<i>Achieved</i>	Functional validation completed for the BPSK Beacon waveform.

10 CONCLUSIONS

The overarching aim of this project was to design, implement and evaluate a configurable and affordable FPGA-based SDR transmitter. This transmitter can support a total of four distinct waveforms, two of which are compliant with CCSDS standards, and one contains a frequency hopping capability. This implementation demonstrates that it is possible to realise a SDR transmitter on a Zynq-7000 platform. Deterministic baseband processing has been achieved using programmable logic. This hardware/software partition ensures that time-critical DSP functions are executed within the FPGA fabric, whereas higher-level control and configuration was controlled using software.

The system met most of the defined functional and non-functional requirements. The transmitter could operate continuously without interruptions to the data stream. The integration with a GNU Radio receiver, COTS modem and spectrum analyser permitted practical validation of the four transmitted waveforms.

However, several limitations were identified. Data rate configurability was not implemented. Ultimately, this reduced the system flexibility. Full performance validation in terms of BER and E_b/N_0 was also not achieved for the telemetry waveform. A BER and an E_b/N_0 value for the Beacon waveform were obtained, but due to limitations with the COTS modem, a receiver compatible with the framing and encoding features of the Telemetry waveform was not available. More comprehensive research into the capabilities of the COTS modem and GNU Radio could have enhanced the final RF evaluation. It would have prevented issues such as the transmitter emitting data at too high a symbol rate and ensured that more comprehensive testing could be performed on additional waveforms.

Despite these limitations, this project successfully demonstrates the practical implementation of an FPGA-based SDR transmitter that closely aligns with the project aims and objectives given at the beginning of the project. There are improvements in spectral purity, user interface usability and FHSS complexity that could be made, but the current transmitter offers a strong base for further development.

From a project conduct perspective, several key lessons can be identified throughout the project. There was insufficient consideration of the receiver compatibility during the early stages of the project. This limited the ability to fully validate the waveforms with framing and encoding aspects. Additionally, the complexity associated with developing and integrating custom IPs was also underestimated. However, learning how to do this efficiently became a central focus during Phase 2, which paid off in Phase 3.

It also became clear at the beginning of Phase 3 that the project scope was too broad. A greater focus on creating, integrating and evaluating a basic transmitter chain during Phase 2 or early in Phase 3 may have helped to avoid this. Instead, too much focus was placed on developing individual stand-alone IP cores and then integrating the whole chain later. Therefore, there were less opportunities for iterative testing as the full system integration was delayed.

11 RECOMMENDATIONS AND FUTURE WORK

11.1 Data and Symbol Rate Configurability

Implementing a feature whereby the user can configure the data rate or symbol rate of the transmitter would enhance the flexibility and configurability of the produce. It possibly would be easiest to allow for a configurable symbol rate due to the design of the Symbol Timer IP core. This could be implemented using one of the unused AXI registers within the Symbol Timer, the value of which could be manipulated by the user using *devmem*.

11.2 Graphical User Interface

It would also be preferable to implement a GUI instead of a CLI. This would be more engaging and intuitive for the user and would perhaps create a more professional appearance to the transmitter. JustPy was originally earmarked as a Python Web Framework that could be utilised to develop such a GUI.

11.3 Improved Frequency Hopping Capability

In this current implementation, the frequency hopping table only contains five distinct frequencies. It would enhance the authenticity of the transmitter if the hopping table could be expanded to encompass a greater number of frequencies.

REFERENCES

ADVANCE MICRO DEVICES, 2017. *Vivado Design Suite: AXI Reference Guide (UG1037)* [viewed 13 April 2026]. Available from: <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>

ADVANCED MICRO DEVICES, 2018. *Zynq-7000 SoC data sheet: overview (DS190)* [viewed 13 April 2026]. Available from: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>

ADVANCED MICRO DEVICES, 2022. *Vivado Design Suite 2022.2* [Software]. Available at: <https://www.amd.com/>

ADVANCED MICRO DEVICES, 2023. *AXI Basics 1 – Introduction to AXI* [viewed 13 April 2026]. Available from: https://adaptivesupport.amd.com/s/article/1053914?language=en_US

ADVANCED MICRO DEVICES, 2024. *DDS Compiler (PG141) Product Guide* [viewed 7 January 2026]. Available from: <https://docs.amd.com/r/en-US/pg141-dds-compiler>

ADVANCED MICRO DEVICES, 2025. *PetaLinux SDK (software-defined embedded Linux for Xilinx platforms)* [viewed 13 April 2026]. Available from: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/embeddedsoftware/petalinux-sdk.html>

ADIIVO ENGINEERING, 2024. *MicroZed Chronicles: Linear Feedback Shift Register* [viewed 13 April 2026]. Available from: <https://www.adiuvoengineering.com/post/microzed-chronicles-linear-feedback-shift-register>

AMAZON WEB SERVICES, n.d. *AWS Ground Station* [viewed 13 April 2026]. Available from: <https://aws.amazon.com/ground-station/>

ANALOG DEVICES, 2023. *ADALM-Pluto: software-defined radio active learning module* [viewed 13 April 2026]. Available from: <https://www.analog.com/en/design-center/evaluation-hardware-andsoftware/evaluation-boards-kits/ADALM-PLUTO.html>

ANALOG DEVICES, 2024. *AD9361 (Rev.G)* [viewed 13 April 2026]. Available from: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad9361.pdf>

ANALOG DEVICES, 2026. *Sample & Buy AD9361* [viewed 13 April 2026]. Available from: <https://www.analog.com/en/products/AD9361/sample-buy.html>

ANALOG DEVICES, 2026a. *FMCOMMS2/3/4 Project* [viewed 14 April 2026]. Available from: <https://analogdevicesinc.github.io/hdl/projects/fmcomms2/index.html>

BEATON, C., 2024. *What are functional and non-functional requirements?* [viewed 7 January 2026]. Available from: <https://www.requiment.com/what-are-functional-and-non-functional-requirements/>

BEHLKE, R., 2025. *Software defined satellites and the ground segment* [viewed 13 April 2026]. Available from: <https://room.eu.com/article/software-defined-satellites-and-the-ground-segment>

BELTON, V. and STEWART, T.J., 2002. *Multiple criteria decision analysis: an integrated approach*. Boston: Springer

BURRELL, S., 2024. *How Frequency Hopping Spread Spectrum Revolutionizes Wireless Communication* [viewed 13 April 2026]. Available from: https://wraycastle.com/blogs/knowledge-base/frequency-hopping-spread-spectrum?srsId=AfmBOopPEX_ke2w0QX6UT_ssD1WHKSI-WZSKFmBVtOBjp-I2I2DFM7An

CICIORA, W., et al., 2004. *Modern Cable Television Technology*. 2nd ed. San Francisco: Morgan Kaufmann, pp.955-983

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2021. *Radio Frequency and Modulation Systems — Part 1: Earth Stations and Spacecraft*. CCSDS 401.0-B-32. Washington, DC: CCSDS [viewed 13 April 2026]. Available from: <https://public.ccsds.org/Pubs/401x0b32.pdf>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2021a. *TC Synchronization and Channel Coding*. CCSDS 231.0-B-4. Washington, DC: CCSDS [viewed 13 April 2026]. Available from: <https://public.ccsds.org/Pubs/232x0b4.pdf>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2021b. *TC Space Data Link Protocol*. CCSDS 232.0-B-4. Washington, DC: CCSDS [viewed 14 April 2026]. Available from: <https://ccsds.org/Pubs/232x0b4e1c1.pdf>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2021c. *TM Space Data Link Protocol*. CCSDS 132.0-B-3. Washington, DC: CCSDS [viewed 13 April 2026]. Available from: <https://public.ccsds.org/Pubs/132x0b3.pdf>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2023. *TM Synchronization and Channel Coding*. CCSDS 131.0-B-5. Washington, DC: CCSDS [viewed 13 April 2026]. Available from: <https://public.ccsds.org/Pubs/131x0b5.pdf>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2026. *About CCSDS* [viewed 13 April 2026]. Available from: <https://ccsds.org/about/>

CONSULTATIVE COMMITTEE FOR SPACE DATA SYSTEMS, 2026a. *Blue Books: Recommended Standards* [viewed 13 April 2026]. Available from: <https://ccsds.org/publications/bluebooks/>

CURTISS-WRIGHT, 2020. *Latency and Determinism, and Which is More Important for Your Application?* [viewed 13 April 2026]. Available from: <https://defense-solutions.curtisswright.com/media-center/blog/latency-determinism-which-more-important-application>

DARLING, D., n.d. *deterministic system* [viewed 13 April 2026]. Available from: https://www.daviddarling.info/encyclopedia/D/deterministic_system.html

DIGILENT INC., 2012. *ZedBoard Zynq™ Evaluation and Development Hardware User's Guide* [viewed 14 April 2026]. Available from: https://digilent.com/reference/media/zedboard:zedboard_ug.pdf?srsId=AfmBOoqQauoRA230ADi0le5i0B-DUCUWUQo7tj-vWy5NyNxG-fj2oB-d

DIGILENT INC., 2024. *ZedBoard Zynq-7000 ARM/FPGA SoC development board* [viewed 13 April 2026]. Available from: <https://digilent.com/reference/programmablelogic/zedboard/start>

DIGILENT INC., 2026. *ZedBoard Zynq-7000 ARM/FPGA SoC Development Board* [viewed 13 April 2026]. Available from: <https://digilent.com/shop/zedboard-zynq-7000-arm-fpga-soc-development-board/?srsId=AfmBOoqmrLmemf5lBVhNZ8oGI4cdLHfaLNqlgcqiKh8daTyBm7NfsfBi>

ELECTROAGENDA, 2024. *Constant Envelope OQPSK vs QPSK* [viewed 13 April 2026]. Available from: <https://electroagenda.com/en/constant-envelope-oqpsk-vs-qpsk/>

ESERGÜN, Y., 2025. XILINX AXI PROTOCOL FIREWALL IP [viewed 13 April 2026]. Available from: <https://www.linkedin.com/pulse/xilinx-axi-protocol-firewall-ip-yunus-eserg%C3%BCn-9rebf/>

ECSS, 2017. *ECSS-E-ST-10C: Space Engineering – System Engineering General Requirements*. Noordwijk: European Cooperation for Space Standardization

ECSS, 2024. *ECSS-E-ST-50C: Space Engineering – Communications*. Noordwijk: European Cooperation for Space Standardization.

ECSS, 2026. *European Cooperation for Space Standardisation* [viewed 13 April 2026]. Available from: <https://ecss.nl/>

ETSI, 2024. *Digital Video Broadcasting (DVB); Second generation framing structure, channel coding and modulation systems for Broadcasting, Interactive Services, News Gathering and other broadband satellite applications; Part 2: DVB-S2 Extensions (DVB-S2X)* [viewed 13 April 2026]. Available from: https://www.etsi.org/deliver/etsi_en/302300_302399/30230702/01.04.01_20/en_30230702v010401a.pdf

GRAND VIEW RESEARCH, 2026. *Software Defined Radio Market (2026 - 2033)* [viewed 13 April 2026]. Available from: <https://www.grandviewresearch.com/industry-analysis/software-defined-radio-sdr-market>

GREAT SCOTT GADGETS, 2024. *HackRF One software defined radio* [viewed 13 April 2026]. Available from: <https://greatscottgadgets.com/hackrf/one/>

GNU RADIO FOUNDATION, 2023. *GNU Radio (Version 3.10)* [Software]. Available at: <https://www.gnuradio.org/>

IEEE, 2017. *IEEE Std 802.16-2017: Air Interface for Broadband Wireless Access Systems*. New York: IEEE.

IEEE, 2020. *IEEE Std 802.11-2020: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*. New York: IEEE

IEEE, 2023. *IEEE 802LMSC* [viewed 14 April 2026]. Available from: <https://www.ieee802.org/>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1994. *ISO/IEC 7498-1:1994 Information technology — Open Systems Interconnection — Basic Reference Model: The Basic Model* [viewed 14 April 2026]. Available from: <https://www.iso.org/standard/20269.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2018. *ISO 31000:2018 – Risk management — Guidelines* [viewed 13 April 2026]. Available from: <https://www.iso.org/standard/65694.html>

JERRED, J., 2022. *Understanding Phase Shift Keying (PSK) and Implementing PSK31 (BPSK & QPSK)* [viewed 13 April 2026]. Available from: <https://joshuaer.red/blog/2022-11-18-BPSK-Encoding>

KRATOS SPACE, n.d. Technology for Government and Defense Missions [viewed 13 April 2026]. Available from: <https://www.kratospace.com/defense-and-government>

L3HARRIS TECHNOLOGIES, 2024. *C/TT-600 software defined radio* [viewed 13 April 2026]. Available from: <https://www.l3harris.com/all-capabilities/c-tt-600>

LIME MICROSYSTEMS, 2023. *LimeSDR family overview* [viewed 13 April 2026]. Available from: <https://limemicro.com/products/boards/>

LINK-PP, 2025. *What is BPSK Modulation and How Does It Work?* [viewed 13 April 2026]. Available from: <https://resources.l-p.com/knowledge-center/bpsk-modulation-basics-how-bpsk-works-in-digital-communication>

MALATEST, B., 2022. *Deterministic Software Solutions for Complex Architectures Involving Software-Defined Radio (SDR)* [viewed 13 April 2026]. Available from: <https://issuu.com/rtcgroup/docs/cots-2204-april-web/s/15741071>

MALATEST, B., 2025. *Examining RF Architectures for Software-Defined Radios* [viewed 13 April 2026]. Available from: <https://www.microwavejournal.com/articles/43239-examining-rf-architectures-for-software-defined-radios>

MARAL, G., *et al*, 2020. *Satellite Communications Systems: Systems, Techniques and Technology*. 6th ed. Chichester: Wiley

MATHWORKS, 2026. *MATLAB (R2025a)* [Software]. Available at: <https://www.mathworks.com/>

MEMON, T., *et al*, 2013. POWER-area-performance characteristics of FPGA-based sigma-delta FIR filters. *Journal of Signal Processing Systems*, 70

- MITCHELL, B., *OSI Model Reference Guide* [viewed 14 April 2026]. Available from: <https://www.lifewire.com/osi-model-reference-guide-816289>
- MITOLA, J., 2000. *Software radio architecture: object-oriented approaches to wireless systems engineering*. New York: Wiley
- MORIA, 2022. *Demystifying the LFSR* [viewed 13 April 2026]. Available from: <https://www.moria.us/articles/demystifying-the-lfsr/>
- NATIONAL INSTRUMENTS / ETTUS RESEARCH, 2024. *USRP software defined radio devices* [viewed 13 April 2026]. Available from: <https://www.ettus.com/products/>
- NUAND LLC, 2023. *bladeRF 2.0 micro* [viewed 13 April 2026]. Available from: <https://www.nuand.com/bladerf-2-0-micro/>
- OFCOM, 2026. *UK Frequency Allocation Table (UKFAT)* [viewed 13 April 2026]. Available from: <https://static.ofcom.org.uk/static/spectrum/fat.html>
- PRBS23, n.d. *About* [viewed 13 April 2026]. Available from: <https://prbs23.com/blog/about/>
- PROAKIS, J.G. and SALEHI, M., 2008. *Digital communications*. 5th ed. New York: McGraw-Hill.
- RF WIRELESS WORLD, n.d. *FHSS: Advantages and Disadvantages of Frequency Hopping Spread Spectrum* [viewed 13 April 2026]. Available from: <https://www.rfwireless-world.com/terminology/fhss-advantages-and-disadvantages>
- ROELANDTS, T., n.d. *Mapping Bits to a Constellation* [viewed 13 April 2026]. Available from: <https://tomroelandts.com/articles/mapping-bits-to-a-constellation>
- SATSEARCH, 2025. *Software Defined Radios (SDRs) for space and satellite applications* [viewed 13 April 2026]. Available from: <https://blog.satsearch.co/2022-06-27-software-defined-radios-sdrs-for-space-and-satellite-applications>
- SCHOLL, S., 2016. *The Xilinx Zynq: a modern system on chip for software defined radio* [viewed 13 April 2026]. Available from: https://kluedo.ub.rptu.de/frontdoor/deliver/index/docId/4442/file/zynq_software_defined_radio.pdf
- SIMON, M.K., et al, 1994. *Spread Spectrum Communications Handbook* (Vol. 2). New York: McGraw-Hill
- SPLASH, 2007. File: PSK BER curves.svg [viewed 13 April 2026]. Available from: https://en.wikipedia.org/wiki/File:PSK_BER_curves.svg
- SPOONER, C., 2009. Chapter 18: Spectrum Sensing Based on Spectral Correlation. *Cognitive Radio Technology*, 593
- STEWART, R.W., et al, 2015. *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. Glasgow: Strathclyde Academic Media

TIAN, F., *et al*, 2019. Design and implementation of AD9361-based software radio receiver. *J Wireless Com Network*, 95

VIAVI SOLUTIONS, n.d. *What is Software Defined Radio (SDR)?* [viewed 13 April 2026]. Available from: <https://www.viavisolutions.com/en-us/what-software-defined-radio-sdr>

WAITE, P., 2024. *Understanding Direct Sequence Spread Spectrum: A Simple Guide for Beginners* [viewed 13 April 2026]. Available from: https://wraycastle.com/blogs/knowledge-base/direct-sequence-spread-spectrum?srsItd=AfmBOoo4cK_IQ7IIFAy7zpg5VeXEgM30p-oqGLi_q-9FK4gxVweMji3a

APPENDICES

Appendix 1: Competitive Market Analysis

This appendix presents the more detailed comparative analysis of existing SDR platforms. These platforms are used across a range of sectors, including aerospace, defence and academia. The four tables included here summarise key technical characteristics, as well as the primary user cases and cost. This analysis is summarised in *Section 2.3*.

Table 35: Existing SDR Platforms Comparison (Architecture and Use Case)

SDR Platform	Primary User Case	FPGA + RF Architecture
L3Harris C/TT-600 SDR (L3Harris Technologies 2024)	Defence and government wideband SATCOM.	Xilinx Kintex/Virtex FPGA + custom RF front end
NI Ettus USRP-2955/2954/2945/29444 (National Instruments / Ettus Research 2024)	Academic and industrial SDR research.	Xilinx Kintex-7 FPGA + modular RF daughtercards (AD9361 or WBX/UBX)
NI Ettus USRP B210 / B205mini (National Instruments / Ettus Research 2024)	Teacher, lab-scale SDR research.	Xilinx Spartan-6 FPGA + AD9361/AD9364 transceiver
Analog Devices PlutoSDR (Analog Devices 2023)	Education for embedded SDR.	Xilinx Zynq Z-7010 (FPGA + ARM PS) + AD9363 RFIC
LimeSDR (Lime Microsystems 2023)	General-purpose prototyping, open-source research.	Intel Cyclone IV/V FPGA + LMS7002M transceiver
BladeRF 2.0 Micro (Nuand 2023)	Academic and applied SDR research.	Xilinx Artix-7 FPGA + AD9361 transceiver
HackRF One (Great Scott Gadgets 2024)	Entry-level SDR education.	Lattice ECP3 FPGA + MAX2837 RFIC
<i>Proposed ZedBoard + AD9361 Transmitter (Digilent 2024)</i>	<i>For BEng (Hons) project.</i>	<i>Xilinx Zynq Z-7020 (PL + PS) on Digilent ZedBoard + AD9361 RFIC</i>

Table 36: Existing SDR Platforms Comparison (Bandwidth and Cost)

SDR Platform	Frequency Range	Bandwidth	Approximate Cost*
L3Harris C/TT-600 SDR (L3Harris Technologies 2024)	30 MHz to 6 GHz	Up to 50 MHz	££££ to £££££
NI Ettus USRP- 2955/2954/2945/2944 (National Instruments / Ettus Research 2024)	Between 10 MHz and 6 GHz (model-dependent)	80 or 160 MHz	£££
NI Ettus B200/B210/B200mini/B205mini/B206mini (National Instruments / Ettus Research 2024)	70 MHz – 6 GHz	Up to 56 MHz	££
Analog Devices PlutoSDR (Analog Devices 2023)	325 MHz to 3.8 GHz (extendable to 6 GHz)	Up to 20 MHz	£
LimeSDR (Lime Microsystems 2023)	10 MHz to 3.5 GHz	30 – 60 MHz	£ to ££
BladeRF 2.0 Micro (Nuand 2023)	47 MHz to 6 GHz	Up to 56 MHz	££
HackRF One (Great Scott Gadgets 2024)	1 MHz to 6 GHz	About 20 MHz	£
Proposed ZedBoard + AD9361 Transmitter (Digilent 2024)	70 MHz to 6 GHz	Up to 56 MHz	££

* Cost tier key: £ (< £500), ££ (£1000 to £2000), £££ (£3000 to £15000), ££££ (£15,000 to £50,000), £££££ (> £50,000)

Table 37: Existing SDR Platform Characteristics Comparison (Timing and Power)

SDR Platform	Timing Determinism	Typical Tx Power (dBm)
L3Harris C/TT-600 SDR (L3Harris Technologies 2024)	Deterministic (hardware-scheduled)	+30 to +40
NI Ettus USRP- 2955/2954/2945/ 2944 (National Instruments / Ettus Research 2024)	Moderate (PL + RFNoC timed execution)	+10 to +20
NI Ettus B200/B210/B200mini/B205mini/B206mini (National Instruments / Ettus Research 2024)	Moderate (FPGA-scheduled with PPS/10 MHz clocks)	0 to +10
Analog Devices PlutoSDR (Analog Devices 2023)	Non-deterministic (USB bus timing)	-3 to 7
LimeSDR (Lime Microsystems 2023)	Moderate (FX3 USB introduces jitter, but frontend timing is deterministic)	0 to +10
BladeRF 2.0 Micro (Nuand 2023)	Moderate (USB3 with FPGA and onboard timing control)	0 to +7
HackRF One (Great Scott Gadgets 2024)	Low (USB2-base with no hardware timing)	0 to +10
<i>Proposed ZedBoard + AD9361 Transmitter (Digilent 2024)</i>	Deterministic after the baseband data enters the PL.	+5 to +10 dBm typical for AD9361 at 2.4 GHz, with variation ± 2 dB across band.

Table 38: Existing SDR Platforms Comparison (Openness and Configurability)

SDR Platform	System Openness	Configurability
L3Harris C/TT-600 SDR (L3Harris Technologies 2024)	Proprietary/closed firmware and software.	Limited – configuration through vendor waveforms and mission loads only.
NI Ettus USRP- 2955/2954/2945/ 2944 (National Instruments / Ettus Research 2024)	Semi-open (licensed API)	High – modular FPGA and RFNoC blocks.
NI Ettus B200/B210/B200mini/B205mini/B206mini (National Instruments / Ettus Research 2024)	Semi-open (open drivers and API, but licensed FPGA firmware)	Medium – user logic can be added but does not allow FPGA redesign.
Analog Devices PlutoSDR (Analog Devices 2023)	Open-source firmware	High – allows modification of HDL, Linux kernel and rootfs.
LimeSDR (Lime Microsystems 2023)	Open hardware and community support	Very high – FPGA and RF parameters configurable.
BladeRF 2.0 Micro (Nuand 2023)	Open	High – Full access to HDL and calibration parameters.
HackRF One (Great Scott Gadgets 2024)	Open	Moderate – CPLD-based, not FPGA, so limited digital modification is permitted.
<i>Proposed ZedBoard + AD9361 Transmitter (Digilent 2024)</i>	Reference design available on GitHub (Github 2025). Allows for custom Vivado integration	Very high – FPGA and RF parameters configurable on a user-friendly interface.

Appendix 2: Detailed Space Communications Standards

CCSDS Physical Layer

For the physical layer, the CCSDS have developed a single omnibus (CCSDS 2023). This standard applies to links between spacecraft and ground systems. *CCSDS 401.0-B-32: Radio Frequency and Modulation Systems Part 1* establish the standard for RF and modulations systems for both ground stations and spacecraft (CCSDS 2021). *Part 2* relates to data relay satellite systems and therefore falls outside the scope of the project. Its technical recommendations include modulation methods, frequency utilisation, power limitations and choice of pulse code modulation (PCM) waveforms.

CCSDS Data Link Layer

Within an SDR transmitter, the data link layer would be mainly concerned with the structure of transmitted data, the synchronisation of the transmitter and receiver and the improvement of communications reliability using channel coding. These features are especially important within space communications systems, where data must be transmitted over long distances. As a result, they are subject to effects such as Doppler shift, noise, attenuation and other channel impairments.

There are two main CCSDS standards that are applicable to telecommand and telemetry links. *CCSDS 131.0-B-5: Telemetry Synchronisation and Channel Coding* and *CCSDS 231.0-B-4: Telecommand Synchronisation and Channel Coding* (CCSDS 2022, CCSDS 2022a). These two standards define the frame structure, as well as the acceptable channel coding schemes.

For frame synchronisation, telemetry transmissions usually incorporate an Attached Sync Marker (CCSDS 2023). This allows the receiver to determine the start of a telemetry frame when it is fed a continuous data input.

ECSS Standards

The ECSS was established through the collaboration between ESA, individual national space agencies and the wider aerospace industry in Europe (ECSS 2023). This intends to build a common framework for the development and verification of space systems. It is widely adopted across European space programmes.

There are several different standards that are potentially applicable. *ECSS-E-ST-10C: System Engineering General Requirements* outlines the wider systems engineering requirements for the implementation of space systems, and product development within the space sector (ECSS 2017). *ECSS-E-ST-50C: Communications* specifies the specific requirements for the development of space data communications systems, including telemetry and telecommand architectures and link performance considerations (ECSS 2024).

The ECSS standards heavily reference the CCSDS standards mentioned in the previous section. The difference is that the ECSS standards define the wider engineering framework for developing space communications systems. It was determined, therefore, that the CCSDS standards were more applicable to this project.

To avoid confusion, the specific technical requirements of this project will be based on the CCSDS standards.

ITU-R Standards

The International Telecommunication Union Radiocommunication Sector (ITU-R) is responsible for establishing international regulations governing the use of the radio spectrum (ITU 2024). The regulations are articulated formally under the *ITU Radio Regulations* which are agreed internationally and are revised periodically. As a result, a satellite communication system must only operate within the frequency bands allocated to the service that it provides. This ensures that there is no interference with other critical radio services. As a result, during the development of a satellite communication system, the ITU spectrum allocation tables must be consulted. In the United Kingdom, Ofcom uses the ITU's guidance to provide the United Kingdom Frequency Allocation Table (UKFAT) (Ofcom 2026). The UKFAT can be viewed visually by using Ofcom's UK spectrum map tool (Ofcom 2026).

DVB-S2 Standard

The *Digital Video Broadcasting – Satellite Second Generation* standard is a widely-utilised commercial satellite communications standard developed by the DVB Project and codified by the European Telecommunications Standards Institute (ETSI) (ETSI 2024). This standard is generally employed in television broadcasting and satellite broadband services. This is because high data throughput and high spectral efficiency must be prioritised for these applications. DVB-S2 has achieved this using advanced modulation techniques, as well as enhanced forward error correction techniques.

The DVB-S2 is certainly a flexible standard and can be utilised for a variety of satellite applications. However, it is optimised for high-throughput commercial satellite broadcasting applications. For this reason, it is likely that the DVB-S2 standard is less pertinent to this project than the more general CCSDS standards which give more specific recommendations for telemetry and telecommand links.

IEEE Wireless Communications Standards

The Institute of Electrical and Electronics Engineers (IEEE) are responsible for developing a wide range of technical standards related to wireless communications systems. It is especially valuable to review those in the IEEE 802 family, because they have been instrumental in the development of modern wireless and wired networking technologies (IEEE 2023). Significant examples include *IEEE 802.11* which specifies wireless local area networks (Wi-Fi), and *IEEE 802.16* which is the standard for broadband wireless access systems (IEEE 2026, IEEE 2017).

These IEEE standards generally relate to terrestrial communications systems, and not specifically to space communications systems. Consequently, these standards provide important context for understanding the development of wireless communication technologies but are unlikely to be within the scope of this project.

Appendix 3: OSI Model Reference

The following figure displays the OSI model as defined by ISO/IEC 7498 (ISO 1994). It is important to note that only the Physical Layer (Layer 1) and Data Link Layer (Layer 2) fall within the scope of this project. Within the context of developing an SDR transmitter, the Physical Layer is responsible for modulation and RF transmission, whereas the Data Link layer is responsible for frame structure, channel coding, and frame synchronisation.

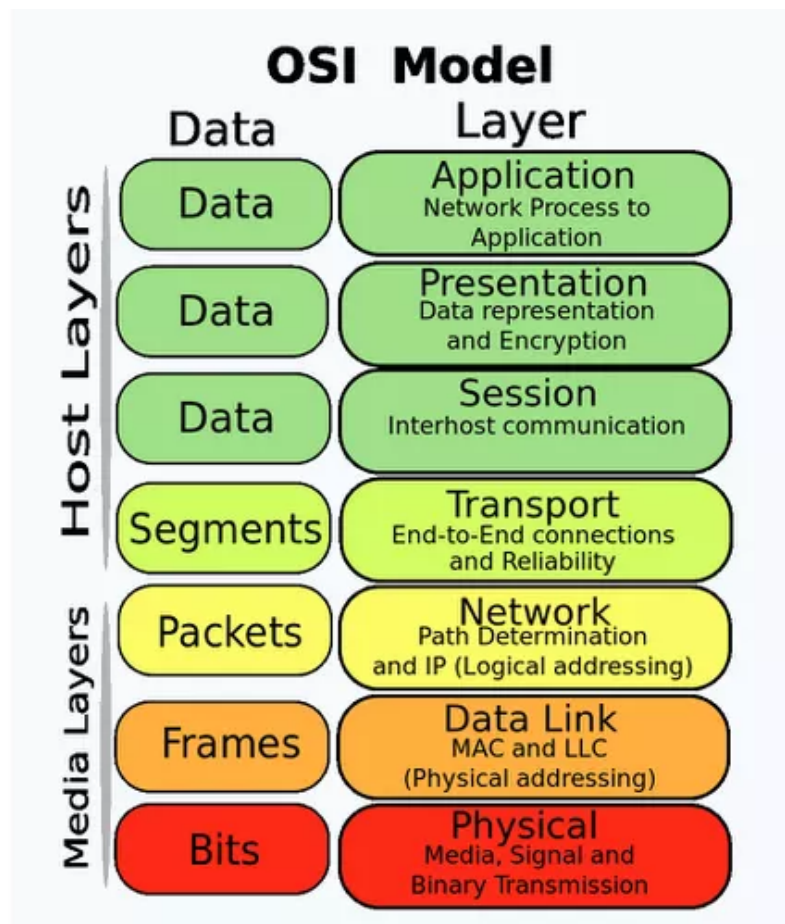


Figure 73: OSI Model (Mitchell 2019)

Appendix 4: Resource Requirements

The following tables contain a list of the hardware and software resources that were required during the project.

Table 39: Required Resources (Hardware)

Hardware	Implications of Non-Availability
For Development	
Development Laptop <i>Ubuntu 22.04.5 LTS</i>	No VHDL code can be written and installed on the ZedBoard. This will therefore not permit the development of any components within the programmable logic part of the Zynq-7000 system (e.g PRBS, modulation, filtering). No simulations can be carried out on Vivado, MATLAB or other ECAD software.
FPGA <i>Digilent ZedBoard Zynq-7000 Development Board</i>	This is the FPGA development board and is central to the SDR system. Provided that Vivado is available, the design and simulation of any VHDL code can be completed in Vivado, as well as the synthesis, implementation and bitstream generation of that code. However, there can be no physical implementation onto the board itself, and so any testing in conjunction with GNU Radio and the COTS modem cannot occur. Any configuration of the operating system (if installed) cannot be carried out.
RF Transceiver Board <i>Analog Devices AD9361 RF Transceiver</i>	Again, this board is central to the SDR system and produces the transmitted RF signals. Without it, no physical implementation can occur, and therefore the system testing and overall demonstration cannot happen.
SD Card with Micro SD Card Adapter	Required if an operating system (e.g Kuiper Linux) has been chosen and is to be run on the FPGA board.
Ethernet Cables	Required for the development phase (of the waveforms and user interface) if using an Ethernet connection between the FPGA and the development laptop. This is not required if the UART interface is utilised instead.
2 x USB-A to Micro USB Cable	Required if the communication between the development laptop and the FPGA is facilitated using UART.
SD to USB-A Card Adapter	No new bitstream can be run on the FPGA. This could cause a catastrophic delay in the development of the fully integrated design (custom IP and AD9361 reference design).
For Testing	
Test Laptop	Having a second laptop allows for the user interface to be run on the development laptop, and the GNU Radio receiver can be run simultaneously on the test laptop. This may make it easier to view the output of the transmitter in real-time, as various

	parameters are changed. GNU Radio is installed on the development laptop, but it will enhance the system testing and demonstration phases if a separate laptop is available (both within Phase 3).
Computer Switch	If this is unavailable, the development laptop must be used to run GNU Radio. A separate laptop cannot be used. Hence, whilst GNU Radio testing is still possible, using two distinct laptops is not possible. This may be less convenient for the test engineer.
COTS Modem	The availability of the COTS modem is required for the testing phase. Required directly for the verification of the following requirements: • FR-13 • FR-14
Spectrum Analyser	Required for the testing phase and to verify the following requirements have been met: • FR-08 • FR-09 • FR-11 • FR-15 • NFR-03 • NFR-06 • NFR-07 • NFR-08 • NFR-09
2 x RF Splitters	Required for the testing phase so that the RF transmission from the SDR system can be fed into the COTS modem, spectrum analyser and other test equipment (if used).
2 x UK Plug Kettle Lead	Essential for powering the COTS modem and spectrum analyser. Non-availability will result in potential delays to the testing phase. More of these cables will be required if additional test equipment is used (e.g an additional COTS modem).
6 x SMA Male to SMA Male Cables	Required for the RF transmission to be fed from the AD9361 to the spectrum analyser, COTS modem and other test equipment (if utilised). Non-availability may cause delays to the system testing (except for the GNU Radio testing).

Table 40: Required Resources (Software)

Software	Implications of Non-Availability
AMD Vivado 2022.2	Project cannot progress beyond the ideas phase as no VHDL can be written and installed on the FPGA.
FPGA Operating System	To have control of the system during runtime, a bare metal architecture utilising Vitis would have to be used instead. The system testing phase would be potentially impaired if the user cannot switch between different waveforms.
GNU Radio	Required to satisfy FR-14 and is essential for the test development and testing phases.
MATLAB	Can be utilised in filter design and simulation for the CCSDS-compliant waveforms.
Xilinx Linux Kernel	The Linux kernel provides the middle layer between the hardware and the written software. It also provides the driver support for a Linux OS to run on the FPGA. Without this, there can be no physical testing, or development of software.
Analog Devices AD9361 Reference Design	Without the reference design none of the RF front-end can be configured. Any VHDL code cannot be integrated into the AD9361's existing architecture, and therefore a higher-level simulation cannot take place. Physical implementation cannot progress either.
Minicom	Minicom will be used during the development of the user interface, but also during integration and testing to access the system for fault recovery.
Git	Git provides change control, thus mitigating the risk of essential files becoming lost and allowing for gradual changes to be made, and verified, before those changes are committed. Non-availability or failure would affect all phases of the project.
GitHub	GitHub provides a user-friendly, web-based interface, and it is therefore easier to see any changes that are made to the repository. Git and GitHub use the same underlying version control system.

Appendix 5: Custom IP Cores Summary

This table lists all the IP cores that were added to the AD9361 reference design. Of the 18 cores listed, 15 were developed from scratch and 3 are Constant blocks taken from the Vivado IP catalogue.

Table 41: Custom IP Blocks Function

Name of Block	Purpose	AXI Used?
BCH Encoder	Provides BCH encoding capability [see Section 6.1.4.1]	NO
Convolutional Encoder	Provides convolutional encoding capability [see Section 6.1.4.2]	NO
Fifo_0_tie16 (Constant)	Ties unused data inputs to zero on the AD9361 DAC FIFO [see Section 6.4]	NO
Fifo_0_tiebit (Constant)	Ties din_unf to 0 on AD9361 DAC FIFO [see Section 6.4]	NO
Modulation Mapper	Provides BPSK and OQPSK modulation mapping [see Section 6.1.2]	YES
Mute Multiplexer	Mutes output on request and during waveform switching [see Section 6.1.5].	NO
Path Selector	Will convert the waveform selection (telecommand, telemetry, beacon or FHSS) into a path selection value (telecommand path, telemetry path or bypass encoding and framing). Will also tell the Mute Multiplexer to mute the transmitter output if waveform switching is underway.	NO
Path Enable Controller	Will take the path selection value from the Path Selector and enable the telecommand framing and encoding chain if that is selected, or likewise the telemetry framing and encoding chain. Neither will be activated if the Path Selector suggests that the beacon or FHSS waveform is in operation.	NO
PRBS23	Provides the PRBS23 data source [see Section 6.1.1].	YES
PRBS Step Multiplexer	Controls when the PRBS advances. If operating in beacon or FHSS mode, the PRBS will advance when the PRBS Step Select Block tells it to. If in telecommand mode, it will advance when the Telecommand Framer block is packing the payload into the frame (as opposed to the header or CRC). If in telemetry mode, the PRBS will advance when the Telemetry Framer block is packing the payload into the frame.	NO
PRBS Step Select	Utilised only when in beacon or FHSS mode. When in beacon (BPSK), the PRBS will advance on each new symbol period.	NO

	When in FHSS (OQSPK), the PRBS will advance on every symbol and half-symbol period to ensure that the half-symbol period offset is maintained.	
SRRC Filter	Provides SRRC filtering [see <i>Section 6.1.6</i>].	YES
Symbol Timer	Provides the symbol and half-symbol strobes required for correct BPSK and OQPSK operation.	YES
Transmit Controller	Controls the waveform selection and mute requests from the user.	YES
Transmit Multiplexer	Combines the telecommand, telemetry and bypass (beacon/FHSS) output bit paths back together before the modulation stage.	NO
Telecommand Framer	Provides framing for the telecommand waveform [see <i>Section 6.1.3.1</i>].	NO
Telemetry Framer	Provides framing for the telemetry waveform [see <i>Section 6.1.3.2</i>].	NO
Upack_0_tiebit	Ties unused enable pins in util_upack2_v1_0 [see <i>Section 6.4</i>].	NO

Appendix 6: AXI Address Map

The following table gives the address and purpose of every AXI register included within the custom transmit chain.

Table 42: AXI Address Map

IP Core Name	Register Address	Register Function	Register Access
Modulation Mapper	0x40000000	Enables the modulation mapper block. Bit 0: 0 – Modulation Off 1 – Modulation On	Read-Write
	0x40000004	Unused	Read-Write
	0x40000008	Unused	Read-Write
	0x4000000C	Unused	Read-Write
PRBS23	0x40001000	Serves as the control register. Bit 0: 0 – PRBS Off 1 – PRBS On Bit 1: Writing 1 will emit a reset pulse.	Read-Write
	0x40001004	Serves as the status register and allows the user to view the current output bit the PRBS block. This is highly useful for debugging.	Read Only
	0x40001008	Serves as the state register. Exposes the internal state of the LFSR. It provides a 23-bit output. This is useful for debugging.	Read Only
	0x4000100C	Unused	Read/Write
SRRC Filter	0x40002000	Serves as the control register. Bit 0: 0 – Filter Bypassed 1 – Filter Enabled	Read/Write
	0x40002004	Status mirror of the control bit. Exposes current state of the SRRC filter.	Read Only
	0x40002008	Unused	Read/Write
	0x4000200C	Unused	Read/Write
Symbol Timer	0x40003000	Serves as the control register. Bit 0: 0 – Symbol Timer Off 1 – Symbol Timer On	Read/Write
	0x40003004	Unused	Read/Write
	0x40003008	Unused	Read/Write
	0x4000300C	Unused	Read/Write

Transmitter Control	0x40004000	Serves as the waveform control register. Bits 0 and 1: 00 – Beacon 01 – Telecommand 10 – Telemetry 11 - FHSS	Read/Write
	0x40004004	Serves as the mute control register. Bits 0: 0 – Transmitter not muted 1 – Transmitter muted	Read/Write
	0x40004008	Unused	Read/Write
	0x4000400C	Unused	Read/Write

Appendix 7: FMCOMMS2/3/4 Block Diagram

This block diagram demonstrates the basic internal architecture of the AD9361. It shows the transmit and receive data paths. During this project, only the transmit path was modified. Any IP cores associated with only the receive path remained unmodified.

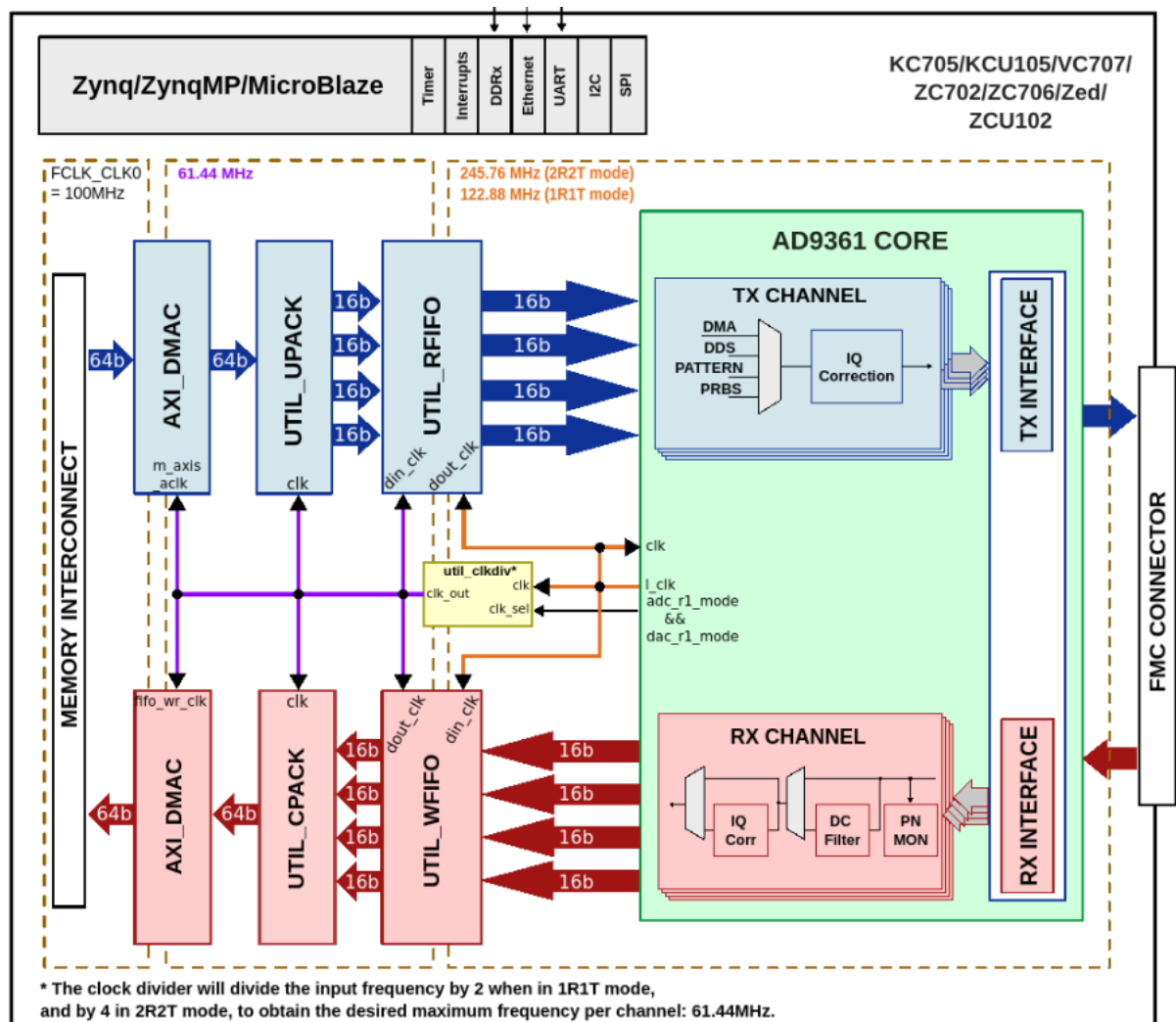


Figure 74: FMCOMMS2/3/4 Block Diagram (Analog Devices 2026a)

Appendix 8: GNU Radio Receiver

Below is the block diagram for the receiver implemented in GNU Radio. It was used to provide constellation plots for both BPSK and QPSK waveforms.

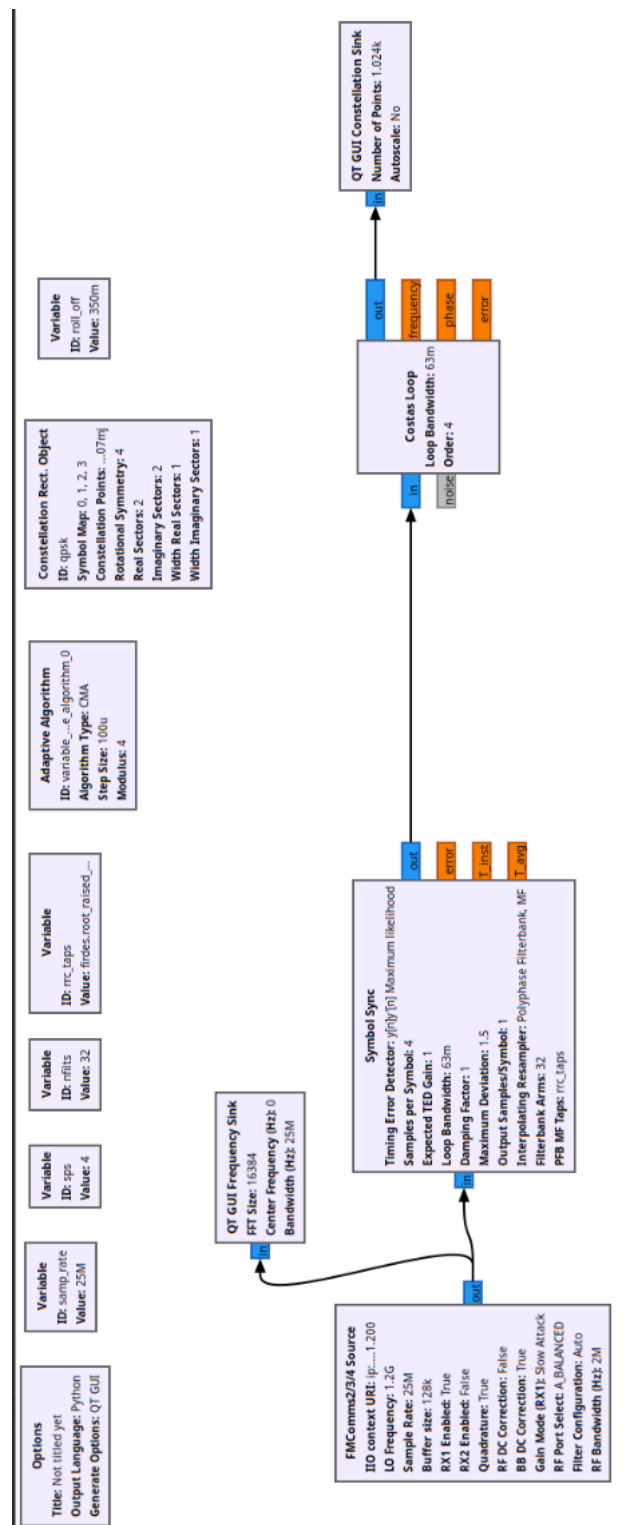


Figure 75: GNU Radio Receiver (GNU Radio Foundation 2023)

Appendix 9: Digital Evidence in GitHub

Git was used throughout the project to provide change control. The main branch contains the most up-to-date code used in this project, however branches containing earlier versions of the code have been retained. This is to demonstrate the evolution of the project.

The GitHub repository can be accessed using the following URL:

https://github.com/VClark24/Project_Transmitter/tree/main